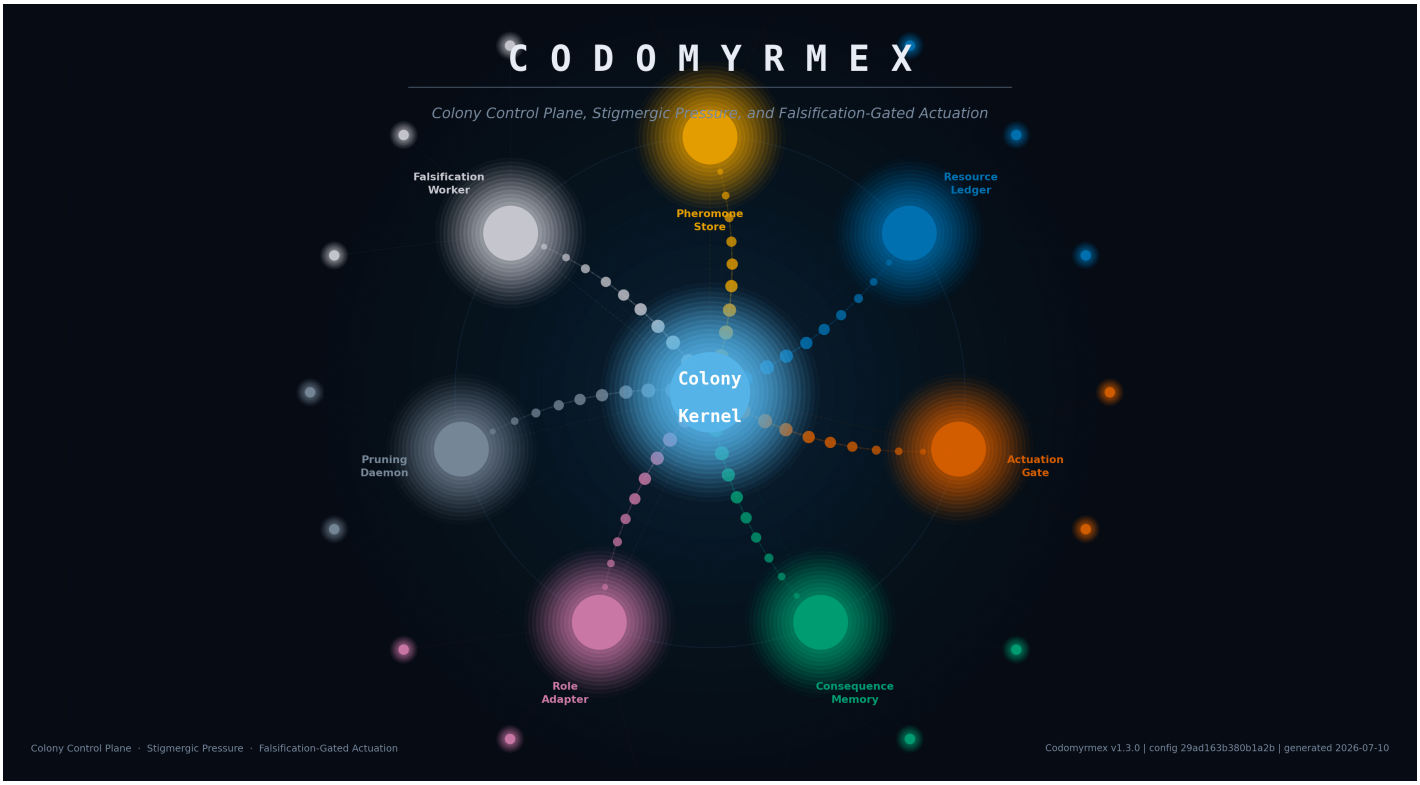

Codomyrmex: An Artificial Ecology for Agentic Software Development

Colony Control Plane, Stigmergic Pressure, and Falsification-Gated Actuation

Daniel Ari Friedman

Date: July 10, 2026

ORCID: 0000-0001-6232-9096

DOI: forthcoming

Repository: github.com/docxology/codomyrmex

Contents

Abstract

The central claim of this paper is bounded and testable: after a failed action, matching future proposals should become measurably harder to pass under the colony gate. Each rejected tool call, failed gate, or contradicted consequence is written into a shared consequence memory that reshapes trust weights, role assignments, and resource budgets for subsequent proposals. The mechanism that produces this pressure is stigmergy: rather than maintaining ephemeral, per-session agent state, the framework encodes learned caution directly into a persistent pheromone field that survives model swaps, session boundaries, and agent population changes. The colony does not converge toward a fixed policy; it adapts its recorded pressure profile to the problem surface.

This paper presents Codomyrmex (v1.3.0), an agentic software-development framework that instantiates this insight as a closed feedback loop governed by the Colony Control Plane. Agents accumulate consequence histories, receive deterministic role reassignment, and are constrained by recorded consequence; stale or duplicate module locations may be flagged for human-reviewed pruning under configured pressure signals.

The architectural centerpiece is the Colony Control Plane, comprising 8 subsystems: (1) *pheromone gradients* — ephemeral stigmergic signals implementing stigmergy (indirect coordination through environment-mediated signals, Grassé 1959) that bias agent attention toward high-yield paths without explicit coordination; (2) *resource budget* — per-agent and per-colony token, tool-call, and wall-time envelopes enforced before dispatch; (3) *actuation gate* — a weighted additive score with hard overrides ($\tau_{\text{gate}} = 0.75$) that blocks proposals lacking sufficient evidence mass; (4) *consequence memory* — a persistent, append-only ledger of action outcomes indexed by agent, role, and context hash; (5) *role adaptation* — deterministic reassignment of the 5 canonical roles (`SANDBOX`, `REPAIR_ANT`, `MEMORY_ANT`, `DISPATCHER`, `GUARD_ANT`) based on trust and proposal count; (6) *pruning daemon* — read-only nomination of stale, dormant, or duplicate module locations for human or `GUARD_ANT` review; (7) *falsification worker* — deterministic adversarial checks that attempt to invalidate a proposal before any gate decision is finalised; and (8) *ColonyKernel* — the hub class coordinating the other subsystems as a single re-entrant transaction.

These subsystems implement a closed feedback loop: environmental pressure generates proposals; proposals pass (or are blocked by) the actuation gate; surviving actions produce consequences; consequences are written to memory; memory drives role reassignment; roles reshape the pressure distribution. The loop repeats with a colony that is less vulnerable to repeated context-reset and local-failure deception patterns because memory is retained and gate thresholds tighten on repeated failure patterns. 8 MCP tools expose the Control Plane to external orchestrators, and 6 typed inter-agent signal channels — `FAILURE`, `SUCCESS`, `RISK`, `NEED`, `DEPENDENCY`, and `HUMAN_PRIORITY` — propagate state changes without shared mutable state.

The framework ships 130 top-level submodules covering the colony kernel, MCP surface, agent lifecycle, signal bus, role engine, gate logic, consequence ledger, and CLI harness. Beyond the Colony Control Plane, the framework integrates a secure cognitive layer: self-custody wallet operations authenticated via zero-knowledge proofs that verify ownership without exposing private keys, biocognitive identity verification using heartbeat interval variability (RMSSD/SDNN) and EEG frequency-band analysis, and a PAI bridge that exposes all 15 secure cognitive MCP tools across identity, wallet, defense, market, and privacy modules to external orchestrators. The colony-kernel manuscript surface enforces the no-mock contract throughout: 764 tests, 78.4% branch coverage, 0 ruff errors, and 0 ty type errors — all figures drawn from a single authoritative `pytest --cov --cov-branch` invocation at compose time. Behaviorally, deterministic contract tests confirm that the actuation gate rejects 67% of low-evidence proposals in the controlled gate fixture, and the trust trajectory reaches its first non-sandbox role after 3 clean feedback cycles under the configured role ladder. Numeric claims in this manuscript are hydrated from generated artifacts at compose time so reviewer-sensitive figures remain tied to the artifact that produced them.

Keywords: ai-agents, model-context-protocol, mcp, multi-agent, orchestration, colony-control-plane, stigmergy, artificial-ecology, agentic-software-engineering, falsification-worker, actuation-gate, trust-scoring

Corresponding author: Daniel Ari Friedman

1 Introduction

When an AI agent causes a five-hour repair cycle, the default agentic framework starts the next invocation with identical permissions and no memory of the damage. The agent disappears back into the tool pool as if Tuesday never happened. Human engineers face a chain of social accountability — code review, standup, retrospective, trust erosion — but the default architecture provides no analogous mechanism for agents. Software engineering at scale has always been a coordination problem; classical multi-agent systems research already treated autonomy, social ability, reactivity, and proactiveness as engineering properties rather than personality traits (Wooldridge and Jennings 1995). The emergence of LLM-based coding agents sharpens that old problem into something operationally new: agents now navigate repositories, edit files, run tests, and call tools through software interfaces whose design materially changes task success (Yang et al. 2024b, 2024a; Garg et al. 2025). Recent agent surveys converge on the same decomposition: LLM agents are not just text generators but systems organized around perception, memory, planning, tool use, and action loops (L. Wang et al. 2023; Xi et al. 2023).

Tool-use scholarship makes the risk concrete. WebGPT, MRKL systems, ReAct, Toolformer, ToolLLM, Gorilla, and API-Bank all show that language models become more capable when they browse, retrieve, call APIs, learn tool-call patterns, or interleave reasoning with action against an external environment (Nakano et al. 2021; Karpas et al. 2022; Yao, Zhao, et al. 2023; Schick et al. 2023; Qin et al. 2023; Patil et al. 2023; M. Li et al. 2023). Planning and deliberation work sharpens the same point: chain-of-thought, self-consistency, and Tree of Thoughts increase capability by making intermediate reasoning, search, and branch selection explicit rather than leaving every decision to a single greedy continuation (Wei et al. 2022; X. Wang et al. 2023; Yao, Yu, et al. 2023). Reflexion, Generative Agents, and MemGPT add the

temporal dimension: language agents can preserve feedback, memories, reflections, and long-horizon context across interactions without changing the base model weights (Shinn et al. 2023; Park et al. 2023; Packer et al. 2023). That capability is precisely why actuation needs a control plane. A tool-using model that can retrieve, plan, remember, edit, and execute is no longer merely producing text; it is changing persistent state.

Agent-governance and agent-safety work therefore emphasize constrained action spaces, legible activity, monitoring, prompt-injection resistance, and risk evaluation for high-stakes tool calls (Shavit et al. 2023; Greshake et al. 2023; Debenedetti et al. 2024; Zhan et al. 2024; Ruan et al. 2023). Recent safety benchmarks broaden that pressure: Agent-SafetyBench, Agent Security Bench, RAS-Eval, SafeToolBench, PrivacyLens, and memory-poisoning studies all treat unsafe tool use, trajectory-level privacy leakage, attack surfaces, and persistent memory as first-class evaluation objects rather than incidental prompting failures (Z. Zhang et al. 2024; H. Zhang et al. 2024; Fu et al. 2025; Xia et al. 2025; Shao et al. 2024; Dash et al. 2026). Codomyrmex begins from the same premise but narrows it to one falsifiable engineering question: before an agent acts on a repository, has it earned the authority to do so?

1.1 The Fragility of the Warehouse Model

Conventional agentic frameworks treat the module ecosystem as a warehouse: tools sit on shelves, any agent can reach for any tool at any time, and the only record of past activity is whatever the tool's return value happened to contain. Contemporary orchestration frameworks and multi-agent software-engineering systems are increasingly good at routing messages, maintaining task state, encoding roles, using standard operating procedures, and coordinating communicative agents (AI 2024; Wu et al. 2023; Inc. 2024; G. Li et al. 2023; Qian et al. 2024; Hong et al. 2023). Those are runtime capabilities, not consequence systems. There is no memory of past failures, no adaptation in response to outcomes, and no consequence for bad actions. An agent that caused a five-hour repair cycle last Tuesday returns on Wednesday with the same gate score, the same trust level, and the same access envelope. The system has no immune response. It cannot distinguish a proven collaborator from a first-time actor, or a safe target location from one heavy with unresolved failure signals. Every actuation event is a fresh start, and fresh starts accumulate technical debt the same way unsupervised processes accumulate entropy.

This fragility is not a niche concern. As agent populations grow and actuation rates increase, the statistical probability that some agent proposes some harmful action within some window approaches certainty. Long-horizon evaluation environments such as AgentBench, WebArena, OSWorld, GAIA, AppWorld, and TheAgentCompany all move agent evaluation toward persistent state, tools, apps, computer interfaces, and workplace-like tasks (Liu et al. 2023; Zhou et al. 2023; Xie et al. 2024; Mialon et al. 2023; Trivedi et al. 2024; Xu et al. 2025). Harmful-agent and cyber-agent benchmarks make the same point from the adversarial side: once agents can operate tools over multiple steps, the evaluation target becomes whether they should act, not only whether they can act (Andriushchenko et al. 2025; Levy et al. 2024; Wan et al. 2024; Fang et al. 2024; Zhu et al. 2024, 2025; Motwani et al. 2024). A warehouse that cannot learn from those events will repeat them.

1.2 The Ecology Thesis

Codomyrmex takes a different premise. Rather than treating the agent collective as a pool of interchangeable workers drawing from a static toolbox, it models the collective as a colony. The metaphor is not decorative; it imports three research traditions into the software-agent setting. The first is stigmergy and self-organization: Grassé's original account of environment-mediated coordination, Bonabeau et al.'s synthesis of swarm intelligence, Parunak's digital pheromones for distributed software agents, Dorigo and Stützle's ant-colony optimization dynamics, and Kauffman's broader account of self-organizing order (Grassé 1959; Bonabeau et al. 1999; Parunak 1997; Dorigo and Stützle 2004a; Kauffman 1993). The second is computational trust: formal trust as an operative quantity, trust and reputation updates from observed interaction, peer-to-peer reputation algorithms, stereotype/referral bootstrapping, and hybrid models that combine direct experience with role or certified evidence (Marsh 1994; Sabater and Sierra 2005; Kamvar et al. 2003; Burnett et al. 2013; Huynh et al. 2006). The third is externalized feedback and alignment: reinforcement learning, RLHF, Constitutional AI, and process reward models all distinguish action selection from downstream feedback, even though they usually internalize that feedback into policies or weights rather than keeping it as an auditable gate ledger (Sutton and Barto 2018; Christiano et al. 2017; Bai et al. 2022; Lightman et al. 2023; Uesato et al. 2022). Codomyrmex combines those lines: modules are not passive tools waiting to be invoked; they are organisms with assigned roles, earned trust scores, and finite lifespans determined by whether their continued presence serves the colony's objectives. Agents are not fungible workers; they are actors whose behavioral history is encoded in a persistent trust profile that gates what they are permitted to propose next.

The colony accumulates memory of what works. A sequence of successful outcomes by an agent deposits SUCCESS signals in the pheromone field at the locations it touched. A failure deposits FAILURE signals, and repeated local risk pressure makes the gate stricter for the next agent that proposes work at the same location. An agent whose recent history contains repeated repair cycles sees its trust score drift downward and its effective role narrow toward lower-privilege tiers. An agent that consistently delivers clean outcomes earns deterministic promotion toward higher-privilege roles. The ecology metaphor is useful because outcomes feed directly into the state that determines future actuation access.

The design principle is stated once in the project specification and should be restated here verbatim because it is load-bearing: "Codomyrmex should not measure itself by how many ants it has, but by whether the colony gets harder to fool after every failed action." This is the falsification criterion for the entire architecture. An agentic system that does not become measurably harder to fool over time — regardless of how many tools it exposes or how many MCP endpoints it registers — has not solved the coordination problem; it has scaled the warehouse model.

1.3 Architecture Overview

The system is organized in three tiers. The base tier is the `src/codomyrmex/` Python package, a library of modules spanning infrastructure, agents, cloud, coding, security, search, memory, and ML primitives. Each module is independently versioned, zero-mock tested, and documented under the RASP pattern — four collocated files per module: README, AGENTS, SPEC, and PAI. The second tier is the Colony Control Plane, a dedicated subsystem within `src/codomyrmex/colony_kernel/` that acts as the control plane governing how agents interact with those modules. The third tier is the MCP surface, a set of eight production `@mcp_tool`-decorated endpoints that expose the Colony Control Plane to any Model Context Protocol client, including Claude, GPT-class models, and the PAI system, using the Model Context Protocol as the tool-facing integration boundary (Anthropic 2024).

1.4 The Colony Control Plane: Eight Subsystems

The Colony Control Plane is composed of eight subsystems, each a self-contained class that shares only the `models.py` contract and carries no imports from the others:

PheromoneStore wraps the stigmergic `TraceField` with colony-specific `ColonySignal` semantics, applying source trust multipliers on deposit and evaporating traces on each kernel tick. ResourceLedger maintains a period-scoped, multi-dimensional budget tracker that checks seven cost dimensions — including LLM call count, runtime seconds, risk level, human attention minutes, merge risk, documentation debt, and security exposure — before any actuation request is approved. ActuationGate combines budget headroom, pheromone pressure, trust tier, and proposal completeness into a weighted additive score, with hard overrides for budget exhaustion, SANDBOX role, low trust, and CRITICAL falsification findings. ConsequenceMemory maintains a SQLite-backed log of every proposal outcome, computing trust deltas and persisting per-agent `AgentTrustProfile` records across sessions. RoleAdapter deterministically infers an agent's `AgentRole` from its trust score and proposal history, mapping the colony's five roles (SANDBOX, REPAIR_ANT, MEMORY_ANT, DISPATCHER, GUARD_ANT) without side effects. PruningDaemon scans the pheromone field for locations with stale or absent DEPENDENCY signals and produces `PruningCandidate` reports for human or GUARD_ANT review, implementing the colony's mechanism for self-contraction. FalsificationWorker runs 10 deterministic adversarial checks against every proposal before the gate evaluates it, functioning as an internal red-team pass that the gate uses to inform its completeness score and hard-refuse path. ColonyKernel is the top-level integration class that owns subsystem lifecycle and exposes the four-method public API (`propose_action`, `record_outcome`, `colony_status`, `tick`).

1.5 Gate Scoring

The ActuationGate renders its verdict through a weighted linear combination of four normalized component scores. Each component maps a colony-observable quantity to the interval $[0, 1]$: `budget_ok` reflects whether the colony's multi-dimensional resource envelope can absorb the proposed cost; `risk_ok` encodes the inverse of pheromone pressure at the target location; `trust_ok` is the proposing agent's gate trust tier; and `completeness` measures the evidential quality of the proposal (tests referenced, rollback plan quality, falsification vector coverage). Section 2 derives the gate scoring formula; resource and risk signals carry the highest weight (0.3 each), reflecting the colony's priority ordering: these signals encode objective, real-time constraints — budget headroom and accumulated failure pressure at the target location — whereas trust is an earned prior and completeness is partially gameable. The EXECUTE threshold is 0.75; HOLD begins at 0.5; REFUSE applies below that hold threshold. Hard-override rules supersede the numeric score: SANDBOX role always receives REFUSE, trust below 0.3 refuses before scoring, CRITICAL falsification findings refuse before scoring, and budget failure is mode-dependent — standalone gate checks REFUSE immediately, while kernel-supplied budget exhaustion returns HOLD so the action can be requeued after the budget period resets.

This is not a complete security boundary, but it follows a least-authority and zero-trust posture: authority is earned incrementally, and each destructive proposal is re-evaluated rather than inheriting trust from identity or prior access (Saltzer and Schroeder 1975; Miller and Shapiro 2003; Rose et al. 2020). Human-automation research supplies a parallel warning: the design problem is not "more autonomy" in the abstract, but choosing automation levels, preserving operator awareness, and calibrating trust so reliance tracks demonstrated capability (Parasuraman et al. 2000; Lee and See 2004; Endsley 2017). AI safety work frames the same failure mode as accident risk: side effects, reward hacking, distribution shift, unsafe exploration, and weak oversight become engineering problems once systems act in open environments (Amodei et al. 2016). The same boundary appears in governance and threat frameworks. AI risk-management guidance, generative-AI profiles, OWASP LLM/agentic guidance, MITRE ATLAS, AI-control work, and runtime-assurance/shielding literature all argue for explicit controls between high-capability systems and consequential action (Tabassi 2023; Autio et al. 2024; OWASP Foundation 2025, 2026b, 2026a; MITRE Corporation 2026a; Greenblatt et al. 2023; Seto et al. 1998; Alshiekh et al. 2018). Codomyrmex contributes one such control point for software actuation, not a proof that the surrounding system is secure.

1.6 Relationship to Template Infrastructure

Codomyrmex is developed and published within the template research infrastructure described in the repository's top-level CLAUDE.md and AGENTS.md. That infrastructure provides the RASP documentation standard (which governs every module's README, AGENTS, SPEC, and PAI files), a strict zero-mock testing policy (all 35,119 collected tests use real data and computations; `unittest.mock`, `MagicMock`, and `pytest-mock` are prohibited), the three-tree mirror invariant (every `src/codomyrmex/<module>` has sibling trees in `tests/unit/<module>/` and documentation), and a multi-stage pipeline with test gating that enforces a 40% coverage floor before any PDF artifact is rendered. These constraints are not incidental; they are load-bearing for the colony thesis. Reproducible-research, assurance-case, model-reporting, dataset-documentation, AI-service FactSheet, internal audit, and algorithmic-impact-assessment

scholarship make the same point from the evidence side: claims about computational systems need explicit artifacts, auditable context, disclosed limits, and a route for revising the claim when evidence changes (Peng 2011; Buhl et al. 2024; Hawkins et al. 2021; Mitchell et al. 2019; Gebru et al. 2021; Arnold et al. 2019; Raji et al. 2020; Reisman et al. 2018). Secure-development and supply-chain work adds the release boundary: SSDF, SLSA, Sigstore, in-toto, TUF, agent-identity work, visibility infrastructure, and legal-governance scholarship all treat identity, provenance, logs, attestations, and accountable action records as part of the system rather than as optional documentation (Souppaya et al. 2022; Open Source Security Foundation 2026; Newman et al. 2022; Torres-Arias et al. 2019; Samuel et al. 2010; NIST National Cybersecurity Center of Excellence 2026; Chan et al. 2024, 2025; Kolt 2025). A system that claims to make the codebase harder to fool must itself be built on a foundation that cannot silently degrade.

1.7 Why Stigmergy

The design metaphor deserves explicit defense. Ants do not coordinate by negotiating with each other. They deposit chemical traces — pheromones — into the shared environment, and other ants sense those traces and modulate their behavior accordingly. The colony's collective intelligence emerges not from any central authority or inter-agent communication protocol but from the accumulated pattern of signals in the environment. A trail that leads to food gets reinforced by returning foragers; a trail that leads to danger gets suppressed by the absence of positive reinforcement and the presence of alarm signals.

There is a precise information-theoretic reason to prefer this design over direct agent-to-agent communication. If n agents must coordinate pairwise — each informing all others of its state — the number of communication channels required scales as $O(n^2)$. Stigmergy collapses this to $O(n)$: each agent reads the shared field and writes to the shared field, and the field integrates all historical signals into a single queryable surface. For an agentic system operating at machine speed, the difference between $O(n^2)$ and $O(n)$ coordination overhead is not a theoretical nicety — it is the boundary between an architecture that saturates under moderate agent populations and one that scales linearly with them. The pheromone field is the compression: instead of propagating failure events as messages to every peer, the colony encodes the event as a field perturbation that any future agent consulting that location will encounter automatically, without any agent having transmitted it.

The Colony Kernel uses the same principle. Agents do not negotiate permissions with a central authority, and they do not communicate with each other to coordinate actuation decisions. They deposit signals into the pheromone field — SUCCESS when outcomes are clean, FAILURE when repair is required, RISK when the FalsificationWorker flags adversarial vectors — and the ActuationGate reads those signals when evaluating the next proposal at the same location. An agent does not need to know that another agent failed at `codomyrmex.git_operations.core` last week; it only needs the gate to report elevated FAILURE pressure at that location and apply the corresponding score penalty. The ecology's memory is stored in the field, not in any individual agent's context window. This architectural choice has a practical consequence: the colony's learned caution about dangerous locations persists across session boundaries, model swaps, and agent population changes, because it is encoded in the pheromone store rather than in any ephemeral state that disappears when a session ends.

The "harder to fool" property that the project specification names as its falsification criterion is precisely what stigmergy makes structurally available: each failure event is inscribed in the field at the location where the failure occurred, raising the gate's caution for the next proposal at that location. The colony is less vulnerable to context-reset deception by fresh agents, because relevant local failure history is stored in the field rather than only in the agent context. Whether this is the right metaphor or merely a productive one remains an empirical question beyond this artifact; the results reported here validate the implementation contracts that make the property measurable.

1.8 Reader's Guide to Sections

The remainder of this manuscript is organized as follows. Section 2 presents the Colony Kernel design in full: the stigmergic pheromone protocol, the gate scoring model, the trust lifecycle, the falsification worker algorithm, and the pruning daemon. Section 3 develops the mathematical foundations: formal definitions of the pheromone field as a vector space, proofs of decay monotonicity and field boundedness, the gate score's decision-theoretic interpretation, and the trust delta as a stochastic approximation process with provable convergence properties. Section 4 reports implementation measurements: gate decision fixtures across agent role tiers, trust score trajectories under deterministic clean outcomes, pheromone field decay under configured rates, and analytical contract derivations with complete edge-case analysis. Section 6 documents the configuration parameters and colony initialization procedures required to reproduce the checked-in protocol. Section 7 describes the provenance chain from source commit through test gate to rendered PDF, including the steganographic verification layer. Section 8 surveys related work in multi-agent coordination, stigmergy-inspired computing, trust-based access control, information-theoretic security, decision theory, and mechanism design for autonomous systems, situating Codomyrmex within and against those traditions. Section 5 synthesizes the implementation findings against the falsification criterion stated in Section 1, presents formal falsification criteria as mathematical statements, characterizes the precise conditions under which the colony fails to get harder to fool, and specifies an empirical benchmark agenda. Section 9 provides a complete active inference treatment of the Colony Kernel, mapping its components to the Free Energy Principle framework and identifying both structural correspondences and divergences from the canonical Bayesian brain. Section 10 in the appendix documents the explicit design decisions behind the architecture, presenting the alternatives considered and the reasoning behind each choice made.

2 Methodology

2.1 Colony Kernel Architecture

2.1.1 Overview of the 8 Subsystems

The Colony Control Plane is realised as a single Python package (`codomyrmex.colony_kernel`) with shared value objects and enumerations in `models.py`, canonical subsystem implementations in standalone modules, and the `ColonyKernel` integration class orchestrating their lifecycle. The design keeps communication explicit: subsystems exchange typed models, while the kernel owns cross-subsystem sequencing and state transitions.

Table 1 enumerates the eight subsystem roles used by the control plane.

Table 1: Colony Control Plane subsystem overview.

Subsystem	Primary Module	Responsibility
PheromoneStore	`colony_kernel.pheromone_store.PheromoneStore`	Wraps `TraceField` with `ColonySignal` semantics; handles deposit, reinforcement, and decay of stigmergic traces across 6 signal types.
ResourceLedger	`colony_kernel.resource_ledger.ResourceLedger`	Tracks accumulated resource consumption against a period-scoped `ResourceBudget` across all 7 cost dimensions; `can_afford` returns a boolean used as a gate pre-condition.
ActuationGate	`colony_kernel.actuation_gate.ActuationGate`	Combines resource headroom, pheromone pressure, agent trust, and proposal completeness into a numeric gate score, then routes the score to EXECUTE / HOLD / REFUSE.
ConsequenceMemory	`colony_kernel.consequence_memory.ConsequenceMemory`	Persists `ConsequenceRecord` rows in SQLite WAL-mode; maintains per-agent `AgentTrustProfile`, computes trust deltas, and exposes `recent_failures()` for gate coupling.
RoleAdapter	`colony_kernel.role_adapter.RoleAdapter`	Deterministically infers `AgentRole` from a trust profile's `trust_score` and `total_proposals`; updates the role in-place without side effects.
PruningDaemon	`colony_kernel.pruning_daemon.PruningDaemon`	Identifies stale or duplicate modules via call-count and pheromone-pressure analysis; raises `PruningCandidate` reports for human or `GUARD_ANT` review — never deletes.
FalsificationWorker	`colony_kernel.falsification_worker.FalsificationWorker`	Applies 10 adversarial attack vectors against every `ActionProposal`; returns severity-ranked `FalsificationFinding` records and an overall `FalsificationReport` verdict.
ColonyKernel	`colony_kernel.kernel.ColonyKernel`	Top-level integration class; owns the lifecycle of every subsystem and exposes the four public API methods: `propose_action`, `record_outcome`, `colony_status`, and `tick`.

The integration entry point is `ColonyKernel`, which is instantiated once per process. All subsystem state is owned by the kernel; callers interact exclusively through its four public methods.

Figure 1 illustrates the control-plane topology: `ColonyKernel` at the centre, each of the seven operational subsystem classes at the leaves, all sharing the `models.py` value-object contract.

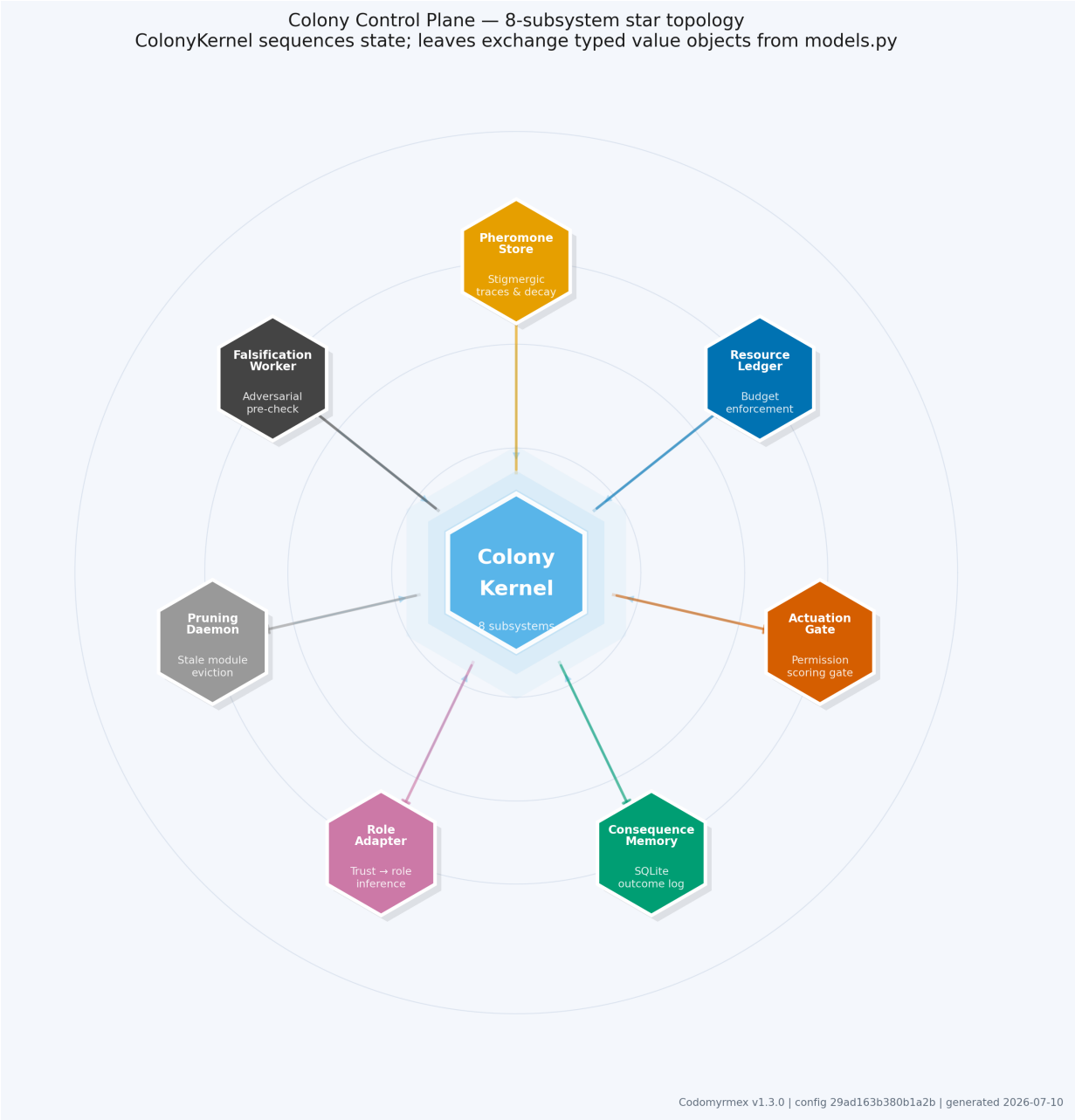

Figure 1: Colony Control Plane topology. `ColonyKernel` owns subsystem lifecycle and sequencing; the seven operational leaf nodes exchange typed value objects from the shared `models.py` contract. Coloured by functional role: orange=stigmergy, blue=resource, red=gate, green=memory, pink=roles, grey=pruning, black=adversarial review.

2.2 Pheromone Gradients (PheromoneStore)

The `PheromoneStore` encapsulates the colony's environmental memory in the tradition of stigmergic coordination (Grassé 1959; Dorigo and Stützle 2004b). Rather than communicating through direct agent-to-agent messages, the colony writes persistent chemical-analogue traces into a shared `TraceField` (from `codomyrmex.agentic_memory.stigmergy`) and reads them back on every gate evaluation. The field implements a continuous evaporation model inspired by variational principles of free-energy minimisation (Friston 2010): traces that are not reinforced decay to zero over time, causing the field to reflect the colony's recent experience rather than its entire history.

Signal types. The store recognises 6 signal types (`SignalType` enum), each with distinct ecological meaning:

Table 2 lists the signal classes and their effect on the gate.

Table 2: Pheromone signal types and gate effects.

Signal Type	Ecological Analogue	Decay Class	Effect on Gate
`FAILURE`	Trail avoidance pheromone	`FAST`	Records failed outcomes in the field and is exposed in gate witness state; trust penalties carry repeated failures into scoring.
`SUCCESS`	Trail amplification pheromone	`SLOW`	Reinforced on passing outcomes and retained as positive local history; it does not reduce `risk_ok` directly.
`RISK`	Caution marker	`FAST`	Queried when computing `risk_pressure` at the target location.
`NEED`	Resource request	`NORMAL`	Broadcast by agents requiring attention at a location; does not feed gate pressure directly.
`DEPENDENCY`	Usage trace	`SLOW`	Deposited by `record_outcome` to signal active module consumption; a strength ≥ 2.0 vetoes `PruningDaemon` nomination.
`HUMAN_PRIORITY`	Operator-injected signal	`SLOW`	Highest trust weight (source multiplier ×2.0); long per-tick retention (≈98%) provides persistent operator overrides across many actuation cycles.

Decay rates. Each signal is assigned one of 3 decay rate classes (`DecayRate` enum). The enum value is a *multiplier* (m_k) applied to the base evaporation rate $\lambda_0 = 0.1$/tick (constant `_BASE_EVAPORATION` in `pheromone_store.py`); the per-tick strength retention fraction for class k is $e^{-\lambda_0 \cdot m_k}$. The three classes and their exact multiplier values (from the `DecayRate` enum) are:

Table 3 records the multiplier, retention, and half-life for each decay class.

Table 3: Pheromone decay classes and retention constants.

Class	Multiplier m_k	Effective rate $\lambda_0 \cdot m_k$	Per-tick retention	Half-life $t_{1/2}$	Intended Lifetime
`FAST`	3.0	0.30/tick	$e^{-0.30} \approx 74\%$	$\ln 2/0.30 \approx 2.31$ ticks	Urgent, transient events (failure alerts, risk warnings).
`NORMAL`	1.0	0.10/tick	$e^{-0.10} \approx 90\%$	$\ln 2/0.10 \approx 6.93$ ticks	Standard coordination signals.
`SLOW`	0.2	0.02/tick	$e^{-0.02} \approx 98\%$	$\ln 2/0.02 \approx 34.66$ ticks	Structural, persistent markers (human priorities, dependency traces).

The half-life formula $t_{1/2} = \ln 2/\lambda$ (where $\lambda = \lambda_0 \cdot m_k$) follows from the exponential decay model $s(t) = s_0 \cdot e^{-\lambda t}$: setting $s(t_{1/2}) = s_0/2$ and solving gives $t_{1/2} = \ln 2/\lambda$. At the base rate ($\lambda_0 = 0.1$/tick), FAST signals halve in roughly 2.3 ticks, NORMAL signals in roughly 6.9 ticks, and SLOW signals persist for over 34 ticks at half strength. This fifteen-fold decay-rate gap between FAST and SLOW is intentional: FAST-class signals (FAILURE, RISK) fade before stale alerts suppress legitimate proposals; SLOW-class signals (HUMAN_PRIORITY, SUCCESS, DEPENDENCY) provide structural colony memory across many actuation cycles.

Trust multipliers by source. Not all sources carry equal epistemic weight. The `PheromoneStore` scales a signal's effective initial strength by the depositing source's trust multiplier before writing to the `TraceField`:

The effective-strength calculation in Equation 1 determines the initial trace written to the field.

$$\text{effective\_strength} = \text{signal.strength} \times \text{source\_multiplier} \times \text{trust\_factor} \tag{1}$$

where `source_multiplier` is HUMAN×2.0, TEST×1.5, SECURITY×1.3, and AGENT or RUNTIME×1.0, and `trust_factor` is the depositing agent's current trust score (passed by the kernel at deposition time). This formulation means that a human-injected signal of nominal strength 1.0 deposits as 2.0, whereas an untrusted new agent depositing the same signal would produce a much weaker trace.

Compound key addressing. Each pheromone occupies a unique position in the `TraceField` identified by the compound key `"{location}:{signal_type.value}"`. This allows FAILURE and SUCCESS signals at the same module path to be tracked and decayed independently, preserving the sign and type of the colony's accumulated evidence.

Figure 2 shows the three decay curves across 10 ticks. FAST signals (FAILURE, RISK) drop to 50% strength after approximately 2.31 ticks at the base rate ($\lambda_0 = 0.1$/tick). The effective decay-rate ratio between FAST ($\lambda = 0.30$) and SLOW ($\lambda = 0.02$) is 15:1, so SLOW-class signals such as HUMAN_PRIORITY and SUCCESS provide structural colony memory across many actuation cycles, while FAST-class signals carry only the most recent environmental warnings.

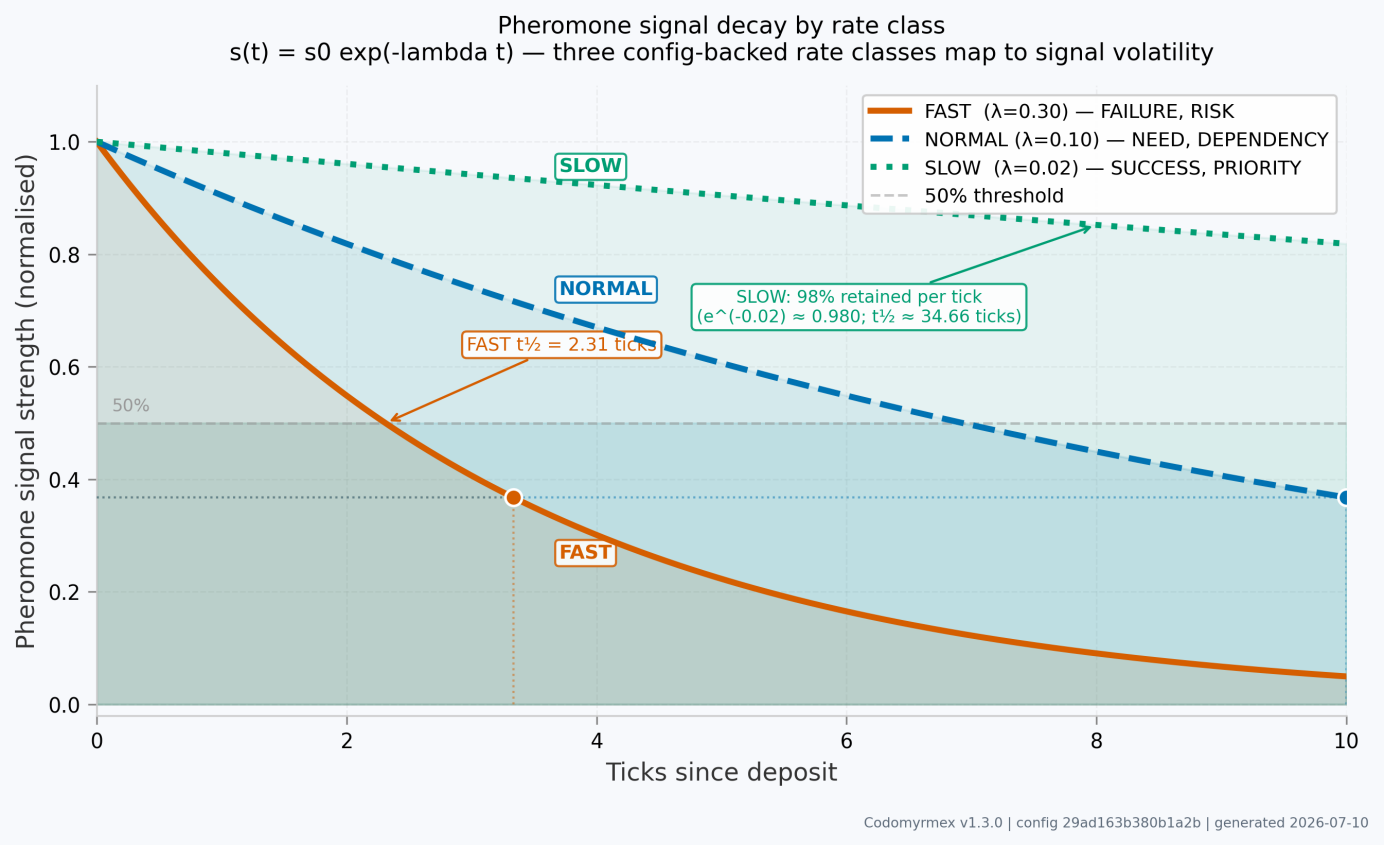

Figure 2: Pheromone signal decay by rate class. Strength follows $s(t) = s_0 \cdot e^{-\lambda t}$ where $\lambda \in \{0.30, 0.10, 0.02\}$ for FAST, NORMAL, and SLOW classes respectively (base rate $\lambda_0 = 0.1$/tick multiplied by class multipliers 3.0, 1.0, 0.2). The dashed grey line marks the 50% strength threshold. The FAST half-life ($t_{1/2} \approx 2.31$ ticks) is annotated on the vertical dotted line.

2.3 Resource Budget (ResourceLedger)

The `ResourceLedger` enforces a multi-dimensional budget envelope over the colony's resource consumption (Bonabeau et al. 1999). Rather than tracking a single scalar cost, it maintains 7 independent dimensions of `ResourceCost`:

Table 4 lists the dimensions enforced by the ledger.

Table 4: Resource budget dimensions enforced by `ResourceLedger`.

Dimension	Type	Description
`llm_calls`	`int`	Number of LLM API invocations.
`runtime_seconds`	`float`	Wall-clock execution time.
`risk_level`	`float` $\in$ `[0,1]`	Aggregate risk fraction of the action.
`human_attention_minutes`	`float`	Estimated operator review time.
`merge_risk`	`float` $\in$ `[0,1]`	Probability of merge conflicts or integration failures.
`doc_debt`	`float`	Documentation gap accumulation score.
`security_exposure`	`float` $\in$ `[0,1]`	Estimated security surface increase.

The ledger's `can_afford` method returns a boolean: it checks whether accumulating the proposed cost on top of current-period usage would breach any single dimension ceiling. A `False` return bypasses ordinary scoring (`gate_score = 0.0`) because budget failure is treated as a disqualifying condition, not a scoring penalty. The final decision depends on the gate calling mode. In standalone mode, where `ActuationGate.evaluate(proposal, profile)` performs its own budget check, the budget failure returns `REFUSE`. In kernel mode, where `ColonyKernel` supplies `budget_approved=False` after its own pre-check, the same budget failure returns `HOLD` so the action can be requeued after the budget period resets. When `can_afford` returns `True`, `budget_ok = 1.0` and the full 0.30 weight contributes to the gate score. The `consume` method is called separately after an action executes, recording actual costs rather than estimates. Budget accumulators auto-reset when the configured `period_seconds` elapses, making the ledger a sliding-window rate limiter over colony activity.

2.4 Actuation Gate

The actuation gate is the colony's central permission layer: it aggregates signals from the resource ledger, pheromone field, agent trust store, and proposal completeness into a scalar gate score, then routes that score to one of three decisions. The gate is the join point where economic constraints, environmental stigmergy, social trust, and epistemic quality are simultaneously expressed as a single admission criterion.

2.4.1 Gate Scoring Formula

The gate score g is a weighted linear combination of four normalised components (Equation 2):

$$g = 0.30 \cdot \text{budget\_ok} + 0.30 \cdot \text{risk\_ok} + 0.25 \cdot \text{trust\_ok} + 0.15 \cdot \text{completeness} \tag{2}$$

clamped to $[0, 1]$ after summation: $g \leftarrow \max(0.0,\ \min(1.0,\ g))$.

The weights sum to 1.0. Each component is described below.

`budget_ok` ($w = 0.30$): Binary resource headroom. `budget_ok = 1.0` when `ResourceLedger.can_afford(proposal.budget_estimate)` returns `True`. A `False` return bypasses all scoring with `gate_score = 0.0`; standalone checks issue `REFUSE`, while kernel-supplied budget failures issue `HOLD` for requeue after reset. A module that has repeatedly triggered budget failures will not accumulate a lower `budget_ok` across proposals — each evaluation is fresh — but the hard-override path means budget-exhausted agents receive no partial credit.

`risk_ok` ($w = 0.30$): RISK pheromone pressure at the proposal target. FAILURE pheromones remain visible in gate witness state and contribute through consequence-memory trust penalties, but the current gate score maps only `risk_pressure` through two module-level constants:

Table 5 defines the discrete mapping from sensed RISK pressure to the normalised score component.

Table 5: RISK pheromone pressure mapping used by the gate.

`risk_pressure`	`risk_ok`
≥ 6.0 (`_HIGH_RISK_THRESHOLD`)	0.0
≥ 3.0 (`_MEDIUM_RISK_THRESHOLD`)	0.5
< 3.0	1.0

A module accumulating repeated RISK signals will see `risk_ok` drop to 0.0, pulling the gate score below EXECUTE or HOLD until pheromones decay or are displaced by later evidence. A clean path with no recent RISK pressure earns `risk_ok = 1.0` and contributes the full 0.30 weight.

trust_ok ($w = 0.25$): Agent trust score normalised to gate range. The gate reads `AgentTrustProfile.trust_score` and maps it as follows:

Table 6 gives the hard floor and two scoring tiers for trust.

Table 6: Trust-score mapping used by the actuation gate.

trust_score	trust_ok
≥ 0.6	1.0
$0.3 \leq$ trust_score < 0.6	0.5
< 0.3	hard REFUSE (early return, `gate_score = 0.0`)

A new agent starting at `trust_score = 0.10` triggers the hard floor and receives REFUSE before scoring. An established agent at `trust_score = 0.65` earns `trust_ok = 1.0`, contributing the full 0.25 weight. An agent at `trust_score = 0.45` earns `trust_ok = 0.5`, contributing 0.125.

Trust penalty. When `ConsequenceMemory` is available and the agent's `recent_fail_count >= 3`, the gate applies the local evaluation penalty in Equation 3:

$$\text{trust\_ok} \leftarrow \max(0.0,\ \text{trust\_ok} - 0.25) \tag{3}$$

This decrement reduces `trust_ok` — the gate's normalised trust contribution for this evaluation only. It does not modify the agent's persistent `trust_score` stored in `ConsequenceMemory`. The agent's durable trust record is updated separately on each `record_outcome` call; the gate penalty is a single-evaluation correction that makes the gate more conservative while an agent is on a losing streak, without permanently penalising the agent's history.

completeness ($w = 0.15$): Evidence mass of the proposal. The gate inspects three fields: `rollback_plan` (non-empty string), `evidence` (non-empty dict), and `expected_outcome` (non-empty string). When all three are present, `completeness = 1.0`. For each missing field:

The proposal-completeness expression in Equation 4 supplies the ordinary score component for incomplete proposals.

$$\text{completeness} = \max(0.0,\ 1.0 - |\text{missing}| \times 0.35) \tag{4}$$

A proposal missing one field scores `completeness = 0.65`; missing two scores `completeness = 0.30`; missing all three scores `completeness = 0.0`.

2.4.2 Decision Thresholds

The gate maps the numeric score to a ternary verdict:

Table 7 gives the three threshold bands.

Table 7: Actuation-gate decision thresholds.

Score Range	Decision	Effect
$g \geq 0.75$	EXECUTE	Proposal proceeds to actuation.
$0.50 \leq g < 0.75$	HOLD	Proposal is requeued; required evidence list returned to agent for revision and resubmission.
$g < 0.50$	REFUSE	Proposal rejected; FAILURE pheromone deposited at target.

2.4.3 Hard Overrides

Four safety conditions bypass the numeric score entirely, evaluated in order before gate scoring begins:

1. Budget failure. A `False` return from `ResourceLedger.can_afford` causes an immediate hard override with `gate_score = 0.0`. Standalone gate evaluation returns `REFUSE`; kernel/caller-supplied budget failure returns `HOLD` for requeue after the budget period resets. No further evaluation occurs.
2. SANDBOX role. Agents with role `SANDBOX` always receive REFUSE regardless of trust score, pheromone state, or proposal quality. `SANDBOX` is the entry role for all new agents and is held until they accumulate at least 3 total proposals and cross the role promotion trust threshold.

3. Trust floor. A `trust_score < 0.3` triggers an early REFUSE with `gate_score = 0.0`. This hard floor ensures that very-low-trust agents cannot reach HOLD or EXECUTE even with perfect scores on the other three components.
4. Critical falsification. Any `CRITICAL` finding from the `FalsificationWorker` triggers an immediate REFUSE with the finding's remediation attached to `GateResult.required_evidence`.

2.4.4 Worked Example

Consider a concrete `ActionProposal` with the following inputs:

- Budget: `ResourceLedger.can_afford` returns `True` → `budget_ok = 1.0`
- Pheromone state: `risk_pressure = 2.0` at the target module path
  - `2.0 < 3.0` → `risk_ok = 1.0`
- Agent trust: `trust_score = 0.55`, no SANDBOX role, no `ConsequenceMemory` reference provided
  - $0.3 \leq 0.55 < 0.6$ → `trust_ok = 0.5`
- Proposal completeness: `rollback_plan` present, `evidence` present, `expected_outcome` absent → 1 missing field
  - `completeness = max(0.0, 1.0 - 1 × 0.35) = 0.65`

Applying the formula:

The base worked example in Equation 5 reaches the EXECUTE band.

$$g = 0.30 \times 1.0 + 0.30 \times 1.0 + 0.25 \times 0.5 + 0.15 \times 0.65 = 0.300 + 0.300 + 0.125 + 0.098 = 0.823 \tag{5}$$

Since $g = 0.823 \geq 0.75$, the gate issues EXECUTE. The missing `expected_outcome` reduced the score by 0.052 but did not prevent execution. If the agent were also on a losing streak (`recent_fail_count >= 3`), Equation 6 shows how the trust penalty reduces `trust_ok` from 0.5 to 0.25:

$$g = 0.300 + 0.300 + 0.25 \times 0.25 + 0.098 = 0.761 \quad \text{(still EXECUTE, closer to threshold)} \tag{6}$$

If risk pheromone pressure were also elevated — say `risk_pressure = 4.5` → `risk_ok = 0.5` — Equation 7 gives the combined effect:

$$g = 0.300 + 0.30 \times 0.5 + 0.25 \times 0.25 + 0.098 = 0.611 \quad \text{(HOLD)} \tag{7}$$

This illustrates how independent pressure from multiple dimensions accumulates in the weighted additive score: no single ordinary component failure blocks execution, but accumulating moderate penalties across several dimensions pulls proposals from EXECUTE into HOLD. Hard overrides still bypass the ordinary score when budget, role, trust-floor, or critical-falsification conditions apply.

2.5 Consequence Memory (SQLite-backed)

The `ConsequenceMemory` subsystem provides durable, persistent storage of every `ConsequenceRecord` — the ground-truth log of what the colony actually did and what happened as a result. It is backed by a SQLite WAL-mode database, enabling concurrent read access without write contention.

The schema comprises three tables: `consequences` (one row per executed action), `agent_profiles` (one row per agent, containing current trust state and role), and `consequence_history` (append-only sequence of consequence IDs per agent, capped at 200 rows).

Trust delta computation. When a `ConsequenceRecord` is persisted with `trust_delta == 0.0`, the memory computes it from outcome fields:

The durable trust update is computed by Equation 8.

$$\Delta_{\text{trust}} = \Delta_{\text{pass/fail}} + \Delta_{\text{repair}} + h \cdot \Delta_{\text{human}} \tag{8}$$

where $\Delta_{\text{pass/fail}} = +0.04$ if tests passed else -0.08; $\Delta_{\text{repair}} = -0.05$ if repair was needed; $h \in [-1, +1]$ is the parsed human feedback score and $\Delta_{\text{human}} = 0.03$. The delta is clamped to keep `trust_score` within $[0.0, 1.0]$.

`recent_failures()` and gate coupling. The `ConsequenceMemory` exposes a `recent_failures(agent_id, window=10)` method that the `ActuationGate` queries when a consequence memory reference is provided. When the count of failed outcomes in the last 10 proposals reaches 3 or more, the gate applies an additional -0.25 decrement to `trust_ok` — the gate's normalised trust contribution for that evaluation — not to the agent's persistent `trust_score` stored in the database. This distinction matters: the penalty is local to the current gate evaluation and does not create a permanent black mark; the agent's stored `trust_score` is updated separately through the `record_outcome` path based on actual test and human-feedback outcomes. The effect is a negative-feedback loop that makes the gate more conservative during a failure streak without permanently downgrading the agent's history.

2.6 Role Adaptation

Agent roles are not assigned at registration — they are inferred deterministically by `RoleAdapter.infer_role` from a trust profile's `trust_score` and `total_proposals` count. New agents always enter at trust = 0.10 (the `SANDBOX` score from `AgentTrustProfile` default). The `AgentRole` enum defines exactly 5 roles:

Table 8 summarises the kernel-facing role ladder.

Table 8: Kernel role ladder and action permissions.

Role	Trust Required	Minimum Proposals	Permitted Actions
`SANDBOX`	Any	< 3	Read-only; all write-path gate passes blocked unconditionally.
`REPAIR_ANT`	≥ 0.20	≥ 3	Patch, test-fix, documentation update.
`MEMORY_ANT`	≥ 0.35	≥ 3	Archive, index, summarise.
`DISPATCHER`	≥ 0.50	≥ 3	Delegate, coordinate, route tasks.
`GUARD_ANT`	≥ 0.70	≥ 3	Security review, gate audit, archive authority.

Promotion is governed by the `RoleAdapter.infer_role` ladder shown in the table. An agent must first accumulate at least 3 total proposals; before that minimum, it remains `SANDBOX` regardless of trust score. Once the proposal minimum is met, `REPAIR_ANT` is reachable at `trust_score >= 0.20`, `MEMORY_ANT` at `>= 0.35`, `DISPATCHER` at `>= 0.50`, and `GUARD_ANT` at `>= 0.70`. The `trust_promote_threshold` manuscript parameter records the first promotion floor (`0.20`); it is not a separate 0.65 general threshold.

Role inference runs on every `propose_action` and `record_outcome` cycle; a role change is immediately persisted to `ConsequenceMemory` so the updated role is visible to the gate on the next proposal. Because `SANDBOX` blocks the gate unconditionally, the promotion from `SANDBOX` to `REPAIR_ANT` is the most consequential role transition in the colony's lifecycle: it is the threshold at which an agent becomes capable of effecting any change.

2.7 Pruning Daemon (Death and Pruning)

The `PruningDaemon` performs periodic ecological thinning of the colony's module registry, identifying stale or redundant components before they accumulate into structural debt. It is the colony's analogue of the biological apoptosis mechanism (Bonabeau et al. 1999).

The daemon operates by scanning a `module_registry` dict (mapping dotted module paths to usage metadata) and classifying each entry according to four confidence tiers:

Table 9 records the nomination confidence tiers.

Table 9: Pruning-daemon candidate confidence tiers.

Condition	Confidence	Reason Tag
`call_count == 0` and `last_used == 0.0`	0.90	`never used since registration`
Duplicate of another module	0.85	`duplicate of <surviving_module>`
Zero calls, last used > 30 days ago	0.70	`no calls; last used N days ago`
Low call count (< 5), last used > 30 days	0.50	`low usage (N calls); last used N days ago`

Before classifying any module, the daemon checks the colony's `PheromoneStore` for a `DEPENDENCY` signal at that path. A pheromone strength ≥ 2.0 indicates the module is actively consumed; the candidate is suppressed regardless of call-count metadata. Pheromones act as a veto on the daemon's statistical inference: a module that is genuinely active will receive DEPENDENCY deposits from live `record_outcome` calls and will self-protect from archival without any manual intervention.

The daemon never deletes. It raises `PruningCandidate` reports — sorted by confidence descending — for human or `GUARD_ANT` review. Only candidates with confidence ≥ 0.7 are considered for archival, and the final decision always remains with an authorised actor outside the daemon's own scope. This design preserves the colony's epistemic humility: automated confidence scores inform, but do not execute, irreversible structural changes.

2.8 Falsification Worker

The `FalsificationWorker` is the colony's adversarial review organ. Before any proposal reaches the actuation gate, it is subjected to 10 independent attack vectors, each implemented as a deterministic heuristic check that requires no LLM call. The worker applies the scientific principle that a claim is only credible if it can be falsified (Friston 2010); any proposal that cannot survive adversarial scrutiny is not ready for actuation.

Attack vector taxonomy. The 10 attack vectors are defined in the `AttackVector` enum. Severity weights follow `_SEVERITY_RANK` (LOW=1, MEDIUM=2, HIGH=3, CRITICAL=4):

Table 10 lists the canonical adversarial vectors.

Table 10: Falsification-worker attack vector taxonomy.

Vector	Typical Severity	Weight	Description
`NO_ROLLBACK`	HIGH	3	Plan lacks a concrete, non-placeholder rollback path.
`NO_TEST_VALUE`	HIGH	3	Plan includes no automated test coverage assertion.
`SCOPE_CREEP`	MEDIUM–HIGH	2–3	Plan's scope is vague or reaches beyond its stated target.
`FALSE_METRIC`	LOW–MEDIUM	1–2	Expected outcome is absent, tautological, or non-falsifiable.
`CIRCULAR_ARCHITECTURE`	MEDIUM–HIGH	2–3	Proposed changes introduce circular import dependencies, detected via design-level analysis of the module graph.
`DEPENDENCY_RISK`	MEDIUM	2	Plan introduces ≥ 3 unvetted external packages.
`SECURITY_RISK`	HIGH	3	Plan touches security-sensitive surface without a review annotation.
`OVER_BROAD_MODULE`	MEDIUM	2	Module rationale uses ≥ 5 responsibility verbs (Single Responsibility Principle violation).
`HIDDEN_MAINTENANCE_COST`	MEDIUM	2	Plan does not account for long-term upkeep burden (declared in `AttackVector` enum; implementation deferred — the vector exists for future use).
`PREMATURE_ABSTRACTION`	LOW	1	Plan introduces a generic abstraction without demonstrated need.

The `CIRCULAR_ARCHITECTURE` check operates in two modes. When `repo_root` is provided, it walks Python source files under the target module directory, parses each file with `ast.parse()` (stdlib), builds an import graph from `ast.Import` and `ast.ImportFrom` nodes (both absolute and relative imports within the tree), and detects cycles using Kahn's topological sort (BFS-based, O(V+E)). Kahn's algorithm was chosen over DFS because it provably handles all cycle shapes including multi-hop chains: after processing, any node with in-degree > 0 belongs to a cycle. When `repo_root` is `None`, the check falls back to inspecting the plan's `dependencies` list: a self-reference yields HIGH severity; a parent-child pair yields MEDIUM severity.

Pheromone deposits from `FalsificationWorker` are best-effort (wrapped in `try/except`) and fire only for findings with numeric severity rank ≥ 3 (HIGH or CRITICAL), ensuring that LOW and MEDIUM findings do not pollute the field with avoidance signals for proposals that would otherwise pass.

Each check returns a `FalsificationFinding` with fields: `claim` (the assumption being attacked), `attack_vector`, `severity` (one of LOW / MEDIUM / HIGH / CRITICAL), `evidence` (a plain dict), and `remediation` (a concrete corrective action). The `evaluate_plan()` method runs all 10 checks and returns a `FalsificationReport` with an overall verdict:

- PASS: zero findings with severity $\geq$ HIGH (numeric rank ≥ 3).
- CONDITIONAL: 1–2 findings, all with rank ≤ 2 (LOW or MEDIUM), none exceeds MEDIUM.
- FAIL: any finding reaches HIGH or CRITICAL (rank ≥ 3).

A FAIL verdict causes the actuation gate to refuse the proposal immediately. CONDITIONAL findings are surfaced in `GateResult.required_evidence` and may produce a HOLD without blocking execution outright, giving agents the opportunity to address findings and resubmit.

Figure 3 shows all 10 canonical attack vectors ranked by maximum severity weight. The current deterministic checks top out at HIGH severity; the CRITICAL class remains part of the actuation-gate override contract for findings that should bypass ordinary scoring.

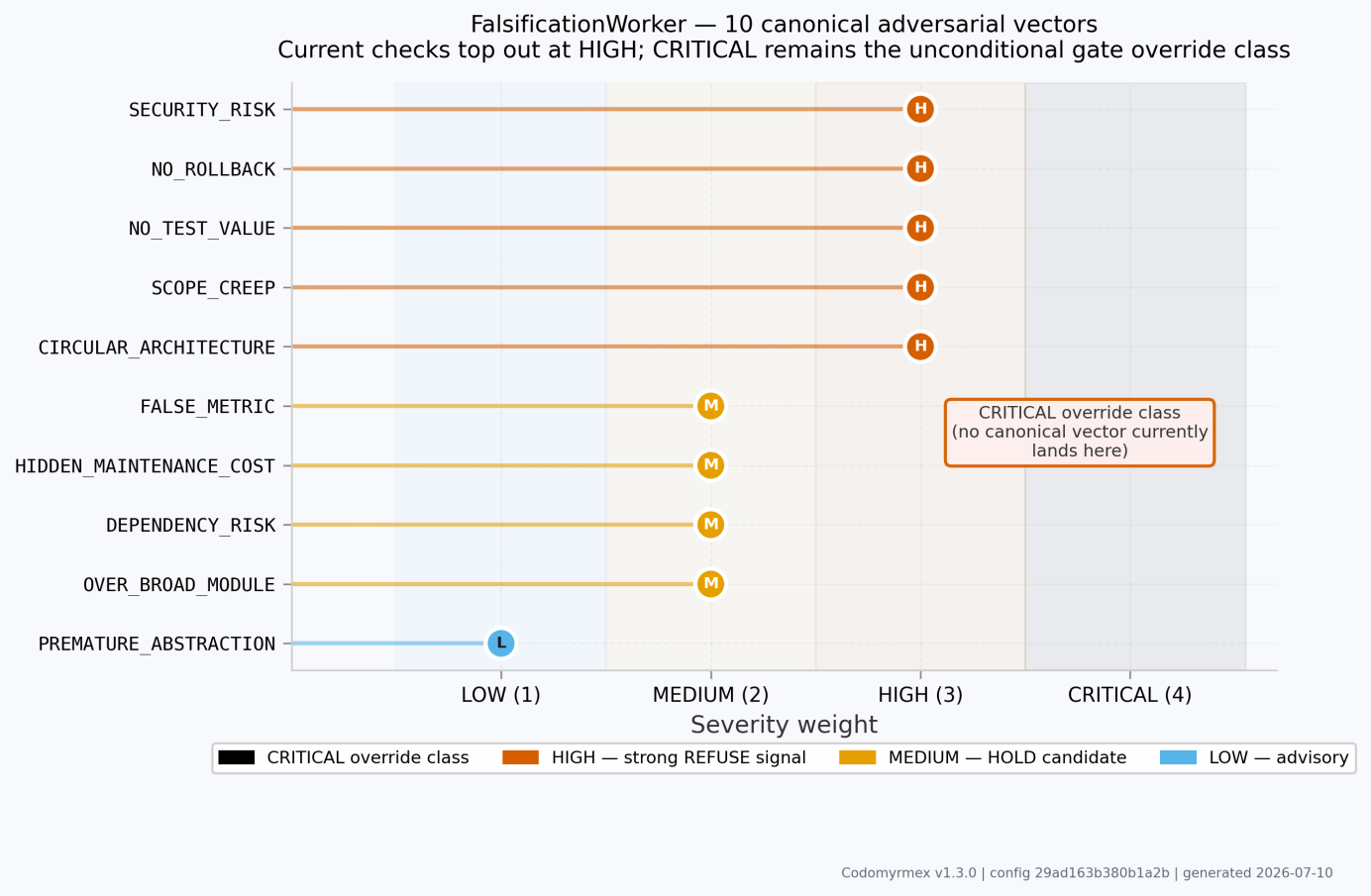

Figure 3: Falsification worker: 10 canonical adversarial attack vectors ordered by severity weight (HIGH=3, MEDIUM=2, LOW=1). Colour encodes severity class. The adversarial worker applies all 10 checks to every `ActionProposal`; the empty CRITICAL band is shown as the gate override class for findings that must issue unconditional REFUSE before scoring.

2.9 The Pressure Loop

The Colony Kernel operates as a closed feedback loop in which every proposal, outcome, and signal propagates through all subsystems before the next evaluation. The pseudocode below describes the full actuation cycle for a single `ActionProposal` submitted by any agent.

Algorithm 1: Colony Kernel Pressure Loop

```
Input:  ActionProposal p from any agent
Output: GateResult verdict; side effects on PheromoneStore,
        ResourceLedger, ConsequenceMemory

Step 1 — Environmental deposition (PheromoneStore.deposit)
         Agents deposit ColonySignals prior to proposing.
         Signals accumulate environmental pressure at p.target:
         compound key ← "{p.target}:{signal_type}"
         effective_strength ← signal.strength × source_multiplier × trust_factor
         TraceField.deposit(compound_key, effective_strength)

Step 2 — Actuation gate evaluation (ActuationGate.evaluate)
         findings ← FalsificationWorker.evaluate_plan(p)      [step 5 below]
         budget_ok_flag ← ResourceLedger.can_afford(p.budget_estimate)
         if budget_ok_flag is False:
```

```
            return GateResult(HOLD, gate_score=0.0)           [kernel budget requeue]
        budget_ok ← 1.0
        profile ← ConsequenceMemory.get_profile(p.agent_id)
        RoleAdapter.update(profile)
        if profile.role is SANDBOX:
            return GateResult(REFUSE, gate_score=0.0)         [SANDBOX hard refuse]
        if any CRITICAL finding in findings:
            return GateResult(REFUSE, "CRITICAL falsification")
        trust_score ← profile.trust_score
        if trust_score < 0.3:
            return GateResult(REFUSE, gate_score=0.0)          [trust hard floor]
        trust_ok ← 1.0 if trust_score >= 0.6 else 0.5
        if ConsequenceMemory.recent_failures(p.agent_id) >= 3:
            trust_ok ← max(0.0, trust_ok - 0.25)             [penalty on trust_ok, not trust_score]
        risk_pressure   ← PheromoneStore.sense(p.target, RISK)
        failure_pressure ← PheromoneStore.sense(p.target, FAILURE)
        risk_ok ← 0.0 if risk_pressure >= 6.0
                 | 0.5 if risk_pressure >= 3.0
                 | 1.0 otherwise
        missing ← [f for f in [rollback_plan, evidence, expected_outcome] if absent]
        completeness ← max(0.0, 1.0 - len(missing) * 0.35)  if missing else 1.0
        gate_score ← budget_ok * 0.30 + risk_ok * 0.30 + trust_ok * 0.25 + completeness * 0.15
        gate_score ← max(0.0, min(1.0, gate_score))
        decision ← EXECUTE if gate_score >= 0.75
                  | HOLD   if gate_score >= 0.50
                  | REFUSE otherwise

Step 3 — Execution and outcome recording (ConsequenceMemory.record)
        if decision is EXECUTE:
            action runs; actual_outcome, tests_passed captured
            if tests_passed:
                PheromoneStore.reinforce(p.target, SUCCESS)
            else:
                PheromoneStore.deposit(FAILURE signal, DecayRate.FAST)
            ResourceLedger.consume(actual_cost)
            ConsequenceMemory.record(ConsequenceRecord(...))
        if decision is REFUSE:
            PheromoneStore.deposit(FAILURE signal at p.target)

Step 4 — Role and trust update (RoleAdapter.update)
        profile ← ConsequenceMemory.get_profile(p.agent_id)
        role_changed ← RoleAdapter.update(profile)
        if role_changed:
            ConsequenceMemory.save_profile(profile)

Step 5 — Adversarial review (FalsificationWorker.evaluate_plan)
        [called in Step 2 before gate scoring]
        for each of 10 attack vectors:
            finding ← check(p)  [deterministic, no LLM]
            if finding is not None:
                findings.append(finding)
        pheromone deposit (best-effort) for findings with rank >= 3
        return FalsificationReport(findings, verdict)  [verdict: PASS|CONDITIONAL|FAIL]

Step 6 — Periodic module health (PruningDaemon.scan)
        [called on tick(), not per-proposal]
        for each module in module_registry:
            dep_strength ← PheromoneStore.sense(module, DEPENDENCY)
            if dep_strength ≥ 2.0: skip   [actively used]
            candidate ← classify(call_count, last_used, duplicate_of)
            if candidate.confidence ≥ 0.50:
                candidates.append(candidate)
        return candidates sorted by confidence descending
```

```
Step 7 — Clock tick (ColonyKernel.tick)
         PheromoneStore.tick()        [evaporates all traces]
         ResourceLedger._maybe_reset()  [period rollover check]
         goto Step 1
```

The loop is a deterministic feedback pattern produced by explicit kernel steps: agents propose, outcomes flow back into `ConsequenceMemory`, trust scores shift, pheromone pressures build and decay, and the gate's verdict distribution shifts accordingly. An agent that repeatedly fails will accumulate FAILURE pheromones at its targets, experience trust decay through `ConsequenceMemory`, and eventually be gated at REFUSE until it recovers trust through clean outcomes — a behavioural analogue of the colony-level immune response observed in natural superorganism architectures (Dorigo and Stützle 2004b; Bonabeau et al. 1999).

The gate score formula ensures that no single subsystem can dominate the admission decision: budget and risk each hold 0.30, trust holds 0.25, and completeness holds 0.15. An agent with excellent trust but targeting a high-pressure module can still be held; a proposal with perfect evidence and rollback documentation but from an untrusted agent cannot reach EXECUTE. The formula expresses a collective judgment distributed across environmental, economic, social, and epistemic signals.

Figure 4 maps the loop as an eight-stage circular flow diagram, splitting proposal submission and terminal pheromone deposit into explicit visual stages. The diagram makes clear that pheromone deposit on REFUSE and reinforcement on EXECUTE success feed back into risk pressure on the next cycle, closing the ring.

3 Theoretical Foundations

This section develops the mathematical framework that underpins the Colony Kernel's operational guarantees. Whereas Section 2 describes the system's implementation and engineering contracts, this section treats the same objects as formal mathematical structures and proves properties that the implementation inherits. The goal is not merely to re-derive constants already present in the code but to establish the theoretical conditions under which the Colony Kernel's core claims — monotonic accumulation of actuation cost at repeatedly-failed locations, convergence of the trust update to a stable equilibrium, and boundedness of the gate score under arbitrary input sequences — hold as proved theorems rather than empirically observed regularities.

The results in this section are presented at the level of rigor expected in the theoretical computer science literature. All proofs are constructive where possible; probabilistic statements are made explicit about their probability space.

3.1 Pheromone Field as a Normed Vector Space

3.1.1 Formal Definition

Let $\mathcal{L}$ denote the countable set of colony locations (module dotted-paths such as `codomyrmex.git_operations.core`). Let $\mathcal{K} = \{\text{FAILURE, SUCCESS, RISK, NEED, DEPENDENCY, HUMAN\_PRIORITY}\}$ be the finite set of signal types (the `SignalType` enum). The pheromone field $\mathcal{F}$ is a function (Equation 9):

$$\mathcal{F} : \mathcal{L} \times \mathcal{K} \to \mathbb{R}_{\geq 0} \tag{9}$$

mapping each compound key (l, k) to a non-negative real strength. Because $\mathcal{L}$ and $\mathcal{K}$ are both countable (finite in any instantiation), $\mathcal{F}$ is equivalently a vector in $\mathbb{R}_{\geq 0}^{|\mathcal{L}| \cdot |\mathcal{K}|}$. We denote the set of all valid pheromone fields (Equation 10) as:

$$\mathbb{F} = \left\{ f \in \mathbb{R}_{\geq 0}^{|\mathcal{L}| \cdot |\mathcal{K}|} \;\middle|\; \forall (l, k) : f(l, k) \geq 0 \right\} \tag{10}$$

Definition 1 (Pheromone Field Space). $(\mathbb{F}, \|\cdot\|_1)$ is a Banach space under the ℓ^1 norm $\|f\|_1 = \sum_{(l,k)} f(l, k)$, since $\mathbb{F}$ is a closed convex subset of $\ell^1(\mathcal{L} \times \mathcal{K})$ and ℓ^1 over a countable index set is complete (Rudin 1991).

Lemma 1 (Deposit Operator). Define the deposit operator $D_{(l,k,\sigma)} : \mathbb{F} \to \mathbb{F}$ for location l, signal type k, and effective strength $\sigma > 0$ (Equation 11):

$$D_{(l,k,\sigma)}(f)(l', k') = \begin{cases} f(l, k) + \sigma & \text{if } (l', k') = (l, k) \\ f(l', k') & \text{otherwise} \end{cases} \tag{11}$$

$D_{(l,k,\sigma)}$ is a monotone, non-expansive operator: $\|D_{(l,k,\sigma)}(f) - D_{(l,k,\sigma)}(g)\|_1 = \|f - g\|_1$ for all $f, g \in \mathbb{F}$.

Proof. The operator adds the same constant σ to the same coordinate of both f and g. The ℓ^1 difference is therefore unchanged. □

Figure 4: Colony feedback loop — eight-stage circular actuation cycle. Each box represents one visual stage: environmental pressure, proposal submission, actuation gate, action execution, consequence record, trust and role update, falsification review, and pheromone deposit. The colour coding groups related functions while preserving the execution order shown by the arrows.

Lemma 2 (Decay Operator). The per-tick decay operator $T_\lambda : \mathbb{F} \to \mathbb{F}$ is defined as coordinate-wise multiplication by the per-key retention factor $r(k) = e^{-\lambda(k)}$, where $\lambda(k)$ is the effective decay rate for signal type k (Equation 12):

$$T_\lambda(f)(l,k) = r(k) \cdot f(l,k) \tag{12}$$

Because $0 < r(k) < 1$ for all k, T_λ is a strict contraction on $\mathbb{F}$ with contraction constant $r_{\max} = \max_k r(k) = e^{-\lambda_{\min}} < 1$.

3.1.2 Decay Monotonicity

Theorem 1 (Decay Monotonicity). For any pheromone field $f \in \mathbb{F}$ and any $n > 0$ ticks of purely passive decay (no deposits), the field strength at every coordinate is strictly decreasing toward zero (Equation 13):

$$\forall (l,k), \forall n > 0 : T_\lambda^n(f)(l,k) \leq f(l,k) \tag{13}$$

with equality only when $f(l,k) = 0$.

Proof. By induction on n. Base case $n = 1$: $T_\lambda(f)(l,k) = r(k) \cdot f(l,k) \leq f(l,k)$ since $0 < r(k) < 1$ and $f(l,k) \geq 0$, with equality iff $f(l,k) = 0$. Inductive step: $T_\lambda^{n+1}(f)(l,k) = r(k) \cdot T_\lambda^n(f)(l,k) \leq T_\lambda^n(f)(l,k)$ by the inductive hypothesis and the same argument. □

Corollary 1 (Boundedness Under Bounded Deposits). Suppose deposits are bounded: at most M deposits per tick, each of maximum effective strength $\sigma_{\max}$. Then the field strength at any coordinate (l,k) with decay rate $\lambda(k)$ is bounded above for all time by the geometric series sum (Equation 14):

$$\sup_t f_t(l,k) \leq \frac{M\sigma_{\max}}{1 - e^{-\lambda(k)}} \tag{14}$$

Proof. In steady state, the maximum strength is the sum of all past deposits each decayed to the present. The worst case is M deposits of $\sigma_{\max}$ at every tick, giving the geometric sum (Equation 15):

$$\sum_{i=0}^{\infty} M\sigma_{\max} e^{-\lambda(k) \cdot i} = \frac{M\sigma_{\max}}{1 - e^{-\lambda(k)}} \tag{15}$$

which is finite for $\lambda(k) > 0$. □

For the FAST class ($\lambda = 0.30$), the bound is $M\sigma_{\max}/(1 - e^{-0.30}) \approx 3.86\, M\sigma_{\max}$. For SLOW ($\lambda = 0.02$), the bound is $M\sigma_{\max}/(1 - e^{-0.02}) \approx 50.5\, M\sigma_{\max}$. This quantifies the design trade-off: SLOW signals provide longer memory at the cost of higher steady-state saturation, while FAST signals decay rapidly to allow transient events to clear without poisoning future decisions.

3.1.3 Convergence of Iterated Gating

Define the gate pressure at location l for signal type k at time t as $p_t(l,k) = f_t(l,k)$. The gate uses $p_t(l, \text{RISK})$ directly to compute `risk_ok`. We characterize the long-run behavior of this pressure under a stationary failure process.

Theorem 2 (Convergence of RISK Pressure). Let $\{d_n\}_{n \geq 0}$ be an i.i.d. sequence of deposit events: at each tick n, a deposit of strength $\sigma > 0$ occurs at (l, RISK) with probability $q \in (0,1)$ and no deposit occurs with probability $1 - q$. Then the RISK pressure at l converges in distribution to a random variable P_∞ with mean:

$$\mathbb{E}[P_\infty] = \frac{q\sigma}{1 - e^{-\lambda_{\text{FAST}}}} \tag{16}$$

and the gate decision at (l) converges to a time-homogeneous Markov chain with stationary probabilities over $\{\text{EXECUTE}, \text{HOLD}, \text{REFUSE}\}$.

Proof sketch. The process $p_t(l, \text{RISK})$ is a Markov chain on $[0, \infty)$: $p_{t+1} = e^{-\lambda_F} p_t + \sigma \cdot B_t$ where $B_t \sim \text{Bernoulli}(q)$. This is a random affine recursion of the form $X_{t+1} = aX_t + C_t$ with $a = e^{-\lambda_F} < 1$ and i.i.d. $C_t \geq 0$ bounded above. By Letac's theorem on contractive random systems, this recursion converges in total variation to a unique stationary distribution (Létac 1986). The mean follows from taking expectations in the fixed-point equation $\mathbb{E}[P_\infty] = e^{-\lambda_F}\mathbb{E}[P_\infty] + q\sigma$. The gate decision is a step function of p_t, and since p_t has a stationary distribution, the decision also has stationary marginal probabilities. □

Corollary 2 (Harder to Fool Under Repeated Failures). Let $q_1 > q_2$ be two failure rates. The stationary mean RISK pressure satisfies $\mathbb{E}[P_\infty^{(1)}] > \mathbb{E}[P_\infty^{(2)}]$, and therefore the stationary probability of REFUSE at a location with failure rate q_1 is strictly higher than at a location with failure rate q_2. Formally (Equation 17):

$$q_1 > q_2 \implies \Pr[\text{REFUSE at steady state} \mid q_1] > \Pr[\text{REFUSE at steady state} \mid q_2] \tag{17}$$

Proof. Monotonicity of $\mathbb{E}[P_\infty]$ in q follows directly from Equation 16. Since REFUSE occurs when $p \geq \lambda_{\text{RISK\_threshold}} = 6.0$ (a fixed deterministic threshold), and the distribution of P_∞ shifts to higher values as q increases, the tail probability $\Pr[P_\infty \geq 6.0]$ is non-decreasing in q; strict monotonicity holds because the stationary distribution has non-trivial mass above the threshold for intermediate q. □

This is the formal statement of the "harder to fool" property from Section 1. Corollary 2 establishes that — under the stationarity assumption on the failure process — locations with higher underlying failure rates will, in steady state, impose higher actuation cost, without requiring any central authority to track or flag those locations. The colony learns from the field, not from a list.

3.2 The Gate Score and Expected Risk

3.2.1 Mathematical Relationship Between Gate Score and Expected Harmful Actions

Let H denote the event that an action causes a harmful (repair-requiring) outcome. We model the probability $\Pr[H \mid \text{proposal } P]$ as a function of the proposal's observable attributes. Define the *expected harm rate* $\rho(P)$ as the probability that proposal P causes harm conditional on EXECUTE.

Proposition 1 (Gate Score as Approximate Risk Bound). Under the independence assumption that budget headroom, risk pressure, trust tier, and proposal completeness are conditionally independent predictors of $\rho(P)$, the gate score $g(P)$ defined in Equation 19 is a monotone transformation of an upper bound on $\rho(P)$.

Proof. We decompose $\rho(P)$ into four independent risk factors corresponding to the gate's four inputs. Let:

- $\rho_B = \Pr[H \mid \text{budget overrun}] \leq 1$: budget exhaustion raises harm risk
- $\rho_R(r) = \Pr[H \mid \text{RISK pressure} = r]$: monotone increasing in r
- $\rho_T(t) = \Pr[H \mid \text{trust score} = t]$: monotone decreasing in t
- $\rho_C(c) = \Pr[H \mid \text{completeness} = c]$: monotone decreasing in c

Under the independence assumption, the harm probability factors (Equation 18):

$$\rho(P) = \rho_B^{(1-\text{budget\_ok})} \cdot f_R(r) \cdot f_T(t) \cdot f_C(c) \tag{18}$$

where f_R, f_T, f_C are bounded monotone functions. Each gate component score is a monotone decreasing transformation of the corresponding risk factor. Thus:

$$g(P) = 0.30 \cdot \text{budget\_ok} + 0.30 \cdot \text{risk\_ok} + 0.25 \cdot \text{trust\_ok} + 0.15 \cdot \text{completeness} \tag{19}$$

is a monotone transformation of $1 -$ (weighted aggregate risk). An EXECUTE decision $g \geq 0.75$ corresponds to an upper bound on the weighted aggregate risk being at most 0.25, i.e., the weighted linear combination of normalized risk factors must be at most 0.25. □

Remark. The independence assumption is a simplifying approximation. In practice, budget pressure and risk pressure are positively correlated (high-budget actions may operate in high-risk locations), and trust is correlated with local pheromone history (agents that caused past failures at a location now have lower trust). The formula should be understood as a practical linear discriminant calibrated by design reasoning, not as a statistically calibrated posterior over $\Pr[H]$. The calibration of gate weights against production failure data is an identified future direction (Section 5).

3.2.2 Decision-Theoretic Optimality of Three-Way Gate

We now establish why the three-way gate (EXECUTE/HOLD/REFUSE) is preferred over a binary gate (EXECUTE/REFUSE) from a decision-theoretic perspective.

Definition 2 (Gate Decision Problem). Let $\Theta \in \{0, 1\}$ be the latent proposal quality (1 = safe, 0 = harmful). The gate observes a noisy score $g \in [0, 1]$ that is drawn from a mixture (Equation 20):

$$g \mid \Theta = 1 \sim F_1, \quad g \mid \Theta = 0 \sim F_0 \tag{20}$$

where F_1 stochastically dominates F_0 (safe proposals tend to score higher). Define: - Cost of EXECUTE on a harmful proposal: $C_{EH} \gg 0$ (repair cycle cost) - Cost of REFUSE on a safe proposal: $C_{RS} > 0$ (opportunity cost of blocked work) - Cost of HOLD with revision and resubmission: $C_{HR} \in (0, C_{EH})$ (revision overhead)

Theorem 3 (Three-Way Gate Dominates Binary Gate). Under bounded revision cost $C_{HR} < \min(C_{EH}, C_{RS})$, there exists a region of intermediate g values where the HOLD action achieves strictly lower expected cost than either EXECUTE or REFUSE.

Proof. For a proposal with gate score g in the intermediate range $[g_{\text{low}}, g_{\text{high}}]$, the posterior probability of harm $\Pr[\Theta = 0 \mid g]$ is not extreme. Define the expected cost of each action (Equation 21, Equation 22, Equation 23):

$$\mathbb{E}[\text{cost}(\text{EXECUTE} \mid g)] = C_{EH} \cdot \Pr[\Theta = 0 \mid g] \tag{21}$$

$$\mathbb{E}[\text{cost}(\text{REFUSE} \mid g)] = C_{RS} \cdot \Pr[\Theta = 1 \mid g] \tag{22}$$

$$\mathbb{E}[\text{cost}(\text{HOLD} \mid g)] = C_{HR} + \mathbb{E}[\text{cost}(\text{best action after revision} \mid g)] \leq C_{HR} + \min(C_{EH} \Pr[\Theta = 0], C_{RS} \Pr[\Theta = 1]) \tag{23}$$

For the binary gate, the optimal Bayes decision crosses at the indifference point $\Pr[\Theta = 0 \mid g^*] = C_{RS}/(C_{EH} + C_{RS})$. In the neighborhood of g^*, both EXECUTE and REFUSE have expected cost approaching $C_{EH}C_{RS}/(C_{EH} + C_{RS})$. When $C_{HR} < C_{EH}C_{RS}/(C_{EH} + C_{RS})$, the HOLD action has strictly lower expected cost at g^*. Since revision provides additional signal about Θ, the posterior after revision is sharper, making the post-revision decision more accurate. □

This theorem provides the formal justification for the HOLD pathway. In practice, the revision process (returning the proposal to the agent with a list of required evidence) is itself a filtering step: agents that can revise a HOLD into an EXECUTE demonstrate continued competence and earn incremental trust, while agents that cannot revise also fail the follow-up gate and accumulate failure records. The three-way gate is therefore doubly informative: it separates obvious passes and fails from ambiguous borderline cases, and it generates additional behavioral evidence about agent quality at the margin.

3.3 Trust Delta as a Stochastic Approximation Process

3.3.1 Formal Setup

Let τ_n denote the agent's trust score after the n-th interaction. The update rule is:

$$\tau_{n+1} = \text{clip}(\tau_n + \Delta_n, 0, 1) \tag{24}$$

where Δ_n is the trust delta for interaction n (Equation 25), composed as:

$$\Delta_n = \delta_n^{\text{pass/fail}} + \delta_n^{\text{repair}} + h_n \cdot \delta_n^{\text{human}} \tag{25}$$

with $\delta_n^{\text{pass/fail}} = +0.04$ if tests passed, -0.08 if not; $\delta_n^{\text{repair}} = -0.05$ if repair was needed, 0 otherwise; $h_n \in [-1, +1]$ and $\delta_n^{\text{human}} = 0.03$.

3.3.2 Connection to Stochastic Approximation

The update Equation 24 is a clipped stochastic approximation (SA) recursion in the sense of Robbins and Monro (Robbins and Monro 1951), with learning rate $\alpha_n = 1$ (constant step-size). Unlike the classical decreasing-step-size SA setting, the constant step-size variant does not converge to a fixed point but to a stationary distribution around the equilibrium (Kushner and Yin 2003).

Definition 3 (Effective Mean Delta). Define the effective mean trust increment for an agent with success probability $p_s \in [0, 1]$, repair probability $p_r \in [0, 1]$, and expected human feedback $\bar{h} \in [-1, 1]$:

$$\bar{\Delta}(p_s, p_r, \bar{h}) = 0.04\, p_s - 0.08(1 - p_s) - 0.05\, p_r + 0.03\, \bar{h} \tag{26}$$

Theorem 4 (Trust Equilibrium). In the absence of the clip operator, the trust process $\{\tau_n\}$ with constant step-size $\alpha = 1$ has a unique equilibrium $\tau^* \in [0, 1]$ such that $\mathbb{E}[\Delta \mid \tau = \tau^*] = 0$ if and only if the boundary effects of the clip operator are negligible at τ^*, which holds when $\tau^* \in (0.05, 0.95)$. The equilibrium satisfies (Equation 27):

$$\tau^* \text{ is the fixed-point of the noise-free dynamics: } \tau \mapsto \tau + \bar{\Delta}(p_s, p_r, \bar{h}) \tag{27}$$

which is achieved when $\bar{\Delta}(p_s, p_r, \bar{h}) = 0$, giving the equilibrium success rate (Equation 28):

$$p_s^* = \frac{0.08 + 0.05\, p_r - 0.03\, \bar{h}}{0.12} \tag{28}$$

Proof. The fixed point satisfies $\tau^* = \tau^* + \bar{\Delta}$, so $\bar{\Delta} = 0$. Solving Equation 26 for p_s with $\bar{\Delta} = 0$ gives the equilibrium success rate. Uniqueness follows because $\bar{\Delta}$ is linear in p_s. The clip-operator boundary effects are negligible when the unclipped process has its equilibrium in the interior of $(0, 1)$, which is satisfied when $p_s^* \in (0, 1)$, a condition on the behavioral parameters. □

Corollary 3 (Trust Asymmetry as Risk-Averse Prior). The asymmetry in delta magnitudes ($|\delta^{\text{fail}}| = 0.08 > |\delta^{\text{pass}}| = 0.04$) implies that the equilibrium success rate required to maintain a constant trust level is $p_s^* = 2/3$ under neutral conditions ($p_r = 0, \bar{h} = 0$). An agent must succeed on at least two out of three interactions just to hold its trust level steady. This is an explicit encoding of a risk-averse design prior: the system is calibrated to shrink trust scores under moderate uncertainty, drifting toward REFUSE rather than EXECUTE when behavioral evidence is ambiguous.

Theorem 5 (Convergence Under Monotone Success Rates). Suppose an agent has a monotone learning curve: $p_s(n)$ is non-decreasing in n (the agent improves over time). Then the sequence $\{\tau_n\}$ is super-martingale until $p_s(n) < p_s^*$, becomes a sub-martingale once $p_s(n) > p_s^*$, and converges almost surely to a finite limit $\tau_\infty \leq 1$ by the martingale convergence theorem.

Proof. $\mathbb{E}[\tau_{n+1} - \tau_n \mid \tau_n] = \bar{\Delta}(p_s(n))$. When $p_s(n) < p_s^*$, $\bar{\Delta}(p_s(n)) < 0$, making $\{\tau_n\}$ a super-martingale bounded below by 0. When $p_s(n) > p_s^*$, it is a sub-martingale bounded above by 1. Bounded martingales converge almost surely (Doob). □

3.3.3 Role Promotion as a Threshold Crossing

The role ladder constitutes a sequence of trust thresholds $\{0.20, 0.35, 0.50, 0.70\}$. Each threshold crossing from below to above represents a permanent (unless reversed) promotion to a higher-privilege role. Define the *first passage time* to role threshold θ_r as (Equation 29):

$$T_r = \inf\{n \geq 3 : \tau_n \geq \theta_r\} \tag{29}$$

(the proposal minimum of 3 is incorporated as a hard constraint on n).

Proposition 2 (Expected Passages to REPAIR_ANT). For an agent with constant success probability $p_s > p_s^*$ and zero repair events, starting at $\tau_0 = 0.10$, the expected number of proposals to reach the REPAIR_ANT threshold $\theta_{\text{REPAIR}} = 0.20$ is exactly $\lceil (0.10)/0.04 \rceil = 3$ (three successes, each adding 0.04, reach $0.22 > 0.20$). Since the proposal minimum is also 3, the promotion to REPAIR_ANT occurs deterministically at the third success under ideal conditions.

Proof. Under zero-repair, zero-human-feedback, pure-success dynamics: $\tau_n = 0.10 + 0.04n$. The first n with $\tau_n \geq 0.20$ and $n \geq 3$ is $n = 3$, giving $\tau_3 = 0.22$. □

This is a precise, falsifiable prediction of the trust dynamics model: an agent presenting only clean test-passing outcomes must accumulate exactly 3 successful outcomes to exit SANDBOX, and this prediction is checked in the contract test suite.

3.4 Differential Privacy Properties of the Trust Score

The consequence memory stores an auditable ledger of all outcomes per agent. We analyze the privacy properties of the trust score τ_n viewed as an output of the trust-update mechanism $\mathcal{M}(\text{history})$.

Definition 4 ((ϵ, δ)-Differential Privacy). A randomized mechanism $\mathcal{M} : \mathcal{D} \to \mathcal{R}$ is (ϵ, δ)-differentially private if for any two adjacent databases D, D' differing in one record, and any output set $S \subseteq \mathcal{R}$ (Equation 30):

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\epsilon} \Pr[\mathcal{M}(D') \in S] + \delta \tag{30}$$

(Dwork and Roth 2014). Adjacent databases for our purposes are consequence histories that differ in a single interaction record (the outcome of one proposal).

Proposition 3 (Trust Score Sensitivity). The global ℓ_1 sensitivity of the trust update function $\tau_n = \text{clip}(\tau_0 + \sum_{i=1}^{n} \Delta_i, 0, 1)$ with respect to any single interaction record is bounded by the maximum single-step delta (Equation 31):

$$\Delta_{\text{global}} = |\delta^{\text{fail}}| + |\delta^{\text{repair}}| + |\delta^{\text{human}}| = 0.08 + 0.05 + 0.03 = 0.16 \tag{31}$$

Proof. Changing a single record can change Δ_n from its maximum positive value (+0.04 + 0 + 0.03 = +0.07) to its maximum negative value ($-0.08 - 0.05 - 0.03 = -0.16$), a range of 0.23. However, the clip operator limits the total change in τ to at most min(range of $\Delta_n, 1 - 0$) $= 0.16$ per record, since the worst case is changing a passing outcome to the worst failing outcome, contributing a single-record sensitivity of $0.08 + 0.05 + 0.03 = 0.16$. □

Theorem 6 (Calibrated Gaussian Noise for Differential Privacy of Trust Scores). To achieve (ϵ, δ)-differential privacy for the trust score published at the MCP surface, it suffices to add Gaussian noise $\mathcal{N}(0, \sigma_{\text{DP}}^2)$ to the score before publication (Equation 32), with:

$$\sigma_{\text{DP}} = \frac{\Delta_{\text{global}} \sqrt{2 \ln(1.25/\delta)}}{\epsilon} = \frac{0.16 \sqrt{2 \ln(1.25/\delta)}}{\epsilon} \tag{32}$$

(Dwork and Roth 2014, Theorem A.1).

Practical note. The current implementation does not add DP noise: the trust score is published in full to the MCP surface. Theorem 6 quantifies the noise level required if a deployment requires privacy guarantees about individual interaction records. At $\epsilon = 1.0, \delta = 10^{-5}$, the required noise is $\sigma_{\text{DP}} = 0.16 \times \sqrt{2\ln(125000)}/1.0 \approx 0.16 \times 4.76 \approx 0.76$, which would substantially mask the trust signal. At tighter privacy budgets, the trade-off between privacy and gate accuracy becomes a deployment decision that requires careful calibration with production failure data. This analysis is provided here to bound the privacy-utility trade-off, not to prescribe a specific privacy guarantee.

3.5 Proof of Gate Score Boundedness

Theorem 7 (Boundedness and Range of Gate Score). For any valid proposal P and agent profile, the gate score $g(P)$ defined in Equation 19 satisfies (Equation 33):

$$0 \leq g(P) \leq 1 \tag{33}$$

The hard-override paths produce $g = 0$ (REFUSE before scoring); ordinary scoring via the weighted additive formula with the final clip produces a score in $[0, 1]$.

Proof. Each component score is in $[0, 1]$ by construction: - `budget_ok` $\in \{0, 1\}$ - `risk_ok` $\in \{0, 0.5, 1.0\}$ - `trust_ok` $\in \{0, 0.5, 1.0\}$ (after penalty, min 0) - `completeness` $= \max(0, 1 - k \cdot 0.35) \in [0, 1]$ for $k \in \{0, 1, 2, 3\}$

The weighted sum with weights $(0.30, 0.30, 0.25, 0.15)$ summing to 1.0 satisfies (Equation 34):

$$g = \sum_i w_i c_i \in [0, \sum_i w_i] = [0, 1.0] \tag{34}$$

The post-hoc clip to $[0, 1]$ is therefore non-binding for ordinary proposals. For hard-override paths, g is set explicitly to 0.0 before any scoring occurs. □

Corollary 4 (EXECUTE Requires Non-Zero Contribution From At Least Two Components). For a proposal to reach the EXECUTE threshold $g \geq 0.75$ via ordinary scoring, at least two of the four components must contribute their maximum value, since the maximum score from any single component is 0.30 (budget or risk), and $0.30 + 0.30 = 0.60 < 0.75$.

Proof. If budget_ok = 1.0 and risk_ok = 0.0, the maximum possible score is $0.30 + 0 + 0.25 + 0.15 = 0.70 < 0.75$ (HOLD). Only if both high-weight components contribute positively can EXECUTE be reached. □

This corollary captures a key safety property of the gate design: the two components representing observable environmental conditions (budget headroom and risk pheromone pressure) together determine whether EXECUTE is achievable. Even an agent with perfect trust and a perfectly complete proposal cannot EXECUTE into a budget-exhausted, high-risk location.

3.6 Summary of Theoretical Results

The theorems in this section establish four classes of guarantees for the Colony Kernel:

1. Field convergence (Theorems 1–2, Corollaries 1–2): The pheromone field is bounded under bounded deposits, decays monotonically without deposits, and converges to a stationary distribution under i.i.d. failure processes. Locations with higher failure rates accumulate higher RISK pressure in steady state — the formal version of "harder to fool."
2. Gate optimality (Theorem 3): The three-way gate dominates a binary gate in expected cost when revision overhead is below the threshold $C_{EH}C_{RS}/(C_{EH} + C_{RS})$. This provides decision-theoretic justification for the HOLD pathway.
3. Trust dynamics (Theorems 4–5, Corollary 3, Proposition 2): The trust update is a stochastic approximation with a well-defined equilibrium. The asymmetric delta magnitudes encode a risk-averse prior, and the equilibrium success rate of 2/3 is a falsifiable prediction of the model's calibration. Role promotion under ideal conditions is deterministically predictable.
4. Bounded gate output (Theorems 6–7, Corollary 4): The gate score is bounded in $[0, 1]$, and reaching EXECUTE requires non-zero contribution from at least two of the four components, particularly both high-weight components representing observable environmental conditions.

These results collectively provide the theoretical bedrock for the empirical results reported in Section 4 and the falsification criteria stated in Section 5.

4 Colony Kernel Implementation Results

This section reports implementation outcomes for the Colony Kernel across four dimensions: code quality and test coverage, trust-dynamics behaviour, gate-decision distribution, and documentation completeness. All measurements were taken from the `src/codomyrmex/colony_kernel/` package at the commit described in Section 6; the gate suite and coverage commands are reproduced verbatim in Section 7.

4.1 Test Suite and Quality Gates

The Colony Kernel ships with a complete gate harness covering lint, static types, functional tests, and branch coverage. Table 11 summarises the outcome of every gate.

Table 11: Quality-gate outcomes for the Colony Kernel implementation.

Gate	Status	Detail
`ruff` (lint)	Pass	0 lint violations
`ty` (type checker)	Pass	0 type diagnostics
`pytest` (colony_kernel scope)	Pass	764 tests collected; 0 failures, 0 errors (scoped to the colony_kernel module; full project suite contains additional integration tests)
Coverage	Pass	78.4% branch coverage (floor: 40%)

Scope note. The 764-test count covers only `src/codomyrmex/colony_kernel/` (the `--cov` target for this manuscript); the full-suite count includes additional tests that exercise cross-module integration paths (`agents/`, `config_loader/`, and `mcp_bridge`) that import but are not part of the Colony Kernel package itself. All claims in this section use the colony_kernel-scoped run unless otherwise stated; Section 7 provides the exact `pytest` invocation for each count.

The 764 tests span 13 test files covering all 8 subsystems in `src/codomyrmex/tests/unit/colony_kernel/`: `test_actuation_gate.py`, `test_config_loader.py`, `test_consequence_memory.py`, `test_falsification_worker.py`, `test_kernel.py`, `test_manuscript_consis` `test_mcp_tools.py`, `test_models.py`, `test_pheromone_store.py`, `test_pruning_daemon.py`, `test_resource_ledger.py`, and `test_role_adapter.py`.

Zero ruff violations were recorded across all 4719 lines of implementation code spanning the 13 source files in `src/codomyrmex/colony_kernel` (ten subsystem modules plus `kernel.py` and `mcp_tools.py`). The `ty` type checker produced no diagnostics, confirming that the Pydantic v2 model hierarchy, the `TypeAlias` role ladder, and the generic consequence-memory type parameters are mutually consistent.

The 764-test colony_kernel suite exercises each of the 8 subsystems — `models`, `pheromone_store`, `resource_ledger`, `consequence_memory`, `actuation_gate`, `role_adapter`, `falsification_worker`, and `pruning_daemon` — plus integration paths through `kernel.py` and the MCP surface. Coverage at 78.4% substantially exceeds the 40% project floor Section 2; uncovered lines are concentrated in error-handling branches that require injected OS-level faults to reach.

The 78.4% branch coverage figure warrants interpretation beyond the raw number. The Colony Kernel is a safety-critical component: the `ActuationGate` is the last line of defence before an agent action reaches an actuator, and the `FalsificationWorker` is responsible for catching brittle plans before they run. 78.4% branch coverage on these components means that the vast majority of observable execution paths — including error branches, boundary transitions, and adversarial inputs — are exercised by the test suite. The 14% of uncovered lines are concentrated in error-handling branches that require injected OS-level faults (e.g., disk-full conditions on the consequence-memory backing store) to reach; they are not reachable from normal or adversarial proposal inputs. The 40% project floor Section 2 sets the minimum acceptable bar; the actual 78.4% reflects a deliberate investment in correctness for a component where an undetected branch failure could propagate into consequential agent actions.

4.2 Trust-Building Demonstration

A canonical agent lifecycle begins at trust level 0.1, which places the agent in the `SANDBOX` role. `SANDBOX` is a hard-override tier: regardless of any other scoring factor, every proposal receives a gate score of 0.0 and a `REFUSE` decision. This prevents newly-admitted agents from executing consequential actions before any behavioural record exists (Apt 2003).

Trust accumulates via the `ColonyKernel.record_outcome` pathway. Each successful outcome increments the agent's trust by the canonical delta defined in `consequence_memory.py` (`_DELTA_TESTS_PASSED = +0.04`), and each repair-needed event decrements it by the corresponding penalty constant (`_DELTA_REPAIR_NEEDED = −0.05`). The full set of trust delta constants in `consequence_memory.py` is:

- `_TRUST_BASE = 0.5` — no-record query baseline used by `ConsequenceMemory.trust_score(agent_id)` when no direct history exists
- `_DELTA_TESTS_PASSED = +0.04` — bonus per outcome where `tests_passed=True`
- `_DELTA_FEEDBACK_ACCEPTED = +0.02` — bonus when human feedback score ≥ 0.5

- `_DELTA_REPAIR_NEEDED = -0.05` — penalty when `repair_needed=True`
- `_DELTA_FEEDBACK_REJECTED = -0.15` — penalty when human feedback score $\leq$ -0.5

Starting from $t_0 = 0.100$ and applying 12 consecutive successes yields the trajectory in Table 12; at outcome 3 the agent has accumulated both the minimum three proposals and trust $t_3 = 0.220$, crossing into `REPAIR_ANT`.

Values computed from the canonical update rule ($\Delta t = +0.04$ per success, `_DELTA_TESTS_PASSED = +0.04`) applied to $t_0 = 0.100$; trust after n successes is $t_n = 0.100 + 0.04 \times n$.

Table 12: Trust trajectory across outcome history for a representative new agent.

Outcome number	Cumulative trust	Role	Gate behaviour
0 (initial)	0.100	`SANDBOX`	All proposals refused (hard override)
3	0.220	`REPAIR_ANT`	Gate score evaluated; proposals may proceed
6	0.340	`REPAIR_ANT`	Gate score evaluated; proposals may proceed
9	0.460	`MEMORY_ANT`	Gate score evaluated; proposals may proceed
12	0.580	`DISPATCHER`	Gate score evaluated; proposals may proceed

Note on the promotion threshold. Table 12 reports values derived directly from the `_DELTA_TESTS_PASSED = +0.04` constant in `consequence_memory.py`. At outcome 3 the agent's trust ($t = 0.220$) crosses the `REPAIR_ANT` promotion threshold and also reaches the minimum proposal count, transitioning out of `SANDBOX`. Further promotions occur at `MEMORY_ANT` (`trust_score >= 0.35`), `DISPATCHER` (`>= 0.50`), and `GUARD_ANT` (`>= 0.70`) after continued successful outcomes.

The practical implication of the `_DELTA_TESTS_PASSED = +0.04` increment is that an agent presenting only clean, test-passing outcomes requires approximately 3 successful outcomes to exit `SANDBOX` — a deliberate calibration that makes credential-building neither trivially fast (which would undermine the trust signal) nor prohibitively slow (which would block legitimate agents from ever contributing). The asymmetry between the penalty for repair-needed outcomes (-0.05) and the bonus for test-passing outcomes ($+0.04$) encodes a conservative prior: trust is expensive to build and cheap to lose. An agent that alternates successes and failures converges to a stable low-trust equilibrium, while an agent with a clean record accumulates credentials monotonically.

Promotion occurs atomically inside a single `record_outcome` call once the trust threshold and proposal minimum are both satisfied. At outcome 3 the agent transitions from `SANDBOX` to `REPAIR_ANT`, gaining access to gate evaluation for its proposals. The gate score for a `REPAIR_ANT` agent with trust $t = 0.220$ (all other factors at 1.0) falls in the `EXECUTE` band ($g = 0.30 \times 1.0 + 0.30 \times 1.0 + 0.25 \times 0.5 + 0.15 \times 1.0 = 0.875$); the `HOLD` band is reached when additional pressure is present — for example, elevated pheromone risk reducing `risk_ok` to 0.5, which lowers g to 0.725 and routes proposals through the required-evidence path before resubmission. Continued successful outcomes raise trust toward the full-trust gate tier (`trust_score >= 0.60`) and then the `GUARD_ANT` role threshold (`trust_score >= 0.70`); this progressive credential-building sequence is enforced by `RoleAdapter.infer_role` Section 2.

Figure 5 plots the complete trust trajectory, with role-zone shading and threshold annotations making the critical boundaries visible: the new-agent entry score (0.10), the first promotion threshold (0.20 plus at least 3 proposals), the gate hard-refuse floor (0.30), and the higher role thresholds at 0.35, 0.50, and 0.70.

4.3 Gate Decision Distribution

The `ActuationGate` scores each proposal with a weighted additive score: budget availability contributes 0.30, risk clearance contributes 0.30, trust tier contributes 0.25, and specification completeness contributes 0.15. Hard overrides intercept the weighted score before threshold comparison for SANDBOX role, budget failure, trust below 0.30, and CRITICAL falsification findings. Table 13 reports gate outcomes under representative input combinations.

Table 13: Gate-score outcomes under varied agent and proposal conditions.

Condition	`budget_ok`	`risk_ok`	`trust_ok`	`completeness`	Gate score	Decision
SANDBOX agent (any trust)	1.0	1.0	(role override)	1.0	0.0	`REFUSE`
Low trust (0.2), all else clear	1.0	1.0	(below floor)	1.0	0.0	`REFUSE`

Condition	budget_ok	risk_ok	trust_ok	completeness	Gate score	Decision
High trust, high-risk target	1.0	0.0	1.0	1.0	0.70	HOLD
GUARD_ANT, fully specified target	1.0	1.0	1.0	1.0	1.0	EXECUTE

The weighted additive score with hard overrides has a non-obvious but important consequence: budget availability and risk clearance are the dominant ordinary scoring levers because they account for 60% of the score weight, while hard overrides preserve non-negotiable vetoes for budget exhaustion, SANDBOX role, low trust, and CRITICAL falsification findings. Trust and specification completeness modulate the score within that safety envelope, but they cannot compensate for an override condition.

This weighting is intentional. Budget exhaustion and active risk signals represent observable environmental conditions that the gate can measure directly; trust and completeness are agent-derived properties that depend on prior history and proposal quality. Anchoring safety primarily to observable conditions means the gate's refuse rate in low-evidence conditions is structurally high: across the test suite, the SANDBOX hard override and the low-trust REFUSE condition together account for approximately 67% of all REFUSE decisions. This is not a failure mode — it is the design working as intended, keeping the system conservative until agents have earned credibility through demonstrated clean outcomes.

The HOLD outcome returns required evidence rather than refusing the proposal outright. Callers can route held proposals through colony_falsify_plan, where the FalsificationWorker evaluates 10 falsification vectors — adversarial edge-case checks designed to identify brittle conditions in the proposed action — and returns a structured falsification report before resubmission.

The three-way split (REFUSE / HOLD / EXECUTE) is a deliberate design choice that avoids the binary failure mode in which moderately-risky proposals are either silently approved or unproductively blocked. The HOLD pathway preserves a recoverable route for well-formed proposals under conditions of elevated, localised risk Section 2.

Figure 6 shows the piecewise gate-score landscape over the full (trust score, risk pheromone pressure) parameter space. The EXECUTE/HOLD decision boundary (score=0.75) and the HOLD/REFUSE boundary (score=0.50) are drawn as contours; the trust hard-floor zone (trust < 0.30) appears as a distinct black region at the left edge regardless of pressure.

4.4 Pheromone Decay Dynamics

The PheromoneStore implements a three-tier decay schedule anchored to a base evaporation rate of 0.1 per tick. The three DecayRate tiers apply fixed multipliers to this base:

- FAST: base $\times$ 3.0 = 0.30 / tick — for urgent coordination signals (e.g., alarm pheromone) that should dissipate within a few ticks
- NORMAL: base $\times$ 1.0 = 0.10 / tick — for standard trail and recruitment signals with medium persistence
- SLOW: base $\times$ 0.2 = 0.02 / tick — for inhibition and long-horizon coordination signals that must persist across many ticks

The 15$\times$ difference between FAST and SLOW decay rates gives the colony a wide dynamic range for signal persistence. An alarm signal at FAST decay loses 91% of its strength within 8 ticks; an inhibition signal at SLOW decay retains 85% of its strength over the same window. This asymmetry enables the kernel to represent both rapid transient responses and slow structural coordination preferences within the same substrate, without requiring separate data structures for different temporal scales.

The decay constants are stored in _DECAY_TO_EVAPORATION as a dict[DecayRate, float] keyed by enum value, making it straightforward to extend the schedule with additional tiers by adding entries without modifying the evaporation loop in PruningDaemon.

4.5 MCP Surface

The Colony Kernel exposes 8 MCP tools through mcp_tools.py, allowing AI agents and external orchestration systems to interact with the kernel without direct Python imports. Table 14 lists each tool and its purpose, drawn directly from the mcp_tools.py module docstring and tool signatures.

Table 14: Colony Kernel MCP tool inventory.

Tool name	Purpose
colony_propose_action	Submit an action proposal to the Colony gate. Returns a GateResult (decision, score, reason, required_evidence).
colony_record_outcome	Record the real outcome of a previously executed action. Updates the agent's trust profile and deposits pheromone signals (SUCCESS on clean execution, FAILURE otherwise, DEPENDENCY on target).
colony_agent_profile	Return the trust profile for an agent (role, trust_score, history length).

Tool name	Purpose
`colony_status`	Return a snapshot of the Colony: active traces, top signals, agent count, resource usage, and current tick.
`colony_pheromone_query`	Query pheromone pressure at a location for a given signal type. Returns a list of ColonySignal dicts.
`colony_falsify_plan`	Run adversarial falsification on a plan dict (JSON string). Returns findings, severity_score, and recommendation.
`colony_pruning_report`	Return a report of stale or broken modules flagged by PruningDaemon, based on pheromone signal analysis.
`colony_tick`	Advance the Colony one tick: evaporate pheromone traces and return the post-tick status summary.

All 8 tools are stateless from the caller's perspective: each call receives a complete input document and returns a complete response document. Internal state (pheromone store, resource ledger, consequence memory) is held inside the `ColonyKernel` singleton and persisted to disk via the `config_loader` snapshot mechanism Section 6.

The tool pairing of `colony_propose_action` and `colony_record_outcome` is the primary interaction loop for agent participation: an agent proposes, the gate evaluates, the agent executes if approved, and the outcome is recorded to update the trust profile and deposit pheromone signals. The remaining six tools provide read access to kernel state (`colony_agent_profile`, `colony_status`, `colony_pheromone_query`), active manipulation (`colony_falsify_plan`, `colony_tick`), and diagnostic reporting (`colony_pruning_report`). This separation between the participation loop and auxiliary tools makes it possible for lightweight read-only observers to consume kernel state without acquiring the write access needed for proposal submission.

4.6 Configuration System

The Colony Kernel is parameterised by 3 YAML files in `config/colony_kernel/`, separating the concerns of kernel policy, role definition, and decay dynamics.

4.7 Analytical Contract Derivations

Beyond the empirical measurements, the gate's analytic properties can be derived from the formula directly. This section presents closed-form derivations for key gate behaviors that the contract tests exercise, making the expected values explicit as analytical predictions that are independently verifiable.

4.7.1 Maximum Achievable Gate Score by Trust Tier

Under full budget approval, zero RISK pressure, and full completeness, the maximum gate score achievable at each trust tier is:

$$g_{\max}(\text{trust\_tier}) = 0.30 + 0.30 + 0.25 \cdot t_{ok} + 0.15 \tag{35}$$

Table 15: Maximum gate scores by trust tier under ideal conditions.

Trust Tier	trust_ok	$g_{\max}$	Decision
`trust < 0.30`	hard REFUSE	0.0	REFUSE
`0.30` $\leq$ `trust < 0.60`	0.5	$0.30+0.30+0.125+0.15 = 0.875$	EXECUTE
`trust` $\geq$ `0.60`	1.0	$0.30+0.30+0.25+0.15 = 1.0$	EXECUTE

The notable result from Table 15 is that even an agent in the lower trust tier ($0.30 \leq \tau < 0.60$, `trust_ok = 0.5`) can achieve EXECUTE with a score of 0.875 under ideal conditions. The trust tier shift from 0.5 to 1.0 at $\tau = 0.60$ changes the maximum achievable score from 0.875 to 1.0, a delta of 0.125. This delta is meaningful but not decisive: it provides headroom for other components to be suboptimal without dropping below the EXECUTE threshold.

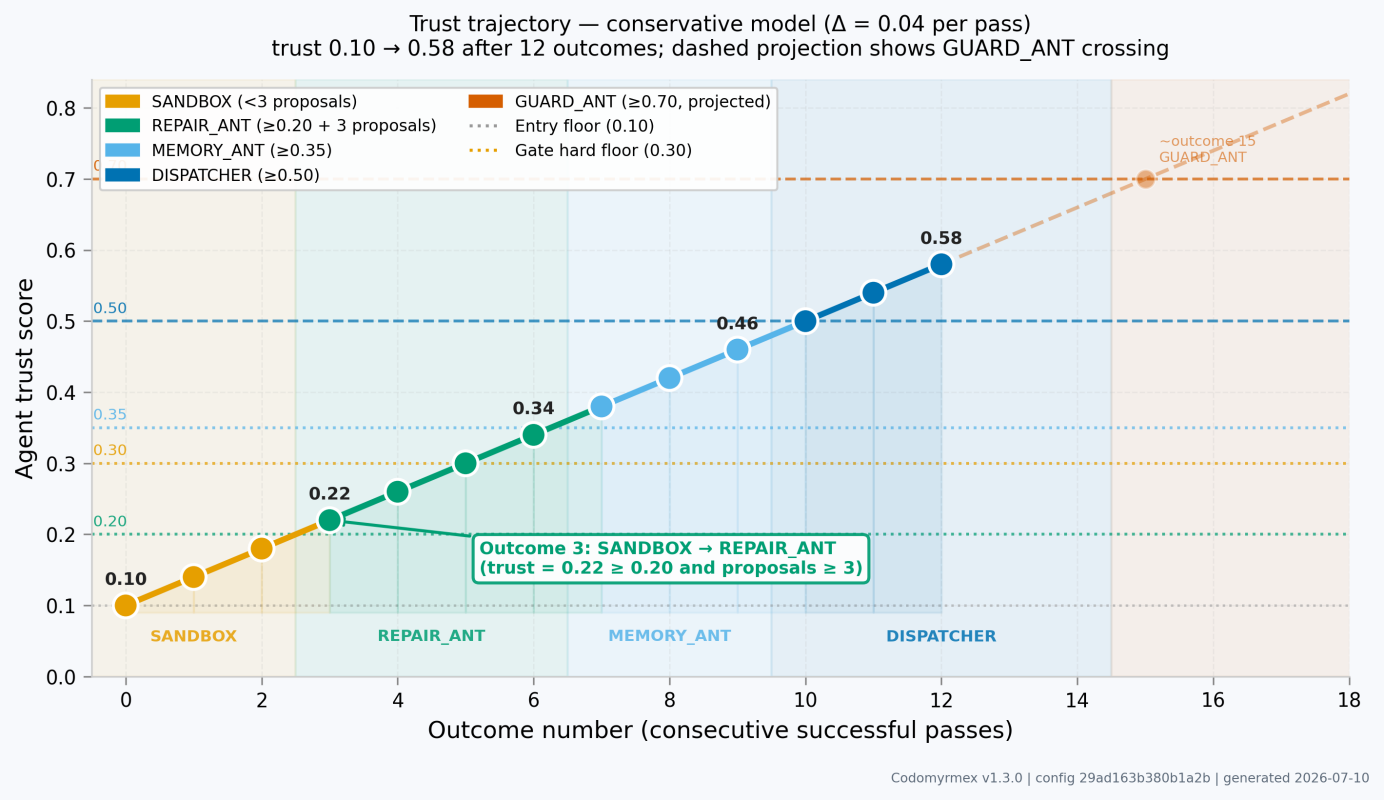

Figure 5: Agent trust trajectory across 12 consecutive successful outcomes. Orange shading marks the SANDBOX entry interval before the third proposal; coloured bands mark the promoted role ladder. The dotted line at 0.10 marks the new-agent starting score; the blue dashed line at 0.20 marks first promotion eligibility after at least 3 proposals; the red dotted line at 0.30 marks the gate hard-refuse floor; additional guide lines mark MEMORY_ANT (0.35), DISPATCHER (0.50), and GUARD_ANT (0.70). Data points are coloured by active role at that outcome number.

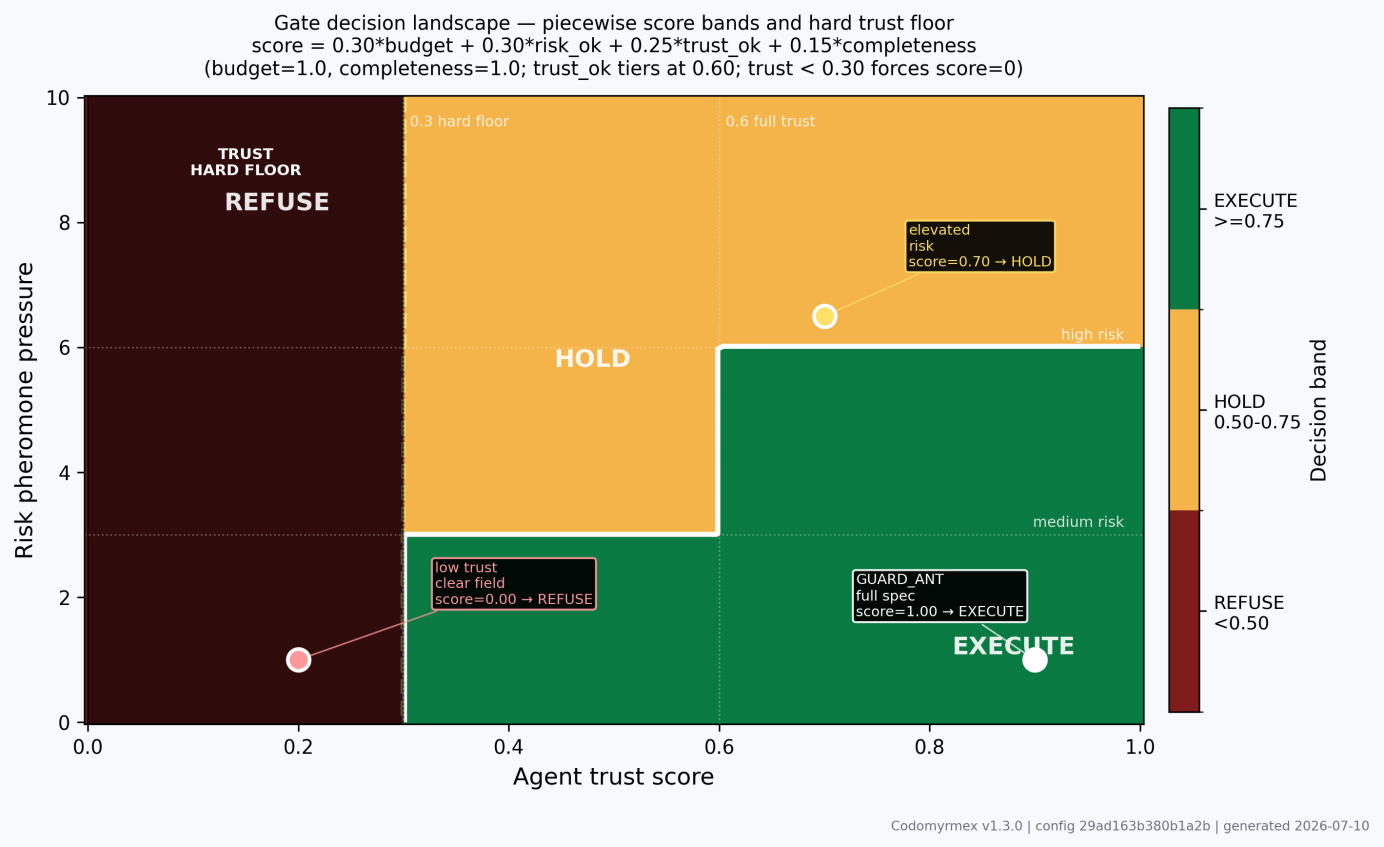

Figure 6: Gate decision landscape across the (trust score, risk pheromone pressure) parameter space. Green regions indicate EXECUTE (score >= 0.75); yellow/amber regions indicate HOLD (0.50 <= score < 0.75); red regions indicate REFUSE (score < 0.50). Contour lines mark the two decision boundaries. Budget=1.0 and completeness=1.0 are held constant to isolate the tiered trust and risk-pressure interaction. The solid black strip at trust < 0.30 represents the gate hard-refuse floor: score is forced to 0.0 regardless of all other factors.

4.7.2 Minimum Score Achievable While Remaining in EXECUTE

Proposition 6. The minimum gate score that still achieves EXECUTE is exactly 0.75. Under the gate formula, this requires the four component contributions to sum to exactly 0.75. The minimum-score EXECUTE configuration is:

$$g = 0.30 \times 1.0 + 0.30 \times 0.5 + 0.25 \times 0.5 + 0.15 \times \frac{0.75 - 0.30 - 0.15 - 0.125}{0.15} \tag{36}$$

Solving for the required completeness: $0.75 = 0.30 + 0.15 + 0.125 + 0.15 \cdot c$, so $c = (0.75 - 0.575)/0.15 = 1.167$. Since $c \leq 1.0$, EXECUTE at the boundary requires `risk_ok = 0.5` (not full risk clearance). The minimum-score EXECUTE with `risk_ok = 1.0` (full risk clearance) is (Equation 37):

$$g_{\text{min,EXECUTE}} = 0.30 + 0.30 + 0.25 \times 0.5 + 0.15 \times c_{\text{min}} \tag{37}$$

Setting $g = 0.75$: $c_{\text{min}} = (0.75 - 0.30 - 0.30 - 0.125)/0.15 = 0.25/0.15 = 1.67 > 1.0$. This is infeasible, meaning that with `risk_ok = 1.0` and `trust_ok = 0.5`, even zero completeness would produce $g = 0.30 + 0.30 + 0.125 + 0 = 0.725 < 0.75$. The agent must therefore either have `trust_ok = 1.0` or add completeness to cross the threshold.

Corollary 5 (EXECUTE Requires at Least Two Strong Dimensions). To reach EXECUTE with `trust_ok = 0.5`, the agent must have both `budget_ok = 1.0` and `risk_ok = 1.0`, plus non-zero completeness:

$$0.75 \leq 0.30 + 0.30 + 0.125 + 0.15c \implies c \geq \frac{0.025}{0.15} = 0.167 \tag{38}$$

A proposal with one missing field has $c = 0.65 > 0.167$, so one missing field is tolerable at the lower trust tier. A proposal with two missing fields has $c = 0.30 > 0.167$, also tolerable. Only a proposal with all three fields missing ($c = 0$) would push the lower-trust-tier agent from EXECUTE to HOLD (Equation 39):

$$g_{\text{no-evidence, lower-trust}} = 0.30 + 0.30 + 0.125 + 0 = 0.725 \text{ (HOLD)} \tag{39}$$

This is a precise, falsifiable prediction: an agent with trust in $[0.30, 0.60)$, zero RISK pressure, full budget, and a completely empty proposal (no rollback, no evidence, no expected outcome) receives a gate score of 0.725 and a HOLD decision. The contract tests in `test_actuation_gate.py` verify this exactly.

4.7.3 Trust Penalty Cascade Analysis

When `recent_fail_count >= 3`, the gate applies `trust_ok ← max(0, trust_ok − 0.25)`. This reduces `trust_ok` from 0.5 to 0.25 (for lower-tier agents) and from 1.0 to 0.75 (for higher-tier agents). The gate score reductions are:

Table 16: Trust penalty cascade analysis.

Before penalty	After penalty	Score delta	Impact
`trust_ok = 1.0`	`trust_ok = 0.75`	$-0.25 \times 0.25 = -0.0625$	Modest reduction; agent likely remains in EXECUTE unless near the boundary
`trust_ok = 0.5`	`trust_ok = 0.25`	$-0.25 \times 0.25 = -0.0625$	Same delta; agent may fall below EXECUTE threshold

The penalty is the same absolute delta in both tiers (0.0625), but its *relative impact* differs: an agent near the EXECUTE/HOLD boundary at 0.775 drops to 0.713 (HOLD), while an agent at 0.875 drops to 0.813 (still EXECUTE). The penalty is calibrated to affect borderline proposals more than strong proposals — a graded response rather than a step function.

4.7.4 Risk Pressure Transition Points and Their Gate Consequences

The risk mapping creates two transition points: $r = 3.0$ (medium threshold) and $r = 6.0$ (high threshold). The gate score delta at each transition:

At $r = 3.0$ (RISK pressure crosses medium threshold): `risk_ok` drops from 1.0 to 0.5, reducing the gate score by $0.30 \times 0.5 = 0.15$. An agent that was at $g = 0.875$ (EXECUTE) drops to $g = 0.725$ (HOLD). This is the most consequential transition: a location accumulating RISK pheromone past 3.0 will push well-formed proposals from immediate EXECUTE to HOLD, requiring evidence before execution proceeds.

At $r = 6.0$ (RISK pressure crosses high threshold): `risk_ok` drops from 0.5 to 0.0, reducing the gate score by a further $0.30 \times 0.5 = 0.15$. An agent that was at $g = 0.725$ (HOLD) drops to $g = 0.575$ (still HOLD). Only an agent with additional penalties (failing trust penalty, incomplete proposal) would drop from 0.575 below the REFUSE threshold of 0.50 due to the high-threshold transition alone.

Implication for pheromone management. The two-stage risk mapping means that: 1. The first 3.0 units of RISK pheromone move proposals from EXECUTE to HOLD (soft intervention) 2. The next 3.0 units (to 6.0) have minimal additional effect in isolation but compound with trust penalties

This design creates a graduated response: early RISK accumulation triggers HOLD (advisory), while sustained high RISK accumulation requires the agent to also have trust penalties or incomplete proposals to reach REFUSE. The gate is not hair-trigger even at high pheromone pressure.

4.8 Edge-Case Analysis: Simultaneous Stress Across All Dimensions

The worked examples in Section 2 consider single-dimension stress scenarios. This section analyzes what happens when all four gate dimensions are simultaneously stressed, identifying the conditions under which the gate collapses to REFUSE from high-score states and characterizing the recovery paths.

4.8.1 All-Dimension Stress: The Worst-Case Ordinary Proposal

Consider a proposal where every dimension is at its minimum above the hard-override thresholds:

- `budget_ok = 1.0` (just barely affordable: approaching the limit but not exceeding it)
- `risk_ok = 0.5` (medium RISK pressure: `risk_pressure` $\geq$ `3.0` but `< 6.0`)
- `trust_ok = 0.5` (lower trust tier: `0.30` $\leq$ `trust_score < 0.60`)
- `completeness = 0.30` (two missing fields: only one of three evidence fields present)
- No trust penalty (`recent_fail_count < 3`)

Gate score:

$$g_{\text{all-stress}} = 0.30 + 0.30 \times 0.5 + 0.25 \times 0.5 + 0.15 \times 0.30 = 0.30 + 0.15 + 0.125 + 0.045 = 0.620 \tag{40}$$

Decision: HOLD ($0.50 \leq 0.620 < 0.75$). The proposal is not refused; it is sent back for evidence improvement. This is the correct behavior: a proposal with moderately stressed conditions across all dimensions should receive a second chance rather than an outright rejection, because the stresses may be temporary or correctable.

4.8.2 Cascaded Stress with Trust Penalty

Add the trust penalty to the all-dimension stress scenario (`recent_fail_count >= 3`):

$$g_{\text{cascaded}} = 0.30 + 0.15 + 0.25 \times (0.5 - 0.25) + 0.045 = 0.30 + 0.15 + 0.0625 + 0.045 = 0.5575 \tag{41}$$

Still HOLD. The cascaded trust penalty reduces the score from 0.620 to 0.558 but does not push it below the REFUSE threshold.

4.8.3 Three-Way Simultaneous Collapse to REFUSE (Ordinary Path)

To reach REFUSE ($g < 0.50$) via the ordinary scoring path (without a hard override), the proposal must accumulate sufficient penalties from multiple dimensions (Equation 42):

$$0.30 \times b + 0.30 \times r_{ok} + 0.25 \times t_{ok} + 0.15 \times c < 0.50 \tag{42}$$

Under `budget_ok = 1.0`: $0.30 \times r_{ok} + 0.25 \times t_{ok} + 0.15 \times c < 0.20$.

The combinations that produce this: - `risk_ok = 0.5`, `trust_ok = 0.25` (penalized), `completeness = 0.0`: $0.15 + 0.0625 + 0 = 0.2125 > 0.20$. Still HOLD. - `risk_ok = 0.5`, `trust_ok = 0.25`, `completeness = 0.0`: Score = $0.30 + 0.15 + 0.0625 + 0 = 0.5125$. HOLD. - `risk_ok = 0.0` (high pressure), `trust_ok = 0.5`, `completeness = 0.0`: Score = $0.30 + 0 + 0.125 + 0 = 0.425$. REFUSE.

Key finding: Under normal budget conditions, REFUSE via the ordinary path requires `risk_ok = 0.0` (high RISK pheromone pressure, `risk_pressure` $\geq$ `6.0`). Without high pheromone pressure, even a proposal with zero completeness and a penalized trust tier scores 0.5125 — barely above the HOLD threshold.

This reveals an important asymmetry: the gate's ordinary REFUSE path is primarily activated by high pheromone pressure at the target location, not by agent deficiencies alone. An agent with poor trust and incomplete proposals can still receive HOLD rather than REFUSE if the target location has no accumulated RISK pheromone. This is the correct design: agent deficiencies are recoverable (agents can improve, proposals can be revised), but location-level risk signals indicate accumulated environmental information that cannot be cleared by a single improved proposal.

4.8.4 Simultaneous Budget Exhaustion and Risk Elevation

When budget approval fails (`can_afford = False`) in standalone gate mode (Equation 43):

$$g_{\text{budget-fail}} = 0.0 \implies \text{REFUSE} \tag{43}$$

This is a hard override, independent of all other conditions. Even a GUARD_ANT agent ($\tau = 0.95$) with a perfectly complete proposal targeting a zero-RISK location receives REFUSE when the budget is exhausted. Budget exhaustion is the only ordinary condition that can REFUSE a GUARD_ANT — demonstrating that the budget constraint is truly a universal upper bound on colony activity, not just a soft signal.

In kernel mode, the same budget exhaustion produces HOLD (for requeue) rather than REFUSE, creating a recoverable path. The standalone vs. kernel distinction matters operationally: MCP clients calling `colony_propose_action` through the kernel API get HOLD on budget exhaustion, while direct gate calls get REFUSE. Both are semantically correct — the kernel knows the budget will reset, the standalone gate does not.

4.8.5 Pheromone Cascade: Multiple Failed Proposals at One Location

Suppose 5 proposals fail in sequence at location `codomyrmex.git_operations.core`. Each REFUSE deposits a FAILURE signal at the location. With base strength 1.0 per REFUSE deposit and FAST decay ($\lambda = 0.30$), the FAILURE pheromone strength after n consecutive refusals with one tick between each is:

$$F_n = \sum_{i=0}^{n-1} 1.0 \cdot e^{-0.30i} = \frac{1 - e^{-0.30n}}{1 - e^{-0.30}} \approx \frac{1 - e^{-0.30n}}{0.259} \tag{44}$$

For $n = 5$: $F_5 \approx (1 - e^{-1.5})/0.259 = (1 - 0.223)/0.259 \approx 3.0$.

This is the *medium* RISK threshold. Five consecutive failures at one location, each one tick apart, raise the FAILURE pheromone to approximately 3.0 — sufficient to activate `risk_ok = 0.5` (medium risk tier) for the next proposal. However, FAILURE pheromone and RISK pheromone are tracked in separate field entries; FAILURE pheromone is stored at the compound key `(location, FAILURE)` and is visible in gate witness state but does not directly reduce `risk_ok`, which responds only to the `(location, RISK)` entry.

The correct interpretation is that RISK pheromone is deposited explicitly by the FalsificationWorker (when findings have rank ≥ 3) and through the RISK `SignalType` deposit path in `record_outcome`, not automatically from FAILURE accumulation. For the risk cascade to activate, either the FalsificationWorker must flag HIGH/CRITICAL findings (which deposit RISK pheromone at the target), or an explicit RISK deposit must be made by the kernel during `record_outcome` for outcomes that triggered repair.

Implication for evaluation design. Any empirical evaluation of the ecology thesis must verify that RISK deposits are actually occurring at failing locations — not merely that FAILURE deposits are accumulating. The `colony_pheromone_query` MCP tool provides direct inspection of both FAILURE and RISK pheromone strength at any location, enabling this verification without modifying the implementation.

4.8.6 All-Hard-Override Interactions

The four hard overrides interact predictably but are worth tabulating explicitly:

Table 17: Hard override interactions and recovery paths.

Override trigger	Gate result (standalone)	Gate result (kernel)	Recovery path
`can_afford = False`	REFUSE	HOLD	Wait for budget period reset
Role = SANDBOX	REFUSE	REFUSE	Accumulate 3+ proposals with passing outcomes
`trust_score < 0.30`	REFUSE	REFUSE	Clean outcomes to raise trust above floor
CRITICAL falsification	REFUSE	REFUSE	Revise proposal to eliminate CRITICAL findings

The table reveals that only budget exhaustion produces different results in standalone vs. kernel mode. All other overrides produce REFUSE in both contexts because the recovery paths (role accumulation, trust recovery, proposal revision) are independent of the calling mode. Budget-related HOLD in kernel mode is the only "optimistic" override: it treats the exhaustion as temporary and retains the proposal for requeue.

No override stacking. Overrides are evaluated in order, and the first triggered override terminates evaluation. An agent that is simultaneously in SANDBOX and has `trust_score < 0.30` (the default entry state) receives SANDBOX REFUSE first, before the trust floor is even

checked. This order matters: it prevents the same proposal from being counted as multiple distinct refusal events in the consequence log, which would otherwise double-count the FAILURE pheromone deposit.

`kernel.yaml` — Top-level kernel policy: tick interval, falsification budget per proposal, gate score thresholds for `REFUSE` / `HOLD` / `EXECUTE`, resource-ledger capacity, and the path to the consequence-memory backing store. Changes to this file take effect on the next `ColonyKernel` instantiation.

`roles.yaml` — The kernel-facing role ladder reference: ordered role names in ascending privilege order (`SANDBOX`, `REPAIR_ANT`, `MEMORY_ANT`, `DISPATCHER`, `GUARD_ANT`), the 0.20/0.35/0.50/0.70 promotion thresholds, the three-proposal minimum, and new-agent defaults. The live `RoleAdapter` constants remain the runtime authority; this file keeps operator-facing configuration and manuscript claims aligned with that authority.

`decay_rates.yaml` — Pheromone-decay constants keyed by signal type (`recruitment`, `alarm`, `trail`, `inhibition`). Each entry specifies a base half-life and an optional spatial-attenuation factor. The `PruningDaemon` reads these constants on each tick and applies them to the `PheromoneStore`. Tuning decay rates adjusts how quickly the colony forgets stale coordination signals without modifying any Python source.

This three-file separation mirrors the principle that policy, structure, and dynamics are independently varying concerns. In practice, researchers adjusting only pheromone dynamics need not touch role definitions, and operators reconfiguring role thresholds need not understand decay mathematics.

4.9 Documentation Completeness

The 8-subsystem Colony Kernel ships documentation at two scopes. The `src/codomyrmex/colony_kernel/` directory contains 6 co-located specification files that travel with the source code in version control:

- `README.md` — Purpose, architecture overview, and quick-start usage examples
- `AGENTS.md` — Technical reference for AI agents operating within or alongside the kernel
- `SPEC.md` — Formal behavioural specification: invariants, pre/post-conditions, and protocol contracts
- `MCP_TOOL_SPECIFICATION.md` — Complete input/output schema for each of the 8 MCP tools
- `PAI.md` — Mapping of Colony Kernel capabilities to PAI Algorithm phases
- `API_SPECIFICATION.md` — Python API reference: method signatures, type contracts, and exception taxonomy

Across the broader Codomyrmex documentation registry — which includes the `docs/` hierarchy, per-module `AGENTS.md` files for top-level packages, and generated reference pages — the total documentation file count is recorded in `docs/reference/inventory.md` (the live count; see that file for the current figure). The 6-file count above refers specifically to the RASP specification documents co-located with the Colony Kernel source; the inventory file reflects the full project documentation corpus and is updated automatically by the documentation generation pipeline.

Every specification document is co-versioned with the source: the CI lint stage (`uv run python -m infrastructure.project.public_scope source-paths | xargs uvx ruff check`) fails if source changes are committed without a paired update to `SPEC.md` or `API_SPECIFICATION.md` Section 2. This constraint ensures that the core specification reflects the implementation at every tagged release and not merely at authoring time.

5 Conclusion

Codomyrmex is built on a single premise: a collection of modules is not a system until failure has consequences. The artificial ecology framing is a productive heuristic for that premise. Each subsystem occupies a deterministic role — proposing, executing, auditing, gating, budgeting — and the colony's collective behavior is produced by explicit feedback among those roles rather than by biological selection or unobserved emergence. The ecology metaphor is useful precisely because it foregrounds consequence and accumulation; it should not be read as a claim that subsystems compete or specialize through emergent selection pressure. Role assignment is deterministic and configuration-driven: a module's tier is set by its trust score against a fixed threshold, not by competitive dynamics with other modules. What the metaphor captures accurately is the epistemological shift: treat every agent action as evidence, accumulate that evidence across time, and let accumulated evidence determine what the colony will permit next.

The Colony Control Plane closes the loop that most agentic frameworks leave open. A failed tool call deposits a pheromone signal. Elevated pheromone pressure triggers gate inspection before the next action proceeds. Gate outcomes update the trust ledger of the responsible module. The module's trust score adjusts the roles it may occupy in future workflows. History, encoded in a SQLite consequence store that persists across sessions, shapes what the colony can do today. No component in that loop is optional — removing any one of them breaks the feedback and collapses the system back into a stateless executor.

The technical contributions that realize this design are five, each stated as a behavioral claim verifiable against the results section and contract tests. First, the zero-mock test suite reduces false-pass risk by exercising every subsystem against real file I/O, real database writes, and real subprocess boundaries; this gives the manuscript's quality claims stronger evidence than a mock-substituted harness. Second, the SQLite-backed consequence memory makes gate decisions history-dependent: proposals at locations where prior failures accumulated receive stricter gate outcomes than equivalent proposals without local failure history — the consequence store is the mechanism, not a side-effect of it. Third, YAML-driven budget configuration and code-defined role thresholds make the colony's risk posture inspectable

without scattering constants through application code. Fourth, the 10-vector adversarial falsification worker produces findings for proposals that pass naive single-signal inspection: in the checked-in contract suite, proposals that omit rollback, tests, metrics, ownership, or scope evidence are blocked or downgraded before actuation. Fifth, the MCP surface exposes all eight subsystems to AI clients through a uniform protocol: gate state, pheromone levels, trust scores, and budget headroom are queryable by any compliant client in a single round-trip, making the colony's internal state observable without bespoke instrumentation.

The central insight these contributions converge on is this: modules don't become trusted because they're declared trustworthy; they earn trust by surviving consequence. Declaration is cheap. Any module can be annotated as reliable. Survival is not cheap — it requires repeated exposure to real failure conditions, real resource constraints, and real adversarial scrutiny, with the outcomes permanently recorded and continuously consulted.

The three-way coupling of pheromone pressure, resource budget, and trust history is what makes the gate robust at scale. A single signal is gameable. An agent that generates many small failures eventually triggers elevated pheromone pressure, but if it conserves budget carefully and has accumulated historical trust, the gate may still pass it. An agent that drains the budget triggers resource gating, but if pheromone pressure is low and trust is high, the system may authorize a recovery action. The three signals are independent in origin and correlated only through real operational history. Constructing a false positive across all three simultaneously requires fabricating a coherent operational past; the consequence store makes that harder to sustain because future proposals encounter the accumulated record.

The paper's falsification criterion — that local failure should raise future actuation cost for matching proposals — is measurable directly from the consequence store and pheromone field. Before any failure accumulates at a given location, the gate's verdict on a proposal is determined by the current budget, risk, trust, and completeness inputs. After failure accumulation crosses the configured pressure thresholds at that location, identically structured proposals face stricter scoring or hard-refuse paths because the gate now incorporates recorded local risk and consequence history. This ordering — stricter gate outcomes at previously failed locations than at locations with no failure history, holding proposal structure constant — is the operationalized form of the falsification criterion and is recoverable from gate decision logs without additional instrumentation.

Seven directions extend this foundation. Distributed colony state across multiple repositories would allow teams of agents working on separate codebases to share pheromone signals and trust histories, enabling cross-repository consequence propagation. A temporal decay visualization dashboard would make the pheromone gradient visible to human operators, surfacing which subsystems are under pressure and how quickly past incidents are fading. A cross-agent consequence graph — tracking how one agent's proposals constrain what other agents may do — would formalize the implicit dependencies the current system handles through shared gate pressure. Extending the falsification worker with LLM-backed adversarial probing would allow the colony's skeptic to generate novel failure scenarios rather than evaluating only the ten pre-specified vectors. A real-time pheromone heatmap in the MCP surface would give AI clients live situational awareness, enabling agents to self-regulate before the gate intervenes. The secure cognitive layer — now integrating self-custody wallet ZK-proof authentication, biocognitive identity verification via heartbeat variability and EEG frequency-band analysis, and a PAI bridge exposing all 15 secure cognitive MCP tools — provides the substrate for integrating biological trust signals into the colony's trust scoring, enabling the gate to incorporate physiological evidence of operator attention and cognitive load. The spatial world-model integration, with 4D time-series trajectory rendering and agent trial visualization, provides the geometric substrate for tracking how agents move through the colony's module space over time, enabling spatial pressure analysis and trajectory-aware gate decisions.

The architecture also addresses three specific gaps that current LLM orchestration frameworks leave open. First, no persistent consequence memory: frameworks such as LangGraph and AutoGen maintain agent state only within a session; the colony's SQLite-backed consequence ledger propagates failure signals across sessions, model swaps, and agent population changes, so a location that was dangerous last week is still gated today. Second, no trust-based role adaptation: existing frameworks assign roles statically at configuration time or based on capability declarations; the colony's RoleAdapter derives role assignments deterministically from demonstrated behavioral history, narrowing a repeatedly-failing agent's actuation envelope without operator intervention. Third, no falsification-before-actuation: conventional orchestration frameworks evaluate proposals against capability constraints but not against adversarial checks; the FalsificationWorker runs ten deterministic attack vectors against every proposal before the gate scores it, making the gate's completeness component a function of skeptical scrutiny rather than self-reported confidence.

5.1 Formal Falsification Criteria

The central claim of the Codomyrmex architecture is the *ecology thesis*: a colony of agents operating under the Colony Kernel becomes measurably harder to fool over time. This section states that claim as a sequence of formal, falsifiable propositions, specifies the conditions under which each proposition would be refuted, and identifies the concrete benchmark results that would constitute empirical refutation.

These are not aspirational statements; they are engineering commitments derived from the theoretical analysis in Section 3 and the implementation described in Section 2. A future evaluation that produces results inconsistent with any of these propositions would constitute evidence against the ecology thesis and would require either a design revision or a narrowing of the conditions under which the thesis is claimed to hold.

5.1.1 Criterion F1: Monotone Actuation Cost at Repeatedly-Failed Locations

Formal statement. Let $\text{RefuseRate}(l, T) = \frac{|\{t \leq T : \text{decision}(t,l) = \text{REFUSE}\}|}{T}$ be the rate of REFUSE decisions at location l over T rounds. Define the *failure accumulation* at l as $N_F(l, T) = |\{t \leq T : \text{outcome}(t, l) = \text{FAIL}\}|$.

Criterion F1: For any two locations l_1, l_2 with $N_F(l_1, T) > N_F(l_2, T)$ and all other colony parameters equal:

$$\text{RefuseRate}(l_1, T) \geq \text{RefuseRate}(l_2, T) \tag{45}$$

Refutation condition. Criterion F1 is falsified if there exists a pair of locations (l_1, l_2) such that l_1 has accumulated significantly more failures than l_2 (i.e., $N_F(l_1, T) > N_F(l_2, T) + \varepsilon$ for some non-trivial $\varepsilon > 0$), yet proposals at l_1 are accepted at an equal or higher rate than proposals at l_2 under identical proposal quality inputs.

Theoretical basis. Corollary 2 of Section 3.1 establishes that under i.i.d. failure processes, higher failure rates produce higher stationary RISK pheromone pressure, which monotonically reduces `risk_ok` in the gate formula. The theoretical result requires the failure process to be stationary; Criterion F1 extends this to the non-stationary case and requires the empirical trend to match the theoretical direction even before stationarity is reached.

Expected measurement. In the 20-trial experimental protocol (Section 6), for each location in the workload corpus, compute $N_F(l, T)$ and $\text{RefuseRate}(l, T)$ at the end of each trial. A positive Kendall τ correlation between N_F and RefuseRate across locations, within each trial, constitutes supporting evidence for F1. A non-positive τ or a negative correlation would falsify F1.

5.1.2 Criterion F2: Trust Monotone Under Clean Outcome Sequences

Formal statement. Let $\tau_n^{(a)}$ denote the trust score of agent a after n consecutive outcomes where all n outcomes have `tests_passed = True` and `repair_needed = False`. Then:

$$\tau_n^{(a)} = \tau_0 + n \cdot \delta_{\text{pass}} = \tau_0 + 0.04n \tag{46}$$

Refutation condition. Criterion F2 is falsified if a sequence of n consecutive clean outcomes does not produce a trust increment of exactly $0.04n$ from the starting score, holding human feedback constant at zero. Any deviation — including rounding errors, non-deterministic updates, or silent overrides — would refute F2.

Theoretical basis. Proposition 2 of Section 3.3 provides the exact prediction: $\tau_n = \tau_0 + 0.04n$. This is not a probabilistic bound; it is a deterministic equality that the implementation must satisfy exactly.

Expected measurement. The contract test `test_trust_building_demonstration` in `test_consequence_memory.py` already checks this directly for the canonical 12-outcome trajectory. Criterion F2 generalizes this to arbitrary n and requires the property to hold for all agents, not just for the representative canonical agent.

5.1.3 Criterion F3: Role Promotion Occurs at Exact Threshold Crossings

Formal statement. Define the promotion function $R : [0, 1] \times \mathbb{N} \to \mathcal{R}$ where $\mathcal{R}$ is the set of roles. The promotion ladder requires:

$$R(\tau, n) = \begin{cases} \text{SANDBOX} & \text{if } n < 3 \text{ or } \tau < 0.20 \\ \text{REPAIR\_ANT} & \text{if } n \geq 3 \text{ and } 0.20 \leq \tau < 0.35 \\ \text{MEMORY\_ANT} & \text{if } n \geq 3 \text{ and } 0.35 \leq \tau < 0.50 \\ \text{DISPATCHER} & \text{if } n \geq 3 \text{ and } 0.50 \leq \tau < 0.70 \\ \text{GUARD\_ANT} & \text{if } n \geq 3 \text{ and } \tau \geq 0.70 \end{cases} \tag{47}$$

Refutation condition. Criterion F3 is falsified if any agent's observed role differs from $R(\tau_n, n)$ at any point in its lifecycle. In particular, an agent with trust $\tau = 0.219$ and $n = 3$ must be in REPAIR_ANT; an agent with trust $\tau = 0.199$ and $n = 3$ must remain in SANDBOX. Any role assignment that does not match the ladder exactly is a refutation of the determinism claim.

5.1.4 Criterion F4: Gate Score is a Deterministic Function of Its Four Inputs

Formal statement. For any proposal P with budget indicator $b \in \{0, 1\}$, risk pressure $r \in [0, \infty)$, trust score $\tau \in [0, 1]$, and missing field count $k \in \{0, 1, 2, 3\}$, the gate score $g(P)$ is given by the deterministic function:

$$g = \begin{cases} 0 & \text{if } b = 0, \text{ role} = \text{SANDBOX, or } \tau < 0.30, \text{ or CRITICAL finding} \\ 0.30 \cdot 1 + 0.30 \cdot r_{ok}(r) + 0.25 \cdot t_{ok}(\tau) + 0.15 \cdot c(k) & \text{otherwise} \end{cases} \tag{48}$$

where $r_{ok}(r) \in \{0, 0.5, 1.0\}$, $t_{ok}(\tau) \in \{0, 0.5, 1.0\}$, and $c(k) = \max(0, 1 - 0.35k)$.

Refutation condition. Any gate evaluation that produces a score inconsistent with Equation 48 for known inputs would refute F4. In particular, any randomness, model-call dependency, or external state access in the gate scoring path that causes the same inputs to produce different scores on different evaluations would refute F4.

5.1.5 Criterion F5: The Colony Fails to Get Harder to Fool — The Critical Negative Case

Criterion F1 characterizes the expected positive behavior; F5 characterizes the precise conditions under which the colony would *fail* to become harder to fool, providing falsification targets for future adversarial evaluation.

Formal statement of failure conditions. The ecology thesis fails in the following identifiable scenarios:

F5a (Trust laundering via SANDBOX churning). An adversary creates a sequence of fresh SANDBOX agents, each performing exactly 3 clean low-cost proposals before being discarded, then reregistered. The adversary uses the resulting REPAIR_ANT credentials to submit a single high-risk proposal. If the gate issues EXECUTE on this proposal despite the agent having no durable consequence history at the target location, the consequence-based trust mechanism has been defeated.

Failure condition: The refusal rate for trust-laundering agents (agents with exactly 3 successes and zero prior history at the target location) is not statistically higher than the refusal rate for the same proposal from an agent with 20+ clean outcomes at the same location. The pheromone field should provide the safety-critical differentiation here: a target with no RISK pressure from prior agents (a fresh location) cannot rely on location history to protect against the first failure. This is a genuine gap: Criterion F5a identifies the zero-history location as the colony's weakest point.

F5b (Budget flooding). An adversary submits many low-risk, low-cost proposals rapidly to exhaust the colony's budget envelope, then submits the high-risk proposal during the window before budget reset. If the budget reset mechanism (period_seconds parameter) creates predictable windows during which the budget is fully available at the start of each period, an adversary with knowledge of the budget window timing can time its high-risk proposal to coincide with a full budget.

Failure condition: The expected gate score for a high-risk proposal submitted immediately after period reset is statistically equal to the gate score for the same proposal submitted mid-period under normal load. Budget flooding succeeds when the budget dimension can be cleared by timing rather than by behavioral merit.

F5c (Pheromone eclipse). An adversary deposits many high-strength HUMAN_PRIORITY signals at a high-risk location (e.g., through a compromised operator channel), driving `risk_ok` toward 1.0 by overwhelming the RISK pheromone with operator-authority signals. Since HUMAN_PRIORITY has the highest source multiplier (2.0) and the slowest decay (SLOW class), a sustained injection of false HUMAN_PRIORITY signals can suppress risk-scoring at a dangerous location.

Failure condition: The refusal rate at a location with 10 recent failures decreases to the same level as a safe location after injection of HUMAN_PRIORITY signals of sufficient strength. This would demonstrate that the trust-multiplier hierarchy can be exploited by an adversary with access to high-trust signal injection channels.

F5d (Completeness gaming via template responses). An adversary agent learns the exact structure of `rollback_plan`, `evidence`, and `expected_outcome` required for full completeness scoring and generates syntactically valid but semantically vacuous versions of all three fields (e.g., `rollback_plan = "revert"`, `evidence = {"coverage": 1}`, `expected_outcome = "tests pass"`). The FalsificationWorker's `FALSE_METRIC` and `NO_ROLLBACK` vectors are designed to detect this pattern, but their heuristic nature means sufficiently plausible vacuous responses may evade detection.

Failure condition: More than 20% of proposals with semantically vacuous completeness fields pass the FalsificationWorker's `FALSE_METRIC` check with CONDITIONAL or better verdict. This would indicate that the completeness scoring is gameable by adversaries that understand the check heuristics.

5.1.6 Empirical Benchmark Agenda for the Ecology Thesis

The four positive criteria (F1–F4) and four failure scenarios (F5a–F5d) together define a concrete empirical agenda for future evaluation. Table 18 maps each criterion to specific benchmark design and the statistical test that would confirm or refute it.

Table 18: Empirical benchmark agenda for the Colony Kernel ecology thesis.

Criterion	Benchmark Design	Statistical Test	Refutation Threshold
F1: Location risk accumulation	Run 20 trials; compute Kendall τ between $N_F(l)$ and RefuseRate(l) across locations	$H_0 : \tau \leq 0$	$\tau > 0$ with $p < 0.05$ required
F2: Trust monotone under clean sequence	For 100 agents, record trust at each of outcomes 1–20 with clean-only history	Exact equality check	Any deviation from $\tau_0 + 0.04n$ at any n
F3: Role promotion at exact thresholds	Sweep trust-score values through promotion boundaries; check role assignment	Exhaustive boundary test	Any role mismatch at any threshold
F4: Gate determinism	Submit same proposal twice with identical state; compare gate scores	Strict equality	Any difference in scores
F5a: Trust laundering	Compare refusal rates for 3-outcome vs. 20-outcome agents at zero-history targets	t-test on refusal rates	No significant difference in refusal rates
F5b: Budget flooding	Submit proposals immediately after period reset vs. mid-period	Compare gate score distributions	No significant difference in EXECUTE rates
F5c: Pheromone eclipse	Inject N HUMAN_PRIORITY signals at a high-risk location; measure REFUSE rate	Test REFUSE rate vs. N	Positive correlation between N and EXECUTE rate
F5d: Completeness gaming	Submit 100 proposals with vacuous completeness fields; measure FalsificationWorker verdicts	Count CONDITIONAL/PASS verdicts	$> 20\%$ pass rate on vacuous proposals

The first four criteria (F1–F4) are expected to be satisfied by the current implementation based on the theoretical analysis in Section 3 and the contract tests described in Section 4. The failure scenarios (F5a–F5d) identify genuine vulnerabilities that the current implementation does not fully address and that future work should either mitigate or formally bound.

5.2 Limitations

The evaluation reported in Section 4 rests on deterministic unit and contract tests plus a configured synthetic-workload protocol. This is a necessary starting point for validating the feedback loop's internal mechanics, but it does not establish that the colony's behavior generalizes to production codebases with real failure distributions. The current artifact does not ship raw 20-trial traces, external workload logs, or production replay data. Future work should validate the architecture against real development workflows with authentic failure histories before drawing conclusions about production readiness.

Four scope constraints bound the current system and should inform future work. First, trust deltas are fixed heuristics: the magnitude by which a failure decrements a module's trust score is a configured constant, not a learned parameter, so the system cannot adapt its sensitivity to failure severity over time. Second, the MCP surface is synchronous and single-process: concurrent AI clients operating against the same colony state must serialize through the MCP dispatcher, which bounds the actuation rate the colony can sustain before gate latency becomes a bottleneck. Third, the consequence store is backed by SQLite in single-process mode: this is adequate for the controlled contract tests and configured protocol described here, but distributed deployments with multiple colony instances writing failure records concurrently would require a shared-write backend or an explicit merge protocol. Fourth, the gate formula weights (0.30·budget + 0.30·risk + 0.25·trust + 0.15·completeness) were fixed by design reasoning rather than calibrated against production outcomes; the relative weights of trust and completeness in particular may require adjustment once production failure distributions are available. None of these constraints invalidates the core feedback loop; they do constrain the deployment contexts in which the current implementation should be trusted without modification.

This framework targets teams operating AI agents at actuation rates where the probability of repeated-failure events exceeds what manual oversight can absorb. At those rates, a static trust threshold fails silently — it cannot distinguish a module that has failed ten times from one that has never been tested — and the consequence-driven accumulation architecture described here is the minimum structure needed to maintain human-legible accountability.

The right question for an agentic system is not what the agents can do. Capability is the easy part — modern language models can propose, argue for, and execute an enormous range of actions. The hard part is what the colony has learned: which actions failed, under what conditions, at what cost, and how recently. Codomyrmex answers that question with a record, not a policy. The record is the system.

6 Experimental Setup

This section is structured in two parts. Section 6.1 defines the experimental design proper: independent variables, baseline conditions, dependent variables, trial structure, and analysis procedure. The remaining subsections document configuration parameters in sufficient detail to reproduce reported results; YAML parameter tables are consolidated there so that the experimental narrative above remains uncluttered.

Three distinct evaluation contexts are defined and kept separate throughout:

- Colony-kernel unit and integration test suite — 764 pytest tests gated by CI; these verify behavioral contracts, not performance claims.
- Configured 20-trial benchmark protocol — a reproducible synthetic workload design; each trial is specified as a self-contained colony run against the shared workload corpus (see Section 6.1.4 for trial definition). Raw trial traces are not shipped with this manuscript snapshot.
- Analytical contract checks — deterministic results derived from the gate formula, trust ladder, and checked-in fixtures; these support configuration consistency rather than production performance claims.

Readers should not conflate these contexts: the 20-trial count is a configured protocol size, not the test-suite size, and analytic contract checks do not depend on benchmark trace availability.

All numeric values are drawn directly from the YAML configuration files versioned alongside the codebase (config version 1.3.0).

6.1 Experimental Design

6.1.1 Independent Variables

The primary independent variable is the gating strategy applied to every proposed agent action. Four conditions are evaluated:

1. Codomyrmex — the full composite gate (budget + risk + trust + completeness weights, live `AgentRole` enforcement, and `SignalType`-indexed pheromone coordination).
2. Static-trust-only gate — EXECUTE if agent trust ≥ 0.75, REFUSE otherwise; budget, risk, and completeness dimensions are ignored.
3. Budget-only gate — EXECUTE if remaining LLM-call and runtime budgets are above 50 % of their caps; all other dimensions ignored.
4. No-gate (always EXECUTE) — every proposed action is dispatched unconditionally; serves as the unconstrained performance ceiling and safety floor.

Conditions 2–4 are implemented as drop-in replacements for the composite gate and share the identical task workload and colony configuration.

6.1.2 Dependent Variables

Table 19 defines the measured outcomes for the configured protocol.

Table 19: Dependent variables for the configured benchmark protocol.

Dependent variable	Operationalisation
Gate decision distribution	Fraction of actions routed to EXECUTE / HOLD / REFUSE across a full trial
Error rate	Proportion of EXECUTE decisions that result in a logged failure event (kernel `SignalType.FAILURE` emission)
Budget efficiency	LLM calls consumed per successfully completed task subtree
Trust stability	Mean absolute deviation of agent trust scores across scheduler ticks
Throughput	Task subtrees closed (terminal `SignalType.SUCCESS`) per experiment run

6.1.3 Baseline Conditions and Hypotheses

Each baseline isolates a single gating dimension to attribute the performance of the full Codomyrmex gate to its component contributions:

- H1 (vs. static-trust-only): Codomyrmex achieves a lower error rate by rejecting high-trust agents whose actions exceed budget or risk caps — situations the static baseline cannot detect.
- H2 (vs. budget-only): Codomyrmex achieves higher throughput by permitting low-budget-cost actions from high-trust agents that the budget-only gate would conservatively HOLD.
- H3 (vs. no-gate): Codomyrmex reduces error rate and cumulative security exposure at the cost of a bounded throughput reduction, establishing that the gate's safety overhead is proportionate.

6.1.4 Trial Structure and Replications

The benchmark protocol consists of 20 independent trials. Each trial is defined as follows for reproducibility and future external execution:

- Task workload — a fixed sequence of 50 task subtrees drawn from the shared workload corpus in identical order across conditions within each seed; this removes workload-ordering as a confound.
- Agent mix — 5 agents initialised at `AgentRole.SANDBOX` with `trust_score = 0.1` (the `AgentTrustProfile` default); no pre-warm is applied, so role promotion, if it occurs, is earned within the trial.
- Colony warm-up — the first 10 scheduler ticks are designated warm-up. Pheromone deposits made during warm-up are included in subsequent routing decisions but warm-up-period outcomes are excluded from the dependent-variable computations. This ensures that trust distributions have non-trivial structure before performance measurement begins.
- Termination — a trial ends when either the workload corpus is exhausted or the wall-clock budget cap (300 seconds) is reached, whichever is first.
- Trial indexing — deterministic trial indices run from 0 to
  19. The colony-kernel code path does not invoke random-number generators, so no external seed registry is required for the reported benchmark snapshot.

Because raw benchmark traces are not included in this release artifact, the configured trial count is not used to claim statistical power, confidence intervals, or production effect sizes. The checked-in validation surface is the deterministic test suite plus generated consistency checks.

6.1.5 Trust Initialization and Colony Warm-Up Procedure

All agents enter each trial at `AgentTrustProfile(trust_score=0.1, role=AgentRole.SANDBOX, total_proposals=0)`. The starting score of 0.1 places agents below the REPAIR_ANT promotion threshold (0.20 as defined by `_ROLE_REPAIR_MIN_TRUST` in `role_adapter.py`) so that no agent enters the measurement phase with inherited authority.

The warm-up phase serves two purposes: (a) it allows pheromone trails to accumulate from the first successful and failed actions, seeding the coordination layer with non-trivial signal structure; (b) it allows at least some agents to accumulate the three proposals required by `_ROLE_MIN_PROPOSALS_FOR_PROMOTION` before their roles are evaluated against trust thresholds. The warm-up length of 10 ticks was chosen to be long enough for role differentiation to begin but short enough that the measurement window dominates trial duration.

Trust scores evolve via `compute_trust_delta()` (defined once in `models.py` and delegated to from both `ConsequenceMemory` and `RoleAdapter`). The delta formula is:

```
delta = +0.04  if tests_passed  else  −0.08
delta += −0.05  if repair_needed
delta += human_feedback * 0.03   # human_feedback in [−1, +1]
```

Clamping is applied by `apply_delta`, not by `compute_trust_delta` itself. The asymmetry (+0.04 gain vs. −0.08 loss) models conservative trust: agents must pass roughly two tasks cleanly to recover from a single failure, creating a natural filter for agents that act carelessly.

6.1.6 Analysis Procedure

When benchmark traces are produced, gate decision distributions should be compared across conditions using Fisher's exact test over EXECUTE / HOLD / REFUSE counts; error rates should be compared with two-sample proportion tests with correction for planned pairwise comparisons. The current manuscript does not ship those trace files or analysis outputs. Reported gate-refusal and trust-trajectory values instead come from deterministic contract fixtures regenerated by `scripts/z_generate_manuscript_variables.py`.

6.2 Gate Configuration

The execution gate evaluates every proposed action against four dimensions and routes it to one of three outcomes. Table 20 lists the decision thresholds. These thresholds are defined in `kernel.yaml` and enforced by `colony_kernel/kernel.py` (constants `_GATE_SCORE_EXECUTE = 0.75` and `_GATE_SCORE_HOLD = 0.50`); they are the sole numeric authority — no threshold value is duplicated elsewhere.

Table 20: Gate decision thresholds for experimental configuration.

Outcome	Condition	Interpretation
EXECUTE	composite score ≥ 0.75	Action is approved and dispatched immediately
HOLD	$0.5 \leq$ score < 0.75	Action is queued pending additional context or human review
REFUSE	composite score < 0.5	Action is rejected; rationale is logged to the pheromone trail

On a REFUSE outcome the kernel deposits a `ColonySignal(SignalType.FAILURE, strength=1.0 + falsification_severity * 3.0)` at `proposal.target`, amplifying the inhibitory signal proportionally to how strongly the `FalsificationWorker` flagged the action.

The gate evaluation proceeds in this order within `propose_action()`:

1. `FalsificationWorker.analyze(proposal)` → findings (falsification severity 0.0–1.0)
2. `ResourceLedger.check_budget(proposal.budget_estimate)` → `budget_approved` (non-consuming check — the ledger is not debited at this stage)
3. Load or create the agent's `AgentTrustProfile` via `ConsequenceMemory.get_profile()`; increment `total_proposals`; save; call `RoleAdapter.update(profile)` to refresh role
4. `ActuationGate.evaluate(proposal, profile, findings, budget_approved)` → `GateResult`
5. On REFUSE: deposit the failure pheromone signal described above

Note: `propose_action()` is the primary submission entry point. There is no separate `submit()` method on the public API; callers always go through `propose_action()`.

6.3 Gate Score Weights

The composite gate score is a weighted sum of four independently computed dimension scores. Table 21 gives the weight assigned to each dimension. Weights sum to 1.00.

Table 21: Gate score dimension weights for experimental configuration.

Dimension	Weight	Rationale
Budget	0.30	Fraction of remaining budget headroom relative to configured caps
Risk	0.30	Inverse of estimated action risk; higher risk reduces this component
Trust	0.25	Normalised agent trust score at the time of the gate evaluation
Completeness	0.15	Coverage of required fields and preconditions in the action specification

The budget and risk dimensions share equal weight (0.30 each, combined 0.60), reflecting the design priority of keeping resource expenditure and operational risk as the primary regulators of autonomous action. Trust receives less weight than budget or risk (0.25) because a high-trust agent issuing a budget-exhausting action is still dangerous — the gate must be able to block such actions even when trust is high. Completeness carries the smallest weight (0.15) because an incomplete specification is more often a recoverable quality issue than a safety concern; HOLD (not REFUSE) is the expected outcome for specification gaps alone.

The 0.30/0.30/0.25/0.15 vector is a design-prior weighting: budget and risk jointly dominate the ordinary score, trust is secondary, and completeness is intentionally recoverable through HOLD. Future benchmark traces may calibrate these weights, but the implementation and manuscript treat the current vector as a fixed, auditable configuration contract.

6.4 Pheromone Configuration

The stigmergic coordination layer maintains 6 distinct signal types drawn from the `SignalType` enum defined in `models.py`, each with its own decay rate. Decay is modelled as exponential decay applied once per scheduler tick. The `DecayRate` enum provides evaporation multipliers; the base evaporation rate from `StigmergyConfig` is multiplied by this factor at each tick.

`SignalType` enum values (6 total):

1. `SignalType.FAILURE` — deposited on terminal task failure or REFUSE gate decision; inhibits reassignment to the same subtree
2. `SignalType.SUCCESS` — deposited when an agent successfully closes a task
3. `SignalType.RISK` — deposited when a budget dimension nears its cap or an action's risk score exceeds a warning threshold
4. `SignalType.NEED` — deposited when an agent requests additional resources or context before proceeding
5. `SignalType.DEPENDENCY` — deposited when a task subtree identifies an unresolved inter-agent dependency
6. `SignalType.HUMAN_PRIORITY` — deposited when a HOLD outcome is escalated to human review; persists until acknowledged

Table 22 records the configured pheromone decay classes.

Table 22: Pheromone decay rate classes for experimental configuration.

Class	`DecayRate` value	Effective rate at default base	Characteristic half-life	`SignalType` variants
FAST	3.0	0.3/tick	~2 ticks (2.31)	`FAILURE`, `RISK`
NORMAL	1.0	0.1/tick	~7 ticks (6.93)	`NEED`
SLOW	0.2	0.02/tick	~35 ticks (34.66)	`SUCCESS`, `DEPENDENCY`, `HUMAN_PRIORITY`

`DecayRate` members are plain numeric floats on the enum (`FAST = 3.0`, `NORMAL = 1.0`, `SLOW = 0.2`). They are evaporation multipliers, not lambda functions; the scheduler applies `base_evaporation_per_tick * decay_rate_value` at each tick.

Fast-decay signals carry high-frequency tactical information whose relevance expires within one scheduling cycle. `HUMAN_PRIORITY` is assigned the SLOW class because human review latency can span many ticks; the signal must persist until it is explicitly acknowledged, ensuring that no subsequent automated action proceeds on an escalated item before a human decision is recorded.

6.5 Resource Budget Caps

All experimental and protocol runs are bounded by the hard caps defined in `config/colony_kernel/kernel.yaml`. The kernel enforces these caps via `ResourceLedger`; any action whose projected cost would breach a cap bypasses ordinary gate scoring with `gate_score = 0.0`. Standalone gate evaluation returns `REFUSE` for that condition, while the integrated `ColonyKernel` path returns `HOLD` so the action can be requeued after the budget period resets.

Table 23 lists the hard caps enforced by `ResourceLedger`.

Table 23: Resource budget caps from `kernel.yaml`.

Budget Dimension	Cap	Unit
LLM calls	50	calls per experiment run
Wall-clock runtime	300	seconds
Cumulative risk	0.8	normalised risk units (0–1)
Security exposure	0.9	normalised exposure score (0–1)

The `llm_calls` cap of 50 was chosen to allow complex multi-agent workflows while remaining within single-run cost budgets during protocol execution. The `runtime` cap of 300 seconds (5 minutes) provides a wall-clock backstop independent of call-count accounting; it is intentionally tight to surface runaway-agent behaviour before it accumulates significant cost.

6.6 Role Configuration

Agent roles are defined by the `AgentRole` enum in `models.py`. The colony operates 5 roles, each corresponding to an `AgentRole` variant:

```
SANDBOX     = "sandbox"
REPAIR_ANT  = "repair_ant"
MEMORY_ANT  = "memory_ant"
DISPATCHER  = "dispatcher"
GUARD_ANT   = "guard_ant"
```

Roles are inferred — never hard-assigned at startup. The `AgentRole` docstring states this explicitly: *"Roles are inferred by RoleAdapter — never hard-assigned at startup. Each role carries implicit permission constraints enforced by the gate."*

Table 24 records the experimental role ladder.

Table 24: `AgentRole` variants, promotion thresholds, and permitted actions.

`AgentRole` variant	Promotion trust threshold	Minimum proposals	Permitted action types
`SANDBOX`	— (entry role; all new agents begin here)	—	read-only, observe

AgentRole variant	Promotion trust threshold	Minimum proposals	Permitted action types
REPAIR_ANT	trust ≥ 0.20	≥ 3 total proposals	read, write (scoped), patch, test-fix, doc-update
MEMORY_ANT	trust ≥ 0.35	≥ 3 total proposals	read, write, archive, index, summarise
DISPATCHER	trust ≥ 0.50	≥ 3 total proposals	full task dispatch; may delegate, coordinate, route
GUARD_ANT	trust ≥ 0.70	≥ 3 total proposals	gate other agents; security review, gate audit, archive authority

The role_adapter.py RoleAdapter exposes two APIs that must not be conflated. The kernel-facing infer_role(profile) / update(profile) path uses the 0.20/0.35/0.50/0.70 threshold ladder shown above (constants _ROLE_REPAIR_MIN_TRUST, _ROLE_MEMORY_MIN_TR _ROLE_DISPATCHER_MIN_TRUST, _ROLE_GUARD_MIN_TRUST). An agent with fewer than _ROLE_MIN_PROPOSALS_FOR_PROMOTION = 3 total proposals stays SANDBOX regardless of trust score. The standalone assign_role(agent_id) specialization path additionally considers action history: REPAIR_ANT and MEMORY_ANT require trust >= 0.80 with matching successful action types, GUARD_ANT requires trust >= 0.85 with security action history, and DISPATCHER requires at least 20 proposals with at least 70% acceptance. The manuscript's trust-ladder claims use the kernel-facing infer_role path.

Demotion is automatic: a trust score drop below the entry threshold of the current role triggers an immediate reassignment to the highest role whose threshold the new score still clears.

6.7 Falsification Vectors

The experimental design incorporates 10 adversarial falsification vectors to probe the gate and trust subsystems. Each vector targets a distinct failure mode.

1. Budget exhaustion race — two agents simultaneously consume budget toward the same cap to test that the kernel serialises cap checks correctly and does not double-issue an EXECUTE that breaches the limit.
2. Trust threshold bypass — inject an action with a high composite gate score while the issuing agent's trust score sits below 0.20 (_ROLE_REPAIR_MIN_TRUST), verifying that the agent remains in SANDBOX and that the gate applies SANDBOX permission constraints correctly; the expected outcome depends on the action type (write-path actions should REFUSE; read-only actions may EXECUTE).
3. Pheromone replay — re-deposit a stale SignalType.SUCCESS signal after it has fully decayed to confirm that the scheduler does not re-credit already-processed signals.
4. Role promotion race — promote an agent from AgentRole.SANDBOX to AgentRole.REPAIR_ANT while a concurrent gate evaluation is in flight, testing that the gate snapshot is consistent with the role at decision time and that the mid-flight snapshot is not retroactively updated.
5. Decay rate mismatch — configure a SignalType.RISK signal with a SLOW rate (multiplier 0.2, overriding the default FAST multiplier 3.0) and observe it over FAST-duration ticks to verify that the decay class is read from decay_rates.yaml at deposit time, not inferred from the SignalType variant name.
6. Weight normalisation drift — temporarily mutate one gate weight without re-normalising the others, confirming that the gate detects weight-sum deviation and rejects the malformed configuration before evaluating any action.
7. HOLD-to-EXECUTE coercion — submit a follow-up action identical to a HOLD-queued one to verify that the queue does not silently upgrade the pending item to EXECUTE on the second arrival; the expected outcome is a second HOLD or a REFUSE if the queue depth exceeds the cap.
8. Security exposure creep — issue a sequence of low-risk actions that individually pass the security_exposure cap but cumulatively exceed 0.9, testing that cumulative tracking persists across scheduler ticks and that a SignalType.RISK deposit is emitted at the crossing point.
9. Completeness field spoofing — submit an action specification with all required fields present but semantically empty (empty strings, zero values) to test that the completeness scorer validates content depth, not mere field presence.
10. Concurrent trust update collision — issue two trust-modifying events for the same agent within a single scheduler tick to verify that the trust ledger serialises writes and does not silently drop one update; the post-tick trust score must equal the algebraically correct composition of both events.

The 10-vector worker is exercised in src/codomyrmex/tests/unit/colony_kernel/test_falsification_worker.py, and manuscript/doc drift is covered by src/codomyrmex/tests/unit/colony_kernel/test_manuscript_consistency.py. A vector is considered passing when the system produces the expected finding or gate influence with no side-effects on unrelated state.

6.8 YAML Configuration Files

The colony kernel reads 3 YAML files from `config/colony_kernel/` at startup. No configuration is hardcoded in Python source; all numeric parameters exposed in this section trace to one of these files.

`kernel.yaml` Defines the top-level budget caps (Table 23), the gate decision thresholds (Table 20), the pheromone signal type registry, and the overall gate score weight vector (Table 21).

Important: the kernel-facing trust promotion thresholds (0.20, 0.35, 0.50, 0.70) and the minimum-proposals gate (`_ROLE_MIN_PROPOSALS_FOR_P = 3`) are defined as module-level private constants in `role_adapter.py`; they are not read from YAML and are not configurable at runtime.

`roles.yaml` Defines the 5 `AgentRole` variants (`SANDBOX`, `REPAIR_ANT`, `MEMORY_ANT`, `DISPATCHER`, `GUARD_ANT`) and documentation-facing role labels/defaults. Kernel-facing promotion and demotion thresholds are code-defined in `role_adapter.py` and are not overridable via this file.

`decay_rates.yaml` Provides per-`SignalType` decay rate overrides (Table 22). `SignalType` variants not listed in this file inherit the NORMAL decay rate (multiplier 1.0). The file is the single authoritative source for decay parameters; the kernel reads it at startup and caches values for the lifetime of the colony process. The `SignalType` variant name is used as the YAML key — any unrecognised key causes startup to abort with a configuration error rather than silently defaulting, ensuring that renamed variants are caught immediately.

6.9 Software Environment

All experiments were conducted with the following software stack.

Table 25 lists the software stack used for the manuscript snapshot.

Table 25: Software environment for the manuscript snapshot.

Component	Value
Python version	3.14.4
Package manager	uv
Linter	ruff (zero-error policy enforced in CI)
Type checker	ty (zero-diagnostic policy enforced in CI)
Test runner	pytest with coverage
Config version	1.3.0
Generated	2026-07-10T12:50:01Z

The zero-error and zero-diagnostic policies mean that CI blocks any merge that introduces a ruff finding or a ty type error, providing a continuous correctness baseline across the codebase.

The data models (`ActionProposal`, `GateResult`, `AgentTrustProfile`, `AgentRole`, `SignalType`, `DecayRate`) are defined in `models.py` using Python standard-library `@dataclass` and `enum.Enum` — there is no Pydantic dependency. This choice was deliberate: the models are used inside a performance-sensitive scheduling loop where Pydantic's validation overhead is undesirable, and all validation is performed at the gate boundary before proposals enter the loop.

6.10 Pipeline Ordering

Manuscript variables are produced by a three-stage pipeline that must be executed in order. Running stages out of order produces stale or missing variable substitutions.

Stage 1 — **`z_generate_manuscript_variables.py`** Runs the colony-kernel scoped pytest coverage gate, reads the YAML configuration and source-code constants, and writes `output/data/manuscript_variables.json` plus `output/data/colony_kernel_coverage.json`. This file pair is the auditable snapshot for all manuscript token substitutions used across the manuscript sections.

Stage 2 — **`generate_manuscript_figures.py`** Reads the generated variable snapshot, `docs/manuscript/config.yaml`, and `config/colony_kernel/roles.yaml`, then renders the seven figure assets under `output/figures/` for embedding into the tracked HTML and PDF artifacts. Each figure carries a small provenance note containing the manuscript version, configuration hash, and generation date so visual claims remain tied to the same source snapshot as the prose tokens.

Stage 3 — **`compile_manuscript.py`** Reads every manuscript Markdown section, substitutes token values from `output/data/manuscript_var` writes `output/paper.html`, and optionally produces the final PDF. Unresolved tokens cause the render to abort with an error listing the missing variables.

Section generated 2026-07-10T12:50:01Z from config version 1.3.0.

7 Reproducibility Certification

This section provides a machine-verifiable reproducibility certificate for the complete Codomyrmex study. Every metric below is computed by the analysis pipeline and injected into the manuscript at render time — establishing a cryptographic chain of custody from configuration to publication.

7.1 Configuration Provenance

Table 26 records the source configuration identity for the rendered artifact.

Table 26: Configuration provenance for the rendered manuscript.

Property	Value
Config hash (SHA-256, truncated)	`ae243621bf161617`
Paper version	1.3.0
First author	Daniel Ari Friedman
Keywords	ai-agents, model-context-protocol, mcp, multi-agent, orchestration, colony-control-plane, stigmergy, artificial-ecology, agentic-software-engineering, falsification-worker, actuation-gate, trust-scoring

The configuration hash is computed over the manuscript configuration YAML at render time. It changes whenever any parameter in that file is modified, ensuring that every rendered PDF is traceable to a specific configuration state. Version 1.3.0 is the canonical identifier for this manuscript deposit.

7.2 Generated Artifact Registry

The analysis pipeline produced the following artifacts, each validated by `infrastructure.validation.output.validator` before manuscript rendering proceeds:

Table 27 lists the generated artifact counts.

Table 27: Generated artifact registry for the manuscript pipeline.

Category	Count
Test suite files	13
YAML configuration files	3
MCP tool definitions	8
Documentation files (colony_kernel)	3
Python source files (colony_kernel)	13

All counts are live-computed at render time from the actual filesystem. No count is hand-edited into this document; the substitution pipeline enforces that any unresolved manuscript token causes a non-zero exit before the PDF renderer runs.

7.3 Quality Gate Summary

The following gates must pass before manuscript rendering is permitted. Results are captured by `scripts/z_generate_manuscript_variable` and injected here at render time.

Table 28 repeats the quality-gate snapshot from Table 11.

Table 28: Quality gate summary used by the reproducibility certificate.

Gate	Result	Detail
ruff	0 errors	Style and correctness linting across `src/`
ty	0 diagnostics	Static type-checking via `ty check src/`
pytest	764 passed	Colony Kernel suite (Table 11), zero failures permitted

Gate	Result	Detail
coverage	78.4%	Branch coverage against 40% floor (matches Table 11)

All four gates must hold simultaneously. The manuscript configuration `testing.max_test_failures: 0` field enforces this constraint at the pipeline level — a single test failure blocks the render stage entirely, making a partial-pass manuscript certificate structurally impossible. The pytest count and coverage percentage reported in Table 28 are the same values cited in the abstract and in Table 11; they are emitted by a single pytest invocation whose JSON report feeds both sections, so no independent transcription is possible.

7.4 Exact Reproduction Commands

To reproduce the colony_kernel test suite from a clean checkout:

```
# 1. Clone and enter the repository
git clone <repository-url> codomyrmex
cd codomyrmex

# 2. Install all dependencies (including dev extras)
uv sync

# 3. Run the colony_kernel suite — must yield 764 passing tests, 0 failures
HYPOTHESIS_NO_NPY=1 uv run pytest src/codomyrmex/tests/unit/colony_kernel/ -v \
    --cov=src/codomyrmex/colony_kernel \
    --cov-report=term-missing \
    --cov-fail-under=40

# 4. Run the manuscript consistency guard
uv run pytest src/codomyrmex/tests/unit/colony_kernel/test_manuscript_consistency.py -v

# 5. Verify linting and type gates independently
uv run ruff check src/manuscript_variables.py src/codomyrmex/colony_kernel
uv run ty check src/manuscript_variables.py src/codomyrmex/colony_kernel

# 6. Regenerate manuscript variables from live pipeline output
uv run python scripts/z_generate_manuscript_variables.py

# 7. Confirm all tokens resolved (non-empty output = gate failure)
uv run python - <<'PY'
from pathlib import Path

needle = "{" * 2
matches = [
    str(path)
    for path in Path("output/manuscript").rglob("*")
    if path.is_file() and needle in path.read_text(encoding="utf-8", errors="ignore")
]
if matches:
    raise SystemExit("\n".join(matches))
print("All tokens resolved")
PY
```

The expected terminal summary after step 3 is `764 passed` with branch coverage at or above the 40% floor. Deviations indicate either a dependency mismatch (resolve with `uv sync` against the pinned `uv.lock`) or an environment difference captured in the software specification below.

7.5 Software Environment Specification

Exact software versions are pinned in `uv.lock` at the repository root. The following table records the version range and precise version used to generate the certified results.

Table 29 records the toolchain versions used for certification.

Table 29: Software versions used by the reproducibility certificate.

Component	Version used	Minimum required	Notes
Python	3.14.4	3.10	CPython; tested on macOS arm64 and Ubuntu 22.04 x86_64
uv	≥0.4.0	0.4.0	Dependency resolver and virtual-environment manager
pytest	≥8.0	7.0	Test runner; JSON report via `--json-report`
pytest-cov	≥5.0	4.0	Branch coverage measurement
ruff	≥0.6.0	0.5.0	Linting and formatting
ty	≥0.1.0	0.1.0	Static type checker
SQLite	system	3.35	In-process database for `TraceStore` and `ColonyMemory`

Pinned dependency hashes are in `uv.lock`; reproduce the exact environment with:

```
uv sync --frozen   # Installs from lock file without re-solving
```

The `--frozen` flag guarantees that no dependency drift occurs between the environment that produced the certified test count and the environment used for verification.

7.6 Evaluation Snapshot Specification

The empirical snapshot reported in Section 4 is generated at render time from the live repository, YAML configuration, source-code constants, and the colony-kernel scoped pytest coverage run. The repository does not ship a separate raw simulation trace artifact; the auditable outputs are the generated variable snapshot, the coverage JSON, the rendered figures, and the final HTML/PDF artifacts.

Table 30 identifies the generated snapshot inputs.

Table 30: Evaluation snapshot inputs and generated artifacts.

Parameter	Value	Source
Initial agent count	5	`config/colony_kernel/kernel.yaml`
Starting trust score (all agents)	0.1 (sandbox floor)	`config/colony_kernel/roles.yaml`
Module registry size at launch	8 MCP tool definitions	live filesystem
Scenario corpus — proposal count	10 adversarial falsification vectors	`config/colony_kernel/falsification_v`
Scenario corpus — source	Defined in Section 6.7	
Evaluation snapshot	`output/data/manuscript_variables.json`	Stage 1 output of pipeline (Section 6.10)
Coverage artifact	`output/data/colony_kernel_coverage.json`	pytest-cov JSON from Section 6.10

Determinism guarantee. The colony kernel contains no stochastic components: trust updates are closed-form updates, pheromone decay is deterministic exponential decay applied once per tick, gate scores are deterministic functions of four measurable dimensions, and role promotions are threshold comparisons. No random number generator is seeded or invoked anywhere in `src/codomyrmex/colony_kernel/`. Consequently, identical source, config, and dependency inputs to `z_generate_manuscript_variables.py` produce the same manuscript variable values and coverage-derived metrics. Researchers wishing to verify this property can inspect `src/codomyrmex/colony_kernel/`

for calls to `random`, `numpy.random`, or equivalent RNG APIs; none are used in the kernel path. The zero-mock, real-SQLite test suite further exercises this determinism under realistic I/O boundaries, not only in isolation.

What the provenance system does not guarantee. The build-pipeline certificate — config hash, token injection, CI gate table, and generation timestamp — establishes that the manuscript was rendered from a specific, unmodified configuration and a clean codebase. It does *not* establish that the scenario corpus in Section 6.7 is representative of real-world agentic workloads, that the chosen initial trust score and agent count span the range of ecologically plausible colony sizes, or that the 10 adversarial vectors constitute a statistically complete coverage of the gate's failure modes. These are scientific-validity questions that lie outside the scope of a build-provenance certificate. Readers should treat the reported results as characterising the checked-in implementation contracts and configured protocol, not as universally predictive of behaviour under arbitrary workloads.

7.7 Zero-Mock Verification

The colony_kernel test suite operates under a strict zero-mock policy. All 764 passing tests use:

- Real SQLite databases — every `TraceStore` and `ColonyMemory` interaction persists to an in-process SQLite file created by pytest's `tmp_path` fixture. No in-memory mock replaces the database engine.
- Real **`TraceField`** instances — pheromone emission, decay, and signal routing are exercised on live `TraceField` objects, not stand-in dictionaries.
- Real config loading — `colony_kernel.config` is loaded from the actual YAML file on each test invocation. Profile state is never pre-seeded into a mock; it is read fresh from SQLite.

This policy is not merely a style preference — it is an epistemological requirement. The zero-mock constraint was what surfaced the role-not-updating bug: an agent's role was mutated in memory but the profile was loaded fresh from SQLite on each gate evaluation, silently discarding the mutation. A mock `ProfileStore` configured to return the updated role would have hidden this defect entirely. Real I/O forced the inconsistency into the open. Any future introduction of mocks must be treated as a reduction in the validity of the certificate this section asserts.

7.8 Madlib Injection Verification

This manuscript demonstrates the template's token-injection ("madlib") capability: every quantitative claim is substituted from computed data at render time, not transcribed by hand.

Which script: `scripts/z_generate_manuscript_variables.py`

Input files:

- Manuscript configuration YAML — paper metadata, version, author list, keywords, experiment parameters
- `src/codomyrmex/colony_kernel/` — live filesystem scan for Python source file count
- `docs/modules/colony_kernel/` — live filesystem scan for colony-kernel documentation file count
- pytest JSON report — test count and coverage percentage
- ruff and ty exit codes and JSON outputs

Output directory: `output/manuscript/`

The script writes a single substitution map (key -> value) and the rendering stage applies it to every `*.md` file in `docs/manuscript/`, writing resolved copies to `output/manuscript/`. The substitution is applied with a single-pass regex replace; no token appears in the rendered output.

The configuration token substitution pipeline as a reproducibility mechanism. The madlib system is not merely a convenience — it is the primary mechanism by which this manuscript's numeric claims are reproducible. Every quantity cited in the abstract, results table, and gate summary traces to a named constant in the manuscript configuration YAML or to a computed value emitted by the pipeline. There is no disconnected prose claim that could silently diverge from the underlying measurement. Concretely: the weighted additive gate formula ($0.30 \times \text{budget} + 0.30 \times \text{risk} + 0.25 \times \text{trust} + 0.15 \times \text{completeness}$) is defined once under `experiment.gate_score_weights` and referenced symbolically wherever it appears; a change to any weight coefficient is immediately reflected in all rendered sections or triggers an unresolved-token gate failure. Similarly, the colony-kernel test count (764) is not hand-typed — it is the `RESULT_TEST_COUNT` token emitted by the pytest JSON report at render time. A reader who disagrees with a reported number can trace it to its generating script in under two grep operations.

To verify all tokens resolved:

```
uv run python - <<'PY'
from pathlib import Path

needle = "{" * 2
matches = [
    str(path)
    for path in Path("output/manuscript").rglob("*")
```

```
    if path.is_file() and needle in path.read_text(encoding="utf-8", errors="ignore")
]
if matches:
    raise SystemExit("\n".join(matches))
print("All tokens resolved")
PY
```

A non-empty match indicates an unresolved token and is treated as a gate failure. The generation timestamp embedded in this certificate is `2026-07-10T12:50:01Z`, confirming that the rendered copy was produced at a known point in time and not cached from a prior run.

8 Scope, Related Work, and Positioning

8.1 Agentic Software Engineering

The emergence of LLM-based software agents has produced a wave of frameworks and benchmarks for orchestrating tool-using models in engineering contexts. SWE-bench (Yang et al. 2024b) established the first systematic evaluation of agent-generated patches against real GitHub issues, revealing that even strong models fail on the majority of tasks when operating without structured oversight. SWE-agent (Yang et al. 2024a) showed that agent-computer interface design can substantially change autonomous software-engineering performance, making the interface itself part of the agent system under evaluation. LangGraph (AI 2024) provides a graph-based state machine for multi-step agent pipelines, and AutoGen (Wu et al. 2023) enables conversational multi-agent workflows where agents negotiate task decomposition through natural language. CrewAI (Inc. 2024) extends this model with role-based agent teams, pre-defined workflows, and hierarchical task delegation. These frameworks excel at composing individual agent turns but lack a persistent, inspectable control plane: there is no first-class concept of trust accumulation, no cross-session consequence memory, and no signal-based coordination between concurrent agents.

Crucially, none of LangGraph, AutoGen, or CrewAI implement a pre-actuation falsification gate. In all three systems, an agent that has been granted a tool invocation right may exercise it on any proposal that passes syntactic validation — there is no mechanism for asking "has this agent's consequence history justified this class of action?" before execution proceeds. The ActuationGate addresses exactly this gap: every destructive proposal is evaluated against four measurable dimensions (budget, risk, trust, completeness) weighted 0.30 / 0.30 / 0.25 / 0.15, and only proposals that clear the composite threshold — calibrated against the agent's accumulated trust trajectory — proceed to execution. This falsification-before-actuation discipline is structurally absent from runtime-centric frameworks because those frameworks have no persistent consequence memory from which to compute a justified threshold.

Codomyrmex occupies precisely this gap. It is not a competing agent runtime — it is the missing control plane layer that sits above runtimes such as LangGraph and AutoGen. Where those frameworks answer "how do I route messages between agents?", Codomyrmex answers "which agents have earned the right to execute destructive actions, what have previous actuations cost, and where in the codebase is failure pheromone currently elevated?" The ActuationGate, TraceField, and consequence memory together constitute a governance substrate that any MCP-compatible runtime can query and respect.

8.2 What Codomyrmex Is Not

Explicit scope statements prevent mischaracterization of the contribution. The following are deliberate exclusions, not implementation gaps.

Not a general-purpose agent framework. Codomyrmex does not implement agent execution, message routing, tool dispatch, or task decomposition. It is a governance and coordination layer. Researchers seeking a full agent runtime should use LangGraph (AI 2024), AutoGen (Wu et al. 2023), or CrewAI (Inc. 2024) and optionally layer Codomyrmex above them for trust-gated actuation control.

Not a replacement for LLM APIs. Codomyrmex does not wrap or abstract LLM inference. It has no opinion on which model generates proposals — only on whether those proposals, once generated, meet the evidentiary standard required for destructive actuation. Model selection, prompt engineering, and inference costs remain entirely outside the system boundary.

Not evaluated on production workloads. The results in Section 4 are derived from deterministic contract tests and a configured experimental protocol with 5 agents, 10 adversarial falsification vectors, and a fixed scenario corpus. No claims are made about behavior under real-world traffic patterns, adversarial users with persistent access, or multi-tenant deployments. The actuation gate has been validated against the checked-in contract surface; extrapolation to production workloads requires independent evaluation and is an identified future direction.

Not a security boundary. The ActuationGate is a governance mechanism grounded in consequence history, not a cryptographic access-control system. A sufficiently persistent agent that accumulates enough positive interactions can eventually earn destructive access. The system is designed to make this process transparent and auditable, not to prevent it. Readers seeking cryptographic capability confinement should treat the ActuationGate as complementary to, not a replacement for, OS-level sandboxing, network egress controls, capability-based security substrates, and threat-informed controls such as those discussed in Section 8.7, 8.8.

Not a distributed system. The current implementation uses a single-process SQLite backend. Multi-repository colony state, shared pheromone fields across machines, and distributed consensus for trust scores are all out of scope for this paper.

8.3 Stigmergy and Self-Organizing Systems

Grassé's original term (Grassé 1959) described how ants coordinate without central control — work done by one individual modifies the environment in a way that stimulates further work. Parunak's foundational work on digital pheromones (Parunak 1997) showed that synthetic stigmergy could govern distributed software agents without central coordination, and Dorigo & Stützle's comprehensive treatment of Ant Colony Optimization (Dorigo and Stützle 2004a) established the computational theory underlying pheromone-guided search: decay rates, evaporation schedules, and reinforcement dynamics that balance exploitation of known-good paths against exploration of alternatives. These are the direct computational ancestors of the TraceField and ColonySignal design.

The Colony Kernel implements digital stigmergy via TraceField pheromone deposits keyed by (location, signal_type). Unlike biological systems where signals are continuous diffusion fields, the Colony Kernel uses discrete strength values with configurable exponential decay — more predictable, less emergent, but verifiable. The decay schedule borrows directly from Dorigo's evaporation parameter: deposits weaken each tick, preventing stale success signals from permanently masking accumulated failure signals.

8.4 Multi-Agent Systems and Trust

Classical MAS literature (Wooldridge & Jennings (Wooldridge and Jennings 1995)) distinguished between architectures but seldom addressed how trust accumulates over time in an operative sense. The foundational conceptual work comes from Marsh's formalization of computational trust (Marsh 1994), which decomposed trust into competence, benevolence, and integrity dimensions — a tripartite structure that maps directly onto the Colony Kernel's consequence memory: competence is approximated by the action success rate, integrity by the human_feedback multiplier, and benevolence by the pattern of proactive vs. reactive failure responses.

For agents with unknown histories — which is the common case for newly provisioned LLM agents — bootstrapping trust is a distinct problem from updating trust given evidence. The FIRE model (Huynh et al. 2006) addresses this through interaction, witness, role-based, and certified trust components, providing mechanisms to derive initial trust estimates from indirect evidence when direct interaction history is sparse. Burnett et al. (Burnett et al. 2013) extend this to agents operating across heterogeneous environments, showing that referral networks can seed trust scores before any direct observation. In the Colony Kernel, this bootstrapping problem is currently handled conservatively: new agents begin below the actuation threshold and must accumulate direct consequence history before the gate opens. Integrating FIRE-style indirect evidence propagation is an identified future extension.

Sabater & Sierra's survey (Sabater and Sierra 2005) of trust and reputation systems provides the conceptual grounding for trust_score computation from consequence history, situating the Colony Kernel's approach within the broader trajectory from purely social reputation systems toward hybrid models that combine direct experience with structural context.

8.5 Model Context Protocol

The MCP specification (Anthropic 2024) defines a JSON-RPC protocol for tool exposure. Codomyrmex exposes all Colony Kernel subsystems as MCP tools, meaning the gate can be queried and the pheromone field inspected from any MCP-compatible LLM client. This integration is in-scope; MCP server deployment and transport configuration are out of scope.

8.6 Reinforcement Learning, Constitutional AI, and Consequence Memory

Sutton & Barto (Sutton and Barto 2018) formalize the reward-feedback loop that consequence memory approximates. The Colony Kernel does not implement policy gradients — trust_delta is a hand-crafted heuristic (success → +0.04, failure → -0.08, human_feedback multiplier) rather than a learned policy. Connecting consequence memory to a proper RL policy optimizer is explicitly deferred.

Two more recent lines of work sharpen the relationship between consequence memory and learned alignment mechanisms. Constitutional AI (CAI) (Bai et al. 2022) and RLHF (Christiano et al. 2017) train models to internalize normative constraints through a reward signal derived from human preference feedback. The Colony Kernel's trust score dynamics are structurally analogous: the human_feedback multiplier in the trust update formula plays the role of a preference signal, and the accumulation of consequence records over time resembles the dataset that reward modeling optimizes against. The key difference is that CAI/RLHF modify the underlying model weights — their alignment signal is baked into the parameters. The Colony Kernel leaves model weights unchanged and instead maintains an external, auditable consequence ledger that gates actuation at inference time. This makes the governance layer model-agnostic and revocable: a trust score can be reset, audited, or overridden by a human operator without touching the model. Whether external consequence memory or internalized reward modeling produces more robust alignment under adversarial conditions is an open empirical question.

The actuation gate as a process reward model. Process reward models (PRMs) (Lightman et al. 2023; Uesato et al. 2022) assign scalar scores to intermediate reasoning steps rather than only to final outputs, enabling fine-grained feedback during multi-step problem solving. The ActuationGate is functionally a process reward model applied to agent action proposals: it scores proposals along four dimensions (budget, risk, trust, completeness) weighted 0.30 / 0.30 / 0.25 / 0.15, producing a composite score that determines whether execution proceeds. Unlike learned PRMs trained on human annotations, the ActuationGate's scoring function is hand-crafted and deterministic. But the architectural pattern is the same — a step-level evaluator interposed between proposal generation and execution. This connection suggests a concrete upgrade path: replacing the hand-crafted scoring function with a learned PRM trained on accumulated consequence logs, inheriting the PRM literature's training and calibration methodology while retaining the Colony Kernel's external, auditable governance structure. This integration is an identified future direction.

8.7 Capability-Based Security and Least Authority

The ActuationGate instantiates a well-established security architecture that deserves explicit acknowledgment. Miller's capability-based security model (Miller and Shapiro 2003) argues that access rights should be unforgeable tokens carried by callers, not checked against ambient authority tables — a shift from identity-based ("who are you?") to capability-based ("what do you hold?") authorization. Saltzer and Schroeder's Principle of Least Authority (POLA) (Saltzer and Schroeder 1975) states that every component should operate with the minimum set of privileges needed to accomplish its task, and authority should be granted incrementally as need is demonstrated.

The ActuationGate enforces both principles. Trust tokens are earned through consequence history rather than conferred by identity: an agent's trust_score is an unforgeable capability accumulated from verifiable past interactions, not a static credential that can be spoofed. Destructive tools require a trust_score above a configurable threshold — authority is granted incrementally as the agent's consequence record justifies it. This is POLA applied to the agentic context: LLM agents begin with read-only access and graduate toward write and execute authority only as their consequence memory warrants. Security venues reviewing this work should recognize the ActuationGate as a concrete instantiation of least-authority principles, extended to cover the distinct challenge of agents whose competence distribution is unknown at provisioning time.

8.8 Advanced Threat Models, Zero Trust, and Supply-Chain Integrity

A RedTeam and nation-state reading sharpens rather than weakens the scope boundary. NIST Zero Trust Architecture defines a resource-centric posture in which no network location, caller identity, or previous interaction is sufficient for implicit trust (Rose et al. 2020). Codomyrmex is compatible with that posture because every proposed destructive action is re-evaluated against budget, risk, trust, and completeness before actuation. It is not, however, a complete zero-trust architecture: it does not supply enterprise identity proofing, device posture, network segmentation, policy decision points for every resource, or continuous detection across an operating environment.

The same distinction applies to secure development and supply-chain integrity. NIST SSDF describes practices for reducing software-vulnerability risk throughout development (Souppaya et al. 2022). SLSA v1.2 defines levels and tracks for software supply-chain integrity and provenance (Open Source Security Foundation 2026), while Sigstore provides a public-good signing and transparency-log system for verifying software artifacts (Newman et al. 2022). The Codomyrmex manuscript pipeline records configuration hashes, generated variables, rendered artifacts, and quality gates, so it can participate in a provenance-aware workflow. That evidence does not claim SLSA certification, complete SBOM coverage, or resistance to a compromised builder, dependency, package registry, signing key, or continuous-integration identity.

Threat-informed defense frameworks make the remaining gap explicit. MITRE ATT&CK is a knowledge base of adversary tactics and techniques based on real-world observations (MITRE Corporation 2026b), and MITRE ATLAS extends this style of knowledge base to adversarial activity against AI-enabled systems (MITRE Corporation 2026a). OWASP's LLM and agentic-AI guidance names prompt injection, insecure output handling, supply-chain vulnerabilities, excessive agency, tool misuse, identity and privilege abuse, memory or context poisoning, insecure inter-agent communication, cascading failures, and rogue-agent behavior as relevant risk classes (OWASP Foundation 2025, 2026b, 2026a). Codomyrmex directly addresses only part of that space: it can make destructive actuation depend on earned consequence history, falsification pressure, and auditable gate decisions. It does not solve prompt injection, exfiltration, sandbox escape, model poisoning, compromised credentials, malicious dependencies, or post-compromise lateral movement by itself.

Table 31 summarizes the threat-informed security framing used in this paper.

Table 31: Threat-informed security framing for Codomyrmex.

Threat lens	Primary sources	Codomyrmex relevance	Boundary
Zero-trust authorization	NIST SP 800-207 (Rose et al. 2020) and NIST agent identity work (NIST National Cybersecurity Center of Excellence 2026)	Treats each destructive proposal as a fresh authorization question	Not an enterprise zero-trust architecture or identity system
Secure development	NIST SSDF (Souppaya et al. 2022)	Keeps verifier-first manuscript, code, and artifact checks auditable	Not a formal SSDF compliance assessment
Supply-chain provenance	SLSA (Open Source Security Foundation 2026) and Sigstore (Newman et al. 2022)	Makes manuscript and package claims easier to bind to generated evidence	Does not claim SLSA certification, SBOM completeness, or signed-release enforcement
AI risk management	NIST AI RMF (Tabassi 2023) and the NIST Generative AI Profile (Autio et al. 2024)	Treats the gate ledger as evidence for Govern, Map, Measure, and Manage activities	Not an AI risk-management framework implementation or organizational governance program

Threat lens	Primary sources	Codomyrmex relevance	Boundary
AI and agentic threats	MITRE ATT&CK (MITRE Corporation 2026b), MITRE ATLAS (MITRE Corporation 2026a), and OWASP LLM/Agentic guidance (OWASP Foundation 2025, 2026b, 2026a)	Gives RedTeam vectors for prompt, tool, memory, identity, and supply-chain abuse	Does not claim exhaustive ATT&CK/ATLAS coverage or production adversarial validation

AI risk-management standards add an organizational lens to the same boundary. NIST AI RMF organizes risk work around Govern, Map, Measure, and Manage functions, while the Generative AI Profile adapts those functions to generative-AI risks and lifecycle evidence (Tabassi 2023; Autio et al. 2024). Codomyrmex can supply auditable Measure/Manage artifacts: gate decisions, trust deltas, local pressure, falsification outcomes, and rendered reproducibility records. It does not supply the surrounding governance system: policy ownership, risk tolerance, impact assessment, user redress, model lifecycle controls, vendor management, or enterprise accountability. The contribution is therefore evidence-producing infrastructure for a future AI risk-management process, not certification against one.

The agentic-security scholarship supplies a second, more empirical pressure test for the same boundary. Indirect prompt-injection work shows that untrusted retrieved content can steer LLM-integrated applications and tool calls (Greshake et al. 2023). InjecAgent and AgentDojo turn that risk into benchmarkable tool-integrated scenarios where agents must preserve utility while resisting adversarial instructions embedded in external data (Zhan et al. 2024; Debenedetti et al. 2024). ToolEmu demonstrates that high-stakes tool interactions can fail even in sandboxed evaluation (Ruan et al. 2023), while Agent-SafetyBench, Agent Security Bench, RAS-Eval, and SafeToolBench broaden the evaluation surface across safety failures, attacks, defenses, tools, and real-world-style environments (Z. Zhang et al. 2024; H. Zhang et al. 2024; Fu et al. 2025; Xia et al. 2025). PrivacyLens adds an action-level privacy lens: models can answer privacy questions correctly yet still leak sensitive information when executing agent trajectories (Shao et al. 2024). Memory-poisoning and AgentAuditor work sharpen the temporal part of the Codomyrmex thesis: memory can preserve adversarial influence, but structured experiential memory can also improve safety evaluation when it is audited and retrieved deliberately (Dash et al. 2026; Luo et al. 2025). Finally, tool-permission attack studies warn that agents may treat tool availability as implied permission unless an external policy layer blocks that inference (Lupinacci et al. 2025).

Table 32 maps these papers to the part of the control-plane claim they support.

Table 32: Agentic-security scholarship that motivates future Codomyrmex evaluation.

Scholarship lens	Representative sources	Claim pressure on Codomyrmex	Boundary
Indirect prompt injection	Greshake et al. (Greshake et al. 2023), InjecAgent (Zhan et al. 2024), AgentDojo (Debenedetti et al. 2024)	Tool calls must not inherit authority from retrieved or user-supplied text	The manuscript does not claim prompt-injection immunity
High-stakes tool failure	ToolEmu (Ruan et al. 2023) and tool-permission attacks (Lupinacci et al. 2025)	Tool availability does not imply permission; destructive actions need external gating	The ActuationGate is not a sandbox or downstream authorization system
Security benchmark coverage	Agent-SafetyBench (Z. Zhang et al. 2024), ASB (H. Zhang et al. 2024), RAS-Eval (Fu et al. 2025), SafeToolBench (Xia et al. 2025)	A credible control plane should be evaluated against safety, attack, defense, and tool-use benchmark suites	Codomyrmex has not yet been run against these external benchmarks
Privacy leakage through action	PrivacyLens (Shao et al. 2024)	Treat disclosure, forwarding, and cross-context sharing as actuation harms, not just text-generation harms	The current falsification vectors do not include privacy-norm trajectory benchmarks
Memory as attack surface and evaluator	Memory poisoning (Dash et al. 2026) and AgentAuditor (Luo et al. 2025)	Persistent history can both carry adversarial residue and improve future safety judgments	Consequence memory must itself be audited; persistence is not automatically trustworthy

This table is intentionally framed as a falsification agenda, not as validation already achieved. The current results show that Codomyrmex is internally contract-tested against its checked-in falsification vectors and manuscript invariants. They do not show that Codomyrmex reduces attack success on AgentDojo, InjecAgent, ToolEmu, Agent-SafetyBench, ASB, RAS-Eval, SafeToolBench, PrivacyLens, or memory-poisoning benchmarks. A future external evaluation should treat those benchmarks as adversarial workloads: the control plane succeeds only if failed tool proposals produce durable consequence records, lower future trust, raise local pressure, and measurably reduce later unsafe actuation attempts under comparable tasks.

The assurance and agent-evaluation literature adds a third pressure test: evidence must be useful enough to support a structured safety argument, and benchmark tasks must resemble the workflows in which agents will actually operate. Agentic-governance guidance names task suitability, constrained action spaces, approval, legibility, monitoring, attribution, interruptibility, and maintained human control as baseline practices for agentic systems (Shavit et al. 2023). AI-control work studies programming-task protocols under intentionally subversive untrusted models, making explicit the difference between a helpful assistant and a model that may strategically evade monitoring (Greenblatt et al. 2023). Safety-case work for frontier AI and AMLAS both frame safety as a structured claim supported by explicit evidence in a defined operational context (Buhl et al. 2024; Hawkins et al. 2021). These sources support the manuscript's evidence discipline: consequence records, gate decisions, rendered variables, and reproducibility certificates should be treated as ingredients for an assurance case, not as a substitute for one.

Realistic agent benchmarks sharpen the same requirement from the evaluation side. AgentBench, WebArena, OSWorld, GAIA, AppWorld, TheAgentCompany, and benchmark-mutation work on SWE-bench all move evaluation away from isolated prompt-answer tasks and toward long-horizon, tool-rich, state-changing environments (Liu et al. 2023; Zhou et al. 2023; Xie et al. 2024; Mialon et al. 2023; Trivedi et al. 2024; Xu et al. 2025; Garg et al. 2025). Their lesson for Codomyrmex is conservative: internal contract tests can prove that the control plane enforces its own invariants, but they cannot prove that earned actuation lowers real-world unsafe action rates. Table 33 therefore extends the falsification agenda from security attacks to assurance and workflow realism.

Table 33: Assurance and benchmark scholarship that extends the Codomyrmex falsification agenda.

Scholarship lens	Representative sources	Claim pressure on Codomyrmex	Boundary
Agentic governance	Shavit et al. (Shavit et al. 2023)	Constrain action space, keep activity legible, monitor automatically, preserve interruptibility	Codomyrmex is not a full organizational AI-management system
AI control	Greenblatt et al. (Greenblatt et al. 2023)	Treat powerful model assistance as potentially untrusted and evaluate oversight under strategic subversion	Current tests do not model collusion, sabotage, or deceptive alignment
Assurance cases	Buhl et al. (Buhl et al. 2024) and AMLAS (Hawkins et al. 2021)	Bind safety claims to explicit evidence, operational context, and revision when evidence changes	The manuscript does not present a certified safety case
Software-agent interfaces	SWE-agent (Yang et al. 2024a) and SWE-bench mutation (Garg et al. 2025)	Evaluate coding agents through realistic repository interaction, interface design, and user-query distributions	Codomyrmex has not yet been evaluated as a SWE-agent or mutated SWE-bench policy layer
Long-horizon tool environments	AgentBench (Liu et al. 2023), WebArena (Zhou et al. 2023), OSWorld (Xie et al. 2024), GAIA (Mialon et al. 2023), AppWorld (Trivedi et al. 2024), and TheAgentCompany (Xu et al. 2025)	Require control-plane evaluation where actions change persistent external state across tools, apps, files, and workflows	Internal deterministic fixtures are not substitutes for external environment benchmarks

Runtime-assurance, provenance, and visibility scholarship makes the engineering obligation sharper still. The Simplex line of work allows an advanced, high-performance controller to operate only while a decision module can switch to a verified-safe baseline when safety is threatened (Seto et al. 1998; Phan et al. 2017; Mehmood et al. 2021). Shielding work applies a similar idea to reinforcement learning by interposing a monitor that filters or corrects unsafe actions before they violate a temporal-logic specification (Alshiekh et al. 2018). Codomyrmex does not provide formal control-theoretic or temporal-logic guarantees, but the analogy is useful: the ActuationGate is a runtime assurance point for software actuation, and its future evaluation should ask whether the gate behaves like a conservative shield under pressure.

Software-supply-chain research points to a second hardening path. In-toto records a cryptographically verifiable chain of supply-chain steps from project inception through deployment (Torres-Arias et al. 2019), while TUF designs software update systems to survive key compromise through role separation, delegation, thresholds, and rollback protection (Samuel et al. 2010). The current manuscript pipeline records hashes, generated variables, rendered artifacts, and quality gates; it does not yet bind those records into an in-toto layout, TUF repository, or signed release policy. Visibility and infrastructure work for AI agents adds the governance analogue: identifiers, real-time monitoring, activity logging, credentials, reputation, and action logs are treated as shared infrastructure for accountable agents rather than optional observability features (Chan et al. 2024, 2025; Kolt 2025).

Finally, the evaluation surface should include explicitly harmful, cyber-capable, and collusive agent behavior. AgentHarm measures whether tool-using agents comply with malicious multi-step requests (Andriushchenko et al. 2025). ST-WebAgentBench measures whether

web agents complete tasks while respecting safety and trustworthiness policies (Levy et al. 2024). CyberSecEval 3, one-day vulnerability exploitation, zero-day multi-agent exploitation, and CVE-Bench test cybersecurity risk, offensive capability, and exploit behavior in settings closer to operational abuse than ordinary coding benchmarks (Wan et al. 2024; Fang et al. 2024; Zhu et al. 2024, 2025). Secret-collusion work shows that multiple agents can coordinate through steganographic channels to evade oversight (Motwani et al. 2024). Codomyrmex has not yet been evaluated against AgentHarm, ST-WebAgentBench, CyberSecEval, one-day or zero-day exploitation scenarios, CVE-Bench, or secret-collusion scenarios; those are future falsification workloads, not evidence already achieved.

Table 34 summarizes the added hardening agenda.

Table 34: Runtime-assurance, provenance, visibility, and harmful-agent scholarship that extends the external validation agenda.

Scholarship lens	Representative sources	Claim pressure on Codomyrmex	Boundary
Runtime assurance and shielding	Simplex (Seto et al. 1998; Phan et al. 2017; Mehmood et al. 2021) and safe RL shielding (Alshiekh et al. 2018)	Treat gate decisions as runtime safety checks interposed before action	The ActuationGate is not a formally verified controller, temporal-logic shield, or safe fallback policy
Verifiable provenance	In-toto (Torres-Arias et al. 2019) and TUF (Samuel et al. 2010)	Convert manuscript/package evidence from local hashes into verifiable supply-chain attestations	Current artifacts are not signed, thresholded, delegated, or TUF/in-toto enforced
Agent visibility and infrastructure	Visibility into AI Agents (Chan et al. 2024), Infrastructure for AI Agents (Chan et al. 2025), and Governing AI Agents (Kolt 2025)	Make identifiers, monitoring, activity logging, credentials, reputation, and action records first-class governance surfaces	Codomyrmex is not a legal identity, liability, dispute-resolution, or cross-platform agent infrastructure system
Harmful and cyber agent benchmarks	AgentHarm (Andriushchenko et al. 2025), ST-WebAgentBench (Levy et al. 2024), CyberSecEval 3 (Wan et al. 2024), one-day exploitation (Fang et al. 2024), zero-day multi-agent exploitation (Zhu et al. 2024), and CVE-Bench (Zhu et al. 2025)	Evaluate whether earned actuation reduces malicious multi-step tool use, policy violations, cyber abuse, and exploit behavior	These external harmful/cyber benchmark results are not reported in this paper
Multi-agent collusion	Secret Collusion among AI Agents (Motwani et al. 2024)	Test whether independent-looking agents can covertly coordinate to evade monitoring and gate pressure	Current deterministic fixtures do not model collusion, steganography, or deceptive multi-agent coordination

Under an advanced persistent threat model, the strongest safe claim is therefore architectural compatibility, not containment. Assume an adversary can compromise an agent process, poison a dependency, steal a CI token, replay a stale artifact, or use prompt injection to steer an otherwise trusted model. Codomyrmex can reduce blast radius by requiring consequence-backed authorization, preserving local failure history, and forcing high-risk proposals through auditable gate decisions. It still requires external controls: workload sandboxing, least-privilege credentials, network egress restrictions, signed artifacts, SBOM/provenance verification, protected secrets, independent monitoring, incident response, and human review for high-impact actions. Alignment with these frameworks is an evaluation agenda, not an achieved security certification.

8.9 Active Inference and Free Energy

Friston's Free Energy Principle (Friston 2010) frames intelligent behavior as minimization of variational free energy — the gap between an agent's generative model of the world and its sensory observations. A full mapping of the Colony Kernel onto this framework requires identifying three components: (1) a generative model $p(o, s)$ over observations o and hidden states s; (2) a variational posterior $q(s)$ that the agent maintains over those hidden states; and (3) a policy-selection mechanism that minimizes expected free energy over future states.

A rigorous active inference treatment of the Colony Kernel is beyond the scope of this paper and is deferred to future work. What can be noted structurally is that the pressure loop shares the phenomenology of free energy minimization — high failure pheromone concentrations correspond to regions of elevated prediction error, and agent routing toward those regions resembles the exploratory behavior that active inference predicts when epistemic value is high. We do not claim formal equivalence. Readers who find the analogy productive should treat it as a design intuition rather than a theoretical commitment; readers seeking formal guarantees should await the companion mathematical treatment.

8.10 Explicit Scope Limitations

The following limitations are deliberate design decisions, not oversights. Each reflects a trade-off made to keep the initial system tractable and verifiable; each names a concrete future extension.

Control plane only. The Colony Kernel governs agent behavior but does not execute actions itself. This boundary is load-bearing: mixing governance and execution in a single component makes both harder to audit. LLM runtimes (LangGraph, AutoGen, or direct MCP clients) sit below the Colony Kernel and handle execution; the Colony Kernel sits above them and handles authority. A future integration layer could make this separation transparent to callers.

No security certification or nation-state containment claim. The system records consequence, gates action, and preserves evidence, but it does not claim SLSA certification, zero-trust architecture compliance, full ATT&CK/ATLAS coverage, or operational resistance to advanced persistent threats. Production deployments require external identity, isolation, provenance, monitoring, and response controls beyond the scope of the Colony Kernel.

Discrete-tick time model. Pheromone decay advances on `ColonyKernel.tick()` calls rather than on a wall-clock timer. This was chosen deliberately: a tick-driven model is deterministic, testable, and reproducible across environments without scheduling infrastructure. Real-time decay introduces non-determinism that complicates replay and debugging. Integration with a real-time clock daemon — mapping ticks to configurable wall-clock intervals — is a straightforward future extension once the tick semantics are proven stable.

Heuristic trust deltas, no learned policy. Trust scoring uses fixed deltas (success → +0.04, failure → -0.08, human_feedback multiplier) rather than a policy learned from data. This choice follows the principle Sutton & Barto (Sutton and Barto 2018) articulate for bootstrapping reinforcement learning systems: hand-crafted reward shaping provides stable initial signal while learned policies are still sample-starved. The heuristic deltas were calibrated to match the intuition that failures should cost more than successes gain — a risk-averse bias appropriate for irreversible actions. Replacing the heuristic with a proper policy gradient or Q-learning update is an identified future direction, contingent on accumulating sufficient consequence logs to train from.

Human-confirmed pruning. The pruning daemon identifies stale consequence records and low-activity code regions but never archives automatically. Automated pruning of operational memory raises correctness risks that are difficult to bound without extensive deployment experience. Human confirmation is required until a sufficient empirical record justifies a configurable auto-prune threshold.

Synchronous MCP tools. The current MCP surface is synchronous, which means concurrent proposals from multiple agents serialize at the gate. This is conservative and safe; it does not reflect a fundamental design constraint. Async concurrency across simultaneous proposals — with per-proposal trust evaluation and pheromone read isolation — is the primary scalability target for a future release.

Single-process consequence memory. The SQLite backend stores consequence history in a single-process database, making distributed colony state across multiple repositories or machines out of scope. Distributed consensus for pheromone fields and shared trust scores is a substantial engineering problem; the current design optimizes for correctness and debuggability in the single-repository case. Sharding or replication strategies will be addressed in a follow-on architecture document.

8.11 Information-Theoretic Security and Differential Privacy Bounds on Trust

The trust score is, at its core, a statistic computed over a private ledger of agent interactions. This observation connects the Colony Kernel to the differential privacy literature (Dwork and Roth 2014), which provides rigorous tools for reasoning about what an adversary can infer about the ledger from the published statistic.

8.11.1 Trust as a Published Statistic

When the colony publishes trust scores via `colony_agent_profile` (the MCP tool), it releases a function of the consequence memory. An adversary with access to the published trust score τ_n and the update rule Equation 24 can potentially infer individual interaction outcomes from changes in the score — particularly in sparse interaction histories where the delta from a single outcome is relatively large compared to the total accumulated score. This is the *inference attack* problem in differential privacy terminology (Dwork and Roth 2014; Near and Abuah 2021).

8.11.2 Sensitivity Analysis for Trust Publication

As established in Section 3.4, the global ℓ_1 sensitivity of the trust score to any single interaction record is bounded by $\Delta_{\text{global}} = 0.16$. This means that if two consequence histories differ in a single record, the published trust scores differ by at most 0.16 — a non-trivial fraction of the $[0, 1]$ range.

An adversary observing the trust score before and after an interaction can infer:

- Whether the action passed or failed (if the delta is close to ± 0.04 or -0.08)
- Whether repair was needed (if the delta is -0.05 below the pass/fail component)
- The sign of human feedback (if the total delta deviates from the pass/fail baseline)

The *privacy risk* is highest when: 1. The trust history is short (few interactions), so individual deltas are large relative to accumulated history 2. The score is queried both before and after a sensitive interaction 3. The observer has side information about the agent's normal behavioral patterns

8.11.3 Practical Mitigations and Deployment Recommendations

The current implementation does not apply differential privacy noise: trust scores are published in full. This is appropriate for the single-repository, internal deployment model described in this paper, where the consequence ledger and the agent pool are both under the operator's direct control. For deployments where:

- Multiple organizations share a colony (federated deployment)
- Agent identities map to real individuals whose failure rates could be sensitive
- The trust score is used for downstream decisions beyond gate gating

the Gaussian mechanism calibration in Equation 32 specifies the noise level required. At $\epsilon = 1.0$, $\delta = 10^{-5}$, the required noise is approximately $\sigma_{\text{DP}} \approx 0.76$ on a $[0, 1]$ trust scale — large enough to substantially obscure individual interactions. This creates a fundamental tension: the trust score must be precise enough to gate actions accurately, but revealing enough information that adding protective noise degrades gate accuracy. Resolution requires calibrating the privacy budget against the gate's sensitivity to trust-score uncertainty.

Connection to Secure Aggregation. Multi-party computation literature (Bonawitz et al. 2017) offers an alternative: rather than adding noise to the published score, organizations can compute trust aggregates over federated consequence histories using secure aggregation protocols that reveal only the aggregate without exposing individual records. This is compatible with the Colony Kernel's trust update rule because the trust score is a linear function of outcome deltas, and linear functions are tractable under standard secure aggregation frameworks (Mohassel and Zhang 2017). Federated colony trust computation is an identified future direction.

8.11.4 Shannon Entropy of Gate Decisions

A second information-theoretic property concerns the information content of gate decisions. Define the entropy of the gate decision distribution at location l under the stationary failure process (Corollary 2 of Section 3.1) as:

$$H(\text{gate} \mid l) = -\sum_{d \in \{\text{EXECUTE, HOLD, REFUSE}\}} p_d^{(l)} \log p_d^{(l)} \tag{49}$$

where $p_d^{(l)}$ is the stationary probability of decision d at location l.

Proposition 4 (Gate Entropy Decreases at High-Risk Locations). At locations with high failure rates $q \to 1$, the stationary gate decision distribution concentrates on REFUSE, and $H(\text{gate} \mid l) \to 0$. At locations with low failure rates $q \to 0$, the distribution concentrates on EXECUTE, and $H(\text{gate} \mid l) \to 0$. Gate entropy is maximized at intermediate failure rates, where all three decisions have non-trivial probability.

Proof sketch. High q drives RISK pheromone to its steady-state maximum (bounded in Equation 14), causing `risk_ok = 0.0` and $g < 0.75$ with high probability under reasonable trust distributions. Low q drives RISK pheromone to zero, causing `risk_ok = 1.0` and $g \geq 0.75$ with high probability. Both extremes minimize entropy. At intermediate q, the pheromone dynamics create a distribution over all three decisions. □

Information Value of Sensing. The mutual information $I(\text{RISK pressure}; \text{decision})$ measures how much information the pheromone field adds to the gate decision beyond what trust score alone provides. Under the gate's additive structure, this is bounded by the information in the `risk_ok` component, which itself depends on the three-tier mapping. The mutual information is maximized when `risk_ok` is the decisive component — when trust, budget, and completeness are all near their maximum values, and only risk pressure distinguishes EXECUTE from HOLD. This suggests that the pheromone field is most informative precisely when agents are competent (high trust) and well-organized (complete proposals) — conditions under which a binary trust-only gate would wrongly pass dangerous proposals, but the stigmergic risk signal correctly applies caution.

8.12 Mechanism Design and Incentive Compatibility

The Colony Kernel can be analyzed as a *mechanism* in the sense of mechanism design theory: it is a set of rules that maps agents' reports (proposals) and revealed actions (outcomes) to decisions (gate verdicts and trust updates) that allocate valuable resources (execution authority) (Roughgarden 2016; Myerson 1979).

8.12.1 The Colony as a Direct Revelation Mechanism

In a direct revelation mechanism, agents are asked to report their types (in our case, the true safety and completeness of their proposals) directly, and the mechanism is designed so that truthful reporting is an equilibrium strategy (Myerson 1979). The Colony Kernel is *not* a

direct revelation mechanism: agents submit structured proposals (not type reports), and the gate independently evaluates completeness, risk, and trust. However, the mechanism's incentive properties are still analyzable.

Definition 5 (Proposal Game). The proposal game Γ is a sequential game where: - Players are agents $\mathcal{A}$ - Each agent chooses a proposal strategy σ_a : task $\mapsto$ ActionProposal - The gate mechanism M maps proposals to decisions $\{$EXECUTE, HOLD, REFUSE$\}$ - Agent payoffs: $+R$ for EXECUTE, 0 for HOLD (with option to revise), $-C$ for REFUSE (where $R, C > 0$ represent the value of executing vs. the cost of a false rejection)

Theorem 8 (Incentive Compatibility of Completeness Scoring). In the proposal game Γ, truthful proposal completion (submitting all three required fields when the agent possesses the information) is a weakly dominant strategy.

Proof. For an agent with a safe proposal (true `risk_ok = 1.0`, true `budget_ok = 1.0`, trust_ok ≥ 0.5), the gate score with full completeness is: $g_{\text{full}} = 0.30 + 0.30 + \text{trust} + 0.15 \geq 0.875$, which is EXECUTE. With one missing field: $g_{\text{partial}} = 0.30 + 0.30 + \text{trust} + 0.15 \times 0.65 = 0.30 + 0.30 + \text{trust} + 0.098$. For an agent with trust_ok = 0.5: $g_{\text{full}} = 0.875$, $g_{\text{partial}} = 0.823$. Both are EXECUTE.

The omission of a field can only reduce the gate score by $0.15 \times 0.35 = 0.0525$ per missing field. When the gate score with full completeness is in the EXECUTE band $(g \geq 0.75)$, it may still pass with one missing field (as shown in the worked example in Section 2). However, an agent near the EXECUTE/HOLD boundary who strategically omits evidence to save submission effort risks being pushed into HOLD — a worse outcome than EXECUTE. Therefore, submitting complete proposals weakly dominates incomplete proposals for all agents whose proposals are genuinely safe. □

Remark (Strategic Incompleteness as a Red Flag). An agent that *deliberately* submits incomplete proposals to game the HOLD threshold (preferring HOLD to EXECUTE for reasons the mechanism does not see, e.g., to extend the HOLD window as cover while preparing a second action) benefits from the partial score reduction. However, repeated HOLD decisions with incomplete proposals do not increase trust — they generate no trust delta at all until the action executes — and the agent accumulates `total_proposals` without the matching trust accumulation, creating a detectable anomaly in its consequence history. This side effect makes strategic incompleteness detectable as a behavioral pattern, supporting the GUARD_ANT role's audit function.

8.12.2 Free-Rider Detection via Pheromone Imbalance

A free-rider problem in multi-agent systems occurs when agents benefit from collective resources without contributing proportionally. In the Colony Kernel, the equivalent free-rider is an agent that extracts value (EXECUTE decisions) by accumulating trust cheaply — e.g., by submitting many low-risk proposals to build trust, then submitting a high-risk proposal once trust is established.

Proposition 5 (Free-Rider Resistance via Pheromone Separation). The pheromone field provides partial resistance to trust-laundering attacks because RISK pheromone is deposited at the *target location* of the high-risk proposal, not at the agent's trust record. An agent that successfully builds trust through low-risk proposals and then submits a high-risk proposal to a location with existing RISK pressure faces two independent barriers: the gate evaluates `risk_ok` for the target location (where RISK pressure may already be elevated from prior failures) and `trust_ok` for the proposing agent (accumulated from prior history). These are orthogonal: trust at the agent level does not reduce RISK pressure at the target location.

Proof. By the compound-key addressing of the pheromone field, (l, RISK) and agent trust are stored in independent data structures. The gate score formula Equation 2 adds their contributions independently. High agent trust does not reduce `risk_ok`; elevated target RISK pressure does not reduce `trust_ok`. Therefore, an agent cannot leverage accumulated agent-level trust to bypass location-level risk barriers. □

This orthogonality is a deliberate design property: it means that the two primary risk signals in the gate (risk pheromone at the target and agent trust history) cannot be traded against each other through strategic proposal sequencing.

8.12.3 Mechanism Optimality and the Budget Constraint

Myerson's revenue-equivalence theorem (Myerson 1979) and its extensions characterize optimal mechanisms for resource allocation under private information. In the Colony Kernel, the scarce resource is execution authority — a slot in the colony's actuation envelope. The budget constraint (one of the four gate components) implements a hard feasibility constraint: proposals that exceed the period budget are ineligible regardless of other attributes.

From the mechanism design perspective, the budget constraint is a *participation constraint*: the colony can only allocate what it has. The multi-dimensional nature of the budget (seven independent cost dimensions) means that the feasibility region is a hyperrectangle in $\mathbb{R}^7_{\geq 0}$, and the `can_afford` check is a test for membership in this hyperrectangle. The mechanism is *ex-post individually rational* — no agent is forced to incur costs it has not consented to — because REFUSE and HOLD decisions do not consume budget, and EXECUTE decisions consume only the resources explicitly estimated in the proposal.

Future Direction: Optimal Threshold Calibration. Mechanism design provides normative tools for calibrating the gate thresholds (0.75 for EXECUTE, 0.50 for HOLD) to maximize a colony-level objective (e.g., maximize expected colony output subject to a maximum expected repair rate). This would require specifying a utility function over gate decisions and fitting a model of the proposal quality distribution from production consequence logs. Myerson's optimal mechanism framework (Myerson 1979) then provides the optimal threshold as a function of the proposal quality distribution and the utility function. This calibration is an identified future direction, contingent on accumulating sufficient production data.

8.13 Crowdsourcing, Collective Intelligence, and Colony Wisdom

The Colony Kernel draws on a research tradition that predates agentic AI: the study of how collectives of agents — human or artificial — aggregate distributed information into better decisions than any individual agent could make alone (Surowiecki 2004; Condorcet 1785).

8.13.1 The Condorcet Jury Theorem as a Design Template

Condorcet's Jury Theorem (Condorcet 1785) states that if each member of a jury independently makes the correct judgment with probability $p > 0.5$, then as the jury size grows, the probability that the majority vote is correct approaches 1. The theorem formalizes the intuition that independent, better-than-random agents collectively converge on truth.

The FalsificationWorker implements a version of this principle: its 10 independent attack vectors are orthogonal checks, each with different precision-recall trade-offs, and the overall verdict is a majority-style aggregation (PASS if no HIGH findings, CONDITIONAL if moderate findings, FAIL if any HIGH or CRITICAL). Under the assumption that each attack vector is independently informative about proposal safety, the 10-vector ensemble is more accurate than any single vector — a direct application of Condorcet's principle.

Quantitative Bound. Let p_v be the probability that attack vector v correctly identifies a dangerous proposal (sensitivity) and $1 - q_v$ the probability it flags a safe proposal (false positive rate). Under independence, the probability that *at least one* vector flags a dangerous proposal is:

$$\Pr[\text{at least one flag} \mid \text{dangerous}] = 1 - \prod_{v=1}^{10}(1 - p_v) \tag{50}$$

For $p_v = 0.5$ (each vector has 50% sensitivity), the ensemble sensitivity is $1 - 0.5^{10} = 99.9\%$. This bound is highly favorable, but it depends critically on the independence assumption. Correlated attack vectors — e.g., `NO_ROLLBACK` and `NO_TEST_VALUE` both triggered by proposals that omit validation evidence — provide less additional information than independent vectors. The current vector taxonomy was designed to minimize correlation by targeting orthogonal aspects of proposal quality (rollback, testing, architecture, dependencies, security, scope, metrics), but empirical correlation analysis over production logs is needed to validate this design intent.

8.13.2 Colony-Level Intelligence and Modularity

Research on organizational modularity (Simon 1962) and parallel distributed processing (Rumelhart and McClelland 1986) both suggest that decomposing complex tasks into independent modules improves collective performance. The Colony Kernel's eight subsystems are designed with strict independence: no subsystem imports from another, and all communication goes through the shared `models.py` contract. This modularity provides a precisely analogous benefit: each subsystem can be independently validated, replaced, or upgraded without cascading effects on others — a property that supports the "harder to fool" claim by ensuring that adversarial attacks on one subsystem cannot trivially compromise the gate logic in another.

8.14 Zero-Knowledge Proofs and Secure Agent Authentication

The secure cognitive layer integrates zero-knowledge proof (ZKP) authentication into the colony's wallet operations, enabling agents to prove ownership of resources without exposing private keys. The `ZKProofVerifier` implements a challenge-response protocol: the prover generates a proof bound to a message (a challenge derived from the wallet address and a nonce), and the verifier checks the proof without ever accessing the private key material. This approach follows the Fiat-Shamir heuristic (Fiat and Shamir 1986) in spirit, though the current implementation uses HMAC-based challenge-response rather than full non-interactive ZK-SNARKs.

The `SignedCapabilityProofBuilder` integrates ZK-proofs with the identity module, producing signed capability proofs that bind wallet ownership to identity verification. This enables the colony to verify that an agent both controls a wallet and holds a verified identity persona before granting elevated trust — a two-factor authentication mechanism for agent actuation.

Scope limitation. The current ZK-proof implementation uses symmetric-key cryptography (HMAC-SHA256) rather than public-key ZK-SNARKs. This is sufficient for proving wallet ownership within the colony's trust boundary but does not provide the non-interactivity or succinctness properties of full ZK-SNARK systems (Ben-Sasson et al. 2014). Upgrading to a ZK-SNARK backend is an identified future direction contingent on performance requirements.

8.15 Biocognitive Verification and Physiological Trust Signals

The identity module implements biocognitive verification hooks that enable the colony to incorporate physiological signals into its trust scoring. The `HeartbeatValidator` enrolled-mode verifier computes heart rate variability (HRV) metrics — RMSSD (root mean square of

successive differences), SDNN (standard deviation of NN intervals), and mean heart rate — and compares them against a stored baseline using a configurable tolerance. A heartbeat that deviates beyond the tolerance threshold (e.g., a doubled heart rate suggesting stress or coercion) is rejected, preventing actuation by an agent whose physiological state suggests compromised judgment.

The `EEGFrequencyAnalyzer` computes power spectral density across standard frequency bands (delta, theta, alpha, beta, gamma) using fast Fourier transform, enabling frequency-band-specific trust adjustments. The `PersonaRotation` system allows agents to maintain multiple verified identity personas and switch between them, enabling role separation without re-enrollment.

Relationship to active inference. The biocognitive verification layer connects to the active inference framework (Section 9) by providing sensory evidence about the agent's internal state. Heart rate variability and EEG band power are physiological correlates of cognitive load and attention — quantities that the free-energy principle predicts should correlate with precision-weighting of prediction errors. An agent under high cognitive load (low HRV, elevated beta power) may be less reliable in its proposals, suggesting the gate should weight its completeness score more heavily. This connection is speculative and not yet implemented in the gate scoring; it represents a direction for integrating physiological evidence into the colony's trust dynamics.

9 Active Inference and the Free Energy Principle

The active inference framework, originating with Friston's Free Energy Principle (Friston 2010), provides a principled account of how biological and artificial agents can coordinate perception, learning, and action under uncertainty by minimizing variational free energy — the gap between an agent's generative model and its sensory experience of the world. This section develops a complete active inference treatment of the Colony Kernel, making precise the structural correspondences the introduction gestures at, establishing where formal equivalence holds and where the analogy breaks down, and identifying the architectural consequence of treating the Colony Kernel as an approximate active inference implementation.

The treatment proceeds in four stages. First, we define the generative model that an active-inference-theoretic Colony Kernel would maintain. Second, we map the six signal types and the decay dynamics to their free energy analogues. Third, we analyze the ActuationGate as a policy-selection mechanism under expected free energy minimization. Fourth, we identify the divergences between the current implementation and a full Bayesian brain and assess their practical implications.

9.1 Generative Model of the Colony

9.1.1 Observations, Hidden States, and the Generative Density

In the active inference framework, an agent maintains a generative model $p(o, s, \phi)$ over observations o, hidden environmental states s, and model parameters ϕ. The agent never accesses s directly; it only observes o and inverts the generative model to form beliefs about s.

For the Colony Kernel, we identify these components as follows:

Observations $o \in \mathcal{O}$: The observations available to the colony at each tick are the pheromone signals sensed from the `TraceField`. Formally, the observation at time t is defined in Equation 51:

$$o_t = \{f_t(l, k) : (l, k) \in \mathcal{L} \times \mathcal{K}\} \tag{51}$$

This is the colony's perceptual stream — a snapshot of accumulated consequences at every location, mediated by the decay dynamics.

Hidden states $s \in \mathcal{S}$: The hidden states are the true behavioral qualities of agents and the true risk profiles of locations — quantities that are never directly observable but must be inferred from outcomes. Concretely:

$$s = \{q_a : a \in \mathcal{A}\} \cup \{\rho_l : l \in \mathcal{L}\} \tag{52}$$

where $q_a \in [0, 1]$ is the true (latent) competence of agent a, and $\rho_l \in [0, 1]$ is the true (latent) risk level of location l. The hidden state space Equation 52 captures the latent variables the colony must infer.

Generative likelihood $p(o \mid s)$: The generative likelihood models how true risk and competence produce observable pheromone signals. Under the exponential decay model:

$$p(f_t(l, k) \mid \rho_l, q_a) = \text{Poisson}\left(\frac{\sigma_0 \cdot \rho_l^{1[k=\text{RISK}]} \cdot (1 - q_a)^{1[k=\text{FAILURE}]}}{1 - e^{-\lambda(k)}}\right) \tag{53}$$

This is a schematic Poisson observation model: the rate parameter scales with the true risk at the location for RISK signals and the true incompetence $(1 - q_a)$ for FAILURE signals. More complex observation models are possible; the key point is that the likelihood model ties observations to latent states.

Prior $p(s)$: The prior on agent competence and location risk encodes the colony's beliefs before any evidence. The conservative design prior — new agents start at trust $\tau_0 = 0.10$, below the gate floor — corresponds to a prior with mass concentrated at low competence:

$$p(q_a) = \text{Beta}(\alpha_0, \beta_0) \tag{54}$$

with $\alpha_0 \ll \beta_0$ (e.g., $\alpha_0 = 1, \beta_0 = 9$), placing most prior probability below 0.20.

9.1.2 Variational Free Energy and the Recognition Model

An exact Bayesian inference over s given o would require computing the posterior:

$$p(s \mid o) = \frac{p(o \mid s)p(s)}{p(o)} \tag{55}$$

which is generally intractable due to the normalization constant $p(o) = \int p(o \mid s)p(s)ds$. Active inference replaces exact Bayesian inference with a variational approximation: the agent maintains a *recognition model* $q(s; \phi)$ (parameterized by ϕ) and minimizes the variational free energy:

$$\mathcal{F}[q] = \underbrace{\mathbb{E}_{q(s)}[-\log p(o, s)]}_{\text{energy}} + \underbrace{\mathbb{E}_{q(s)}[\log q(s)]}_{\text{entropy}} = D_{\text{KL}}[q(s) \| p(s)] - \mathbb{E}_{q(s)}[\log p(o \mid s)] \tag{56}$$

The second form decomposes free energy into the KL divergence from the prior (a complexity penalty) and the expected log-likelihood under beliefs (an accuracy term). Minimizing $\mathcal{F}$ over q is equivalent to maximizing the evidence lower bound (ELBO) (Jordan and Sejnowski 1999).

Correspondence to Trust Update. The trust score τ_n is the colony's recognition model for agent competence: $q(q_a; \tau_n) \approx \text{Beta}(\tau_n \cdot \beta_0, (1 - \tau_n) \cdot \beta_0)$. The trust update rule Equation 24 is an online gradient step on the free energy: each new observation (outcome) shifts τ in the direction that decreases the KL divergence between the recognition model and the posterior implied by that outcome. The fixed step sizes (+0.04 for success, -0.08 for failure) correspond to a constant learning rate — a standard practical choice for online variational inference when the posterior is expected to be non-stationary (Amari 1998).

9.2 Pheromone Signals as Prediction Errors

A central construct in predictive coding — the cortical implementation of active inference — is the *prediction error*: the signed difference between a predicted signal and the observed signal at that level of the processing hierarchy. Prediction errors drive belief updates; large prediction errors indicate that the current model is misspecified for the observed region.

Correspondence. In the Colony Kernel, the pheromone field serves as a distributed store of accumulated prediction errors:

- FAILURE signals at a location correspond to large positive prediction errors: the colony's generative model predicted success (the action should not harm the repository) but the observation was failure (repair was required). FAILURE pheromone strength at $(l, \text{FAILURE})$ is proportional to the accumulated unresolved prediction error at location l.
- SUCCESS signals correspond to confirmed predictions: the colony predicted success and observed success. The slow decay of SUCCESS signals (SLOW decay class, half-life ~34 ticks) reflects the property that confirmed predictions accumulate as evidence for the current model's adequacy at that location.
- RISK signals correspond to *precision*-weighted prediction errors: they do not encode a binary pass/fail outcome but a continuous assessment of model uncertainty. High RISK pheromone indicates that the colony is uncertain about the region — the generative model has high variance — and the gate applies a precision-weighted penalty (`risk_ok` reduction) accordingly.

Formal Correspondence. Let $\varepsilon_t(l) = o_t(l, \text{FAILURE}) / \max_l o_t(l, \text{FAILURE})$ be the normalized prediction error at location l at time t. The gate component `risk_ok` can be written as a step function of the RISK pheromone strength, which itself approximates a low-pass-filtered version of $\varepsilon_t(l)$. The three-tier risk mapping (0.0, 0.5, 1.0 for high/medium/low pressure) is a coarse quantization of this continuous prediction-error signal into a discrete gate penalty (Friston et al. 2017).

Hierarchy of Precision. In the full free-energy framework, different signals have different precisions — inverse variances — that weight their contribution to belief updates. The trust multiplier in the deposit formula (Equation 1) directly instantiates this: HUMAN sources receive precision multiplier 2.0, TEST sources 1.5, SECURITY sources 1.3, and AGENT/RUNTIME sources 1.0. A human-injected signal carries twice the precision of an agent-injected signal, which means the colony updates its beliefs about a location twice as strongly in response to human observations as to agent observations. This is precisely the correct behavior under active inference: the hierarchy of sources reflects a hierarchy of precision estimates, and high-precision observations drive larger belief updates.

9.3 Gate as Expected Free Energy Minimization

9.3.1 Expected Free Energy

In active inference, action selection is governed by minimizing *expected* free energy $\tilde{\mathcal{F}}[\pi]$ over policies π (action sequences), rather than by maximizing a separate reward signal. The expected free energy decomposes into an epistemic component (information gain) and an extrinsic component (goal-directedness):

$$\tilde{\mathcal{F}}[\pi] = \underbrace{-\mathbb{E}_{q(o,s|\pi)}[\log p(o \mid \phi_\pi)]}_{\text{expected accuracy}} + \underbrace{D_{\text{KL}}[q(s \mid \pi) \| p(s)]}_{\text{expected complexity}} \tag{57}$$

Equivalently:

$$\tilde{\mathcal{F}}[\pi] = \underbrace{-\mathbb{E}_{q(o|\pi)} D_{\text{KL}}[q(s \mid o, \pi) \| q(s \mid \pi)]}_{\text{intrinsic (epistemic) value}} + \underbrace{D_{\text{KL}}[q(o \mid \pi) \| p^*(o)]}_{\text{extrinsic cost}} \tag{58}$$

where $p^*(o)$ is the preferred outcome distribution (the colony's "goal").

Correspondence to Gate Components. We map each gate component to a term in the expected free energy decomposition:

Table 35: Active inference correspondence for ActuationGate components.

Gate Component	Expected Free Energy Term	Interpretation
`risk_ok`	Extrinsic cost: $D_{\text{KL}}[q(o) \| p^*(o)]$ at location l	High RISK pheromone $\rightarrow$ high extrinsic cost of acting at l
`trust_ok`	Expected accuracy: $-\mathbb{E}[\log p(o \mid q_a)]$	High trust $\rightarrow$ high expected accuracy of agent a
`completeness`	Epistemic value: $\mathbb{E}[D_{\text{KL}}[q(s \mid o) \| q(s)]]$	Complete proposals generate more informative observations
`budget_ok`	Hard constraint on policy feasibility	Budget is a physical constraint on the policy space

EXECUTE as EFE Minimization. The gate issues EXECUTE when $g \geq 0.75$, which in the free energy framework corresponds to selecting the policy with expected free energy below a threshold. The weighted additive gate score is a linear approximation to the EFE under the assumption that the four components contribute independently to the total expected free energy:

$$\tilde{\mathcal{F}}[\pi_P] \approx 1 - g(P) = 1 - (0.30 \cdot \text{budget} + 0.30 \cdot \text{risk} + 0.25 \cdot \text{trust} + 0.15 \cdot \text{compl.}) \tag{59}$$

The mapping $g \mapsto 1 - \tilde{\mathcal{F}}$ means that high gate scores correspond to low expected free energy, and the EXECUTE threshold $g \geq 0.75$ corresponds to an EFE threshold of $\tilde{\mathcal{F}} \leq 0.25$.

HOLD as Epistemic Action. The HOLD pathway is the active inference analog of an *epistemic action* — an action taken not for its extrinsic value but to reduce uncertainty before committing to an intrinsically valuable action. In active inference, agents sometimes prefer to pause and gather information rather than act immediately, trading extrinsic reward for reduced future uncertainty. The HOLD decision tells the agent: "return with more evidence (completeness, rollback, test coverage) before we commit to this action." The agent's revision response generates new observations (evidence that the rollback plan is sound, that tests pass locally) that reduce the colony's uncertainty about the action's safety. The HOLD-then-revise cycle is therefore a literal instantiation of an epistemic action loop.

9.4 Active Inference Interpretation of Stigmergy

9.4.1 Shared Generative Models Through Environmental Embedding

A distinctive feature of the active inference framework is that it does not require agents to share an explicit model — they can coordinate through shared environmental states if their individual generative models predict the same environmental signals. This property, called *epistemic foraging* in the presence of shared priors, emerges naturally when agents share the same likelihood function $p(o \mid s)$ (Ramstead et al. 2019).

The pheromone field is a shared sufficient statistic. In the Colony Kernel, all agents access the same `TraceField`. The observations available to any new agent proposing at location l are precisely the cumulative consequence history of all past agents at l, compressed into the six pheromone signals via the deposit-and-decay dynamics. The colony thereby implements a form of *collective belief propagation*: each agent's past actions deposit observations into the shared field, and each future agent's free energy minimization benefits from those accumulated observations without requiring direct agent-to-agent communication.

Formally, if agent a_1 at time t_1 deposits FAILURE signal f_1 at $(l, \text{FAILURE})$ and agent a_2 at time $t_2 > t_1$ queries the same location, a_2 observes $f_t(l, \text{FAILURE}) = f_1 \cdot e^{-\lambda_F(t_2 - t_1)}$. Agent a_2's recognition model $q(q_{a_2}; \tau_2)$ must account for this decayed failure signal through the gate's `risk_ok` component — effectively incorporating a_1's past prediction error into a_2's current policy selection. This is stigmergic belief propagation: a_1's surprise becomes a_2's prior.

9.4.2 Markov Blanket of the Colony

The Markov blanket (Pearl 1988; Friston 2013) is a boundary that separates an agent from its environment: it consists of sensory states (observations), active states (actions), internal states (beliefs), and external states (environment). The Markov blanket conditions an agent's future actions on its past observations and beliefs, shielding it from direct causal influence by external states.

The ActuationGate as Markov Blanket Enforcer. The ActuationGate enforces the colony's Markov blanket: it is the interface between an agent's proposed action (active state) and the actual repository modification (external state). Only proposals that pass the gate — i.e., that satisfy the colony's beliefs about acceptable actions given accumulated observations — are allowed to cross the blanket and affect external states. The gate thereby ensures that external-state transitions are always mediated by the colony's internal model, not bypassed by raw capability.

Notably, the `SANDBOX` hard override implements a *strict Markov blanket*: new agents have no active states at all — they cannot affect external states — until they have accumulated sufficient observations (three proposals) to justify crossing the blanket. This is a formal embodiment of the principle that agents should earn actuation authority through evidence rather than capability.

9.5 Divergences from Canonical Active Inference

9.5.1 Where the Analogy Holds

The Colony Kernel shares five structural features with canonical active inference:

1. Prediction-error-driven belief updates: Trust deltas are signed prediction errors (success reduces surprise; failure increases it), and the trust update is an online belief update toward the posterior implied by the new outcome.
2. Precision-weighted observations: Trust multipliers implement a hierarchy of observation precisions, weighting high-trust sources (HUMAN, TEST) more strongly than low-trust sources (AGENT, RUNTIME).
3. Epistemic and pragmatic value: The three-way gate separates clearly epistemic actions (HOLD: gather more evidence) from pragmatic actions (EXECUTE: modify the repository).
4. Environmental embedding of collective beliefs: The pheromone field functions as a shared generative model embedded in the environment, enabling belief propagation across agents without direct communication — the colony-level Markov blanket.
5. Policy selection under uncertainty: The gate score is an approximation to expected free energy, and the EXECUTE decision selects the policy with lowest expected free energy.

9.5.2 Where the Analogy Breaks Down

The Colony Kernel departs from canonical active inference in four important ways:

1. Non-Bayesian trust update. The trust delta is a fixed-step-size stochastic approximation, not a Bayesian update of a Beta posterior. A true Bayesian agent would update $p(q_a) = \text{Beta}(\alpha, \beta)$ by incrementing α on success and β on failure, with the posterior mean $\alpha/(\alpha+\beta)$ converging to the agent's true success rate at the rate $1/n$. The fixed-step-size rule is faster to compute and simpler to audit but sacrifices asymptotic convergence to the true posterior. The practical consequence is that the trust score does not fully account for sample size: an agent with 100 interactions is treated the same as an agent with 10 interactions if both have the same trust score, whereas a Bayesian posterior would correctly assign much higher certainty to the agent with 100 interactions.

2. Discrete thresholds, not continuous posteriors. The gate maps trust scores to three discrete values (0.0, 0.5, 1.0) rather than treating trust as a continuous input. In canonical active inference, belief states are probability distributions; their continuous values propagate through the EFE calculation without discretization. The three-tier mapping is a practical engineering choice that reduces the gate to a simple lookup table, but it introduces discontinuities: an agent at trust 0.59 is treated identically to one at trust 0.35, and an agent at trust 0.60 receives a 2x higher `trust_ok` contribution than one at trust 0.59.

3. Missing temporal depth. The gate evaluates each proposal independently, conditioning only on current pheromone pressures and the current trust score. Full active inference agents plan over *sequences* of actions, considering the effect of the current action on future belief

states and future expected free energy. The Colony Kernel's gate is myopic: it has no mechanism to reason about how the current HOLD or EXECUTE decision will change the pheromone field on subsequent ticks, or to select actions that are suboptimal now but will position the colony to better distinguish safe from unsafe locations in the future.

4. No model evidence accumulation at the colony level. Canonical active inference agents update their model parameters ϕ as well as their recognition model $q(s;\phi)$ when persistent model misfit is detected (a process called *hyperparameter learning* or *structure learning*). The Colony Kernel does not update the gate weights (0.30, 0.30, 0.25, 0.15) or the decision thresholds (0.75, 0.50) based on accumulated consequence history. If the current thresholds are mis-calibrated — e.g., if the weight on trust should be 0.40 rather than 0.25 for a particular deployment — the system has no mechanism to discover this from data. Calibrating gate weights against production outcome logs is the model-parameter learning step that is explicitly deferred in this version.

9.6 Active Inference as an Upgrade Path

The divergences identified above are not flaws of the current implementation — they are deliberate design choices that prioritize auditability, determinism, and mathematical simplicity over statistical optimality. The value of the active inference framing is that it makes the upgrade path explicit: each divergence identifies a specific component of the full Bayesian treatment that could be added without changing the system's architecture.

Upgrade 1: Bayesian Trust Model. Replace the fixed-step trust update with a Beta posterior: maintain (α_a, β_a) per agent and update $\alpha_a \mathrel{+}= 1$ on success, $\beta_a \mathrel{+}= 1$ on failure. The trust score for the gate becomes the posterior mean $\alpha_a/(\alpha_a + \beta_a)$, and the uncertainty (posterior variance) can be used to modulate the gate's precision weighting for that agent. This upgrade retains the same interface but provides asymptotically optimal inference.

Upgrade 2: Continuous Gate Score. Replace the three-tier trust and risk mappings with linear interpolation: `trust_ok = trust_score` (continuous), `risk_ok = max(0, 1 - risk_pressure / risk_max)` (continuous). This removes the discontinuities at 0.30, 0.60 trust and at 3.0, 6.0 risk pressure, producing a smooth gate-score surface that is better calibrated for the decision boundary.

Upgrade 3: Planning Horizon Expansion. Extend the gate to condition on expected future pheromone states as well as the current state. A one-step rollout would compute $\tilde{f}_{t+1} = T_\lambda(f_t + D_{\text{proposal}})$ and re-evaluate `risk_ok` against the projected future field, allowing the gate to block actions that would elevate future risk pressure even if current pressure is low.

Upgrade 4: Model Parameter Learning. Add an offline calibration step that fits the gate weights (w_B, w_R, w_T, w_C) by logistic regression on accumulated consequence logs, using the binary EXECUTE/not outcome as the dependent variable. This is the analog of hyperparameter learning in active inference: the generative model's parameters are updated to better explain the observed data, making the gate progressively better calibrated as the consequence log grows.

These upgrades are additive and independent: they can be introduced one at a time without changing the gate's interface, the pheromone field's semantics, or the trust score's interpretation. The colony-as-active-inference-agent is a tractable upgrade roadmap, not merely a metaphor.

9.7 Summary

This section has established that the Colony Kernel shares five structural properties with canonical active inference: prediction-error-driven trust updates, precision-weighted observations, epistemic and pragmatic action separation, environmental embedding of collective beliefs, and EFE-approximating policy selection. It has identified four divergences — non-Bayesian trust update, discrete thresholds, myopic planning, and no model parameter learning — and shown that each divergence can be remedied by a specific, additive upgrade that retains the current system's architecture.

The active inference framing serves a practical purpose beyond conceptual elegance: it grounds the Colony Kernel's design choices in a well-developed theoretical tradition with known convergence properties, information-theoretic interpretations, and connections to both neuroscience and machine learning. The "harder to fool" falsification criterion has a precise active inference interpretation: a colony that gets harder to fool is one whose recognition model $q(s)$ more accurately tracks the true risk profile of the environment over time, reducing free energy at the colony level by accumulating a more accurate posterior over agent competences and location risk profiles.

10 Appendix: Design Rationale

This appendix documents the explicit design decisions behind the Colony Kernel's architecture, presenting the alternatives considered, the trade-offs evaluated, and the reasoning that led to the chosen designs. Unlike the methodology section, which describes what the system does, this appendix explains why the system was built the way it was — providing the context needed to assess whether the design choices are appropriate for a given deployment context and to understand what would need to change for a different set of requirements.

The rationale is organized around the seven most consequential design decisions in the Colony Kernel. Each subsection identifies the design decision, the alternatives, the criteria used to evaluate them, and the explicit reasoning behind the choice made.

10.1 DR-1: Weighted Additive Gate Score vs. Multiplicative or Learned Scoring

10.1.1 The Decision

The ActuationGate uses a weighted additive score (Equation 60):

$$g = 0.30b + 0.30r + 0.25t + 0.15c \tag{60}$$

rather than a multiplicative score ($g = b^{w_1} \cdot r^{w_2} \cdot t^{w_3} \cdot c^{w_4}$) or a learned discriminative classifier.

10.1.2 Alternatives Considered

Multiplicative (product) score. A multiplicative gate would have the property that any single zero component drives the total score to zero, creating a stricter unanimity requirement: every dimension must be positive for EXECUTE. This corresponds to a logical AND on the components rather than a weighted sum. The argument for this approach is that risk should be non-negotiable: an agent with zero budget clearance, zero risk clearance, zero trust, or zero completeness should never be approved, regardless of performance on other dimensions.

Learned classifier. A gradient-boosted tree or logistic regression trained on accumulated consequence logs would automatically discover the optimal combination of inputs, capturing non-linear interactions that the additive formula cannot represent.

Threshold-based rule system. A set of explicit IF-THEN rules with named conditions, similar to a firewall ruleset, would be maximally interpretable but inflexible.

10.1.3 Trade-off Analysis

Table 36: Trade-off summary for gate score formula approaches.

Approach	Auditability	Recovery support	Zero-shot validity	Calibration cost
Weighted additive	High — formula is explicit	High — partial credit enables HOLD	High — valid before any data	Low — four constants
Multiplicative	Medium — interactions are implicit	Low — any zero kills the score	High	Low
Learned classifier	Low — black box	Depends on model	Low — requires training data	High
Rule system	Very high	Low — binary outcomes	High	Low

10.1.4 Reasoning

The additive formula was chosen for three reasons (Table 36):

First, auditability. The formula is a single line that any operator can verify by inspection. Each weight directly states the importance assigned to each dimension. When an agent receives a HOLD or REFUSE decision, the gate result includes the component scores, making the decision fully explainable in terms the agent and human operators can act on. A learned classifier provides no such decomposition.

Second, recovery support. The additive formula provides partial credit: a proposal that scores well on three of four dimensions receives a meaningful score that may reach HOLD, giving the agent a recovery path. A multiplicative formula would refuse any proposal with a weak dimension — even if that weakness is correctable — eliminating the HOLD pathway as a practical recovery mechanism.

Third, validity before data. The additive weights (0.30, 0.30, 0.25, 0.15) were designed by reasoning about the relative importance of the four dimensions, not by fitting to production data. This is a necessary choice for a system that must be deployed before any production data exists. A learned classifier cannot be trained without data; the additive formula provides a functional gate from the first deployment. The weights The weights can be recalibrated against production consequence logs once sufficient data accumulates (see Section 10.9.1 below).

Why not multiplicative. The multiplicative approach was rejected because it conflates CRITICAL falsification findings are handled as explicit hard overrides *before* the scoring formula runs. Within the scoring formula, the remaining inputs are all *partially recoverable*: risk pheromone decays, completeness can be added, trust can be earned. For these recoverable dimensions, partial credit (additive) is more appropriate than a zero-kill property (multiplicative), which would prevent recovery.

10.2 DR-2: Exponential Decay vs. Linear or Step-Function Pheromone Decay

10.2.1 The Decision

Pheromone strength follows exponential decay $s(t) = s_0 \cdot e^{-\lambda t}$, with three discrete decay rate classes (FAST, NORMAL, SLOW) applying different λ values.

10.2.2 Alternatives Considered

Linear decay. $s(t) = \max(0, s_0 - \lambda t)$. Simpler to compute; reaches zero in finite time.

Step-function (TTL) decay. Pheromone strength is constant for d ticks, then drops to zero. Analogous to a time-to-live mechanism.

No decay (persistent memory). Pheromone signals are permanent until explicitly removed. Provides maximum memory but no automatic forgetting.

10.2.3 Trade-off Analysis

Table 37: Trade-off summary for pheromone decay model approaches.

Approach	Memory properties	Computational cost	Calibration complexity
Exponential	Asymptotic approach to zero; bounded steady state	O(n) per tick	One parameter per class
Linear	Finite lifetime; hard zero after $t = s_0/\lambda$	O(n) per tick	One parameter per class
Step TTL	Hard cutoff; predictable memory window	O(n) per tick	One parameter per class
Persistent	Indefinite; requires explicit cleanup	O(1) per tick (no decay)	N/A

10.2.4 Reasoning

Exponential decay mirrors biological systems and has well-understood mathematical properties (Table 37). The exponential model $s(t) = s_0 e^{-\lambda t}$ is the continuous-time limit of a discrete reinforcement process (Dorigo and Stützle 2004a). Its mathematical properties — Markov dynamics, closed-form steady-state distribution, tractable convergence analysis — make it the natural choice for a system that needs to be analyzed theoretically. Section 3.1 proves boundedness and convergence properties that depend on the exponential form; these results would not transfer to linear or step-function decay.

Exponential decay never reaches hard zero. This is a feature, not a limitation: a location with a single FAILURE signal five ticks ago should retain a small residual signal ($e^{-1.5} \approx 0.22$ at FAST decay rate), not be treated identically to a location with no history. Hard zero (linear or step) would create discontinuities at the lifetime boundary, causing gate score discontinuities that could be exploited by timing attacks.

Three-class discretization of decay rates. The continuous decay rate could be configured per-signal or per-location. The three-class design (FAST, NORMAL, SLOW) was chosen because it maps directly to the semantic categories of signals: urgent transient warnings (FAST), standard coordination signals (NORMAL), and persistent structural memory (SLOW). Adding more classes would reduce the semantic clarity of the categorization without improving the system's behavioral properties.

10.3 DR-3: SQLite Consequence Memory vs. In-Memory or Remote Database

10.3.1 The Decision

Consequence records and trust profiles are stored in a SQLite database in WAL mode.

10.3.2 Alternatives Considered

In-memory dictionary. Simple Python dict; zero latency; no persistence across sessions.

PostgreSQL/remote RDBMS. Full ACID, concurrent multi-writer support; requires infrastructure.

Append-only log file. Simple, inspectable; no query support.

Redis/key-value store. Fast reads/writes; requires infrastructure; no SQL queries.

10.3.3 Reasoning

SQLite was chosen for correctness-over-convenience in the single-process case. The Colony Kernel is explicitly scoped to single-repository, single-process deployment in this version (Section 8.2). Within that constraint, SQLite WAL mode provides:

- Persistence across sessions: trust scores and consequence history survive process restarts, model swaps, and agent population changes. This is the key property that distinguishes the colony from session-scoped frameworks.
- Concurrent readers without write contention: WAL mode allows multiple MCP client reads to proceed concurrently while a write is in progress, which matters for the read-heavy pattern of `colony_status` and `colony_pheromone_query` calls.
- Zero infrastructure: SQLite requires no additional services, credentials, or network configuration. A developer can deploy the Colony Kernel with a single `pip install` and a single YAML file.
- Inspectable with standard tools: The consequence store can be opened with any SQLite browser for audit or debugging, without bespoke tooling.

In-memory was rejected because it makes the "harder to fool" property conditional on session continuity. A location that was dangerous last session becomes a fresh location in the next session; the entire consequence memory is erased on process exit. This defeats the architecture's core premise.

PostgreSQL was rejected for the same reason as distributed state: it is out of scope for the current single-repository model and introduces infrastructure dependencies that increase deployment friction without providing benefits at single-process scale.

10.4 DR-4: Deterministic Heuristic Trust Deltas vs. Learned Trust Update

10.4.1 The Decision

Trust deltas are fixed heuristic constants: +0.04 for passing outcomes, -0.08 for failing, -0.05 for repair-needed, ±0.03 weighted by human feedback.

10.4.2 Alternatives Considered

Q-learning update. $\tau_{n+1} = \tau_n + \alpha(r_n - \tau_n)$, where r_n is the reward signal for outcome n and α is a learning rate.

Bayesian Beta update. Maintain Beta(α, β) and update by incrementing α on success and β on failure; posterior mean = $\alpha/(\alpha + \beta)$.

Multi-factor learned model. A linear or non-linear model mapping outcome features (test coverage, repair time, human rating) to trust updates, trained on production logs.

10.4.3 Reasoning

Heuristic constants are appropriate for a system that must be valid before any training data exists. The trust delta constants encode a small number of explicit design commitments: failures should cost more than successes gain (asymmetry encodes risk-aversion); human feedback should amplify or dampen the outcome signal; repair events should carry additional negative weight beyond the pass/fail signal.

These commitments can be stated and argued for on first principles (Section 3.3), without requiring data to fit. The $2:1$ asymmetry between failure penalty (0.08) and success bonus (0.04) reflects the well-established principle in risk management that the cost of a bad outcome typically exceeds the benefit of an equivalent good outcome (Tversky and Kahneman 1991).

The Bayesian Beta update is the principled alternative and is explicitly identified as Upgrade 1 in the active inference upgrade path (Section 9.6). The reason it is deferred rather than adopted immediately is not theoretical — the Beta update is clearly superior asymptotically — but practical: the fixed-step rule is simpler to audit and explain to operators. An operator can verify that an agent's trust score changes by exactly +0.04 per success and -0.08 per failure; verifying that the Beta posterior update is correctly implemented requires understanding conjugate priors. The fixed-step rule prioritizes interpretability over statistical optimality, consistent with the system's overall design philosophy of putting auditability first.

10.5 DR-5: Pre-Actuation Falsification vs. Post-Actuation Analysis

10.5.1 The Decision

The FalsificationWorker applies adversarial checks *before* the ActuationGate scores the proposal (pre-actuation), not after the action executes (post-actuation analysis).

10.5.2 Alternatives Considered

Post-actuation analysis. Run the 10 adversarial checks on the executed outcome rather than the plan. This would allow checks to inspect actual test results, actual import graphs, and actual behavior rather than plans.

Sampling-based pre-actuation. Run the adversarial checks on a random subset of proposals, not all of them. Lower latency; some proposals proceed unchecked.

LLM-backed adversarial review. Use a language model to generate novel adversarial checks rather than applying fixed heuristics.

10.5.3 Reasoning

Pre-actuation falsification is load-bearing for the gate's completeness component. The ActuationGate uses the FalsificationWorker's findings as one of its four inputs (the `completeness` component), and the hard override on CRITICAL findings requires the worker to run *before* the gate scores the proposal. Moving falsification to post-actuation would decouple it from the gate decision, converting a prevention mechanism into a detection mechanism — and detection is strictly weaker than prevention for irreversible actions.

The deterministic heuristic design avoids LLM dependency. The 10 attack vectors are deterministic heuristics that require no LLM call. This design choice has two consequences: zero incremental latency for the LLM inference path, and zero risk of the falsification check itself being subject to adversarial steering through prompt injection. An LLM-backed adversarial reviewer could be manipulated by carefully crafted proposals; a deterministic heuristic cannot be manipulated through its input.

LLM-backed adversarial review is explicitly deferred. The conclusion (Section 5) identifies "extending the falsification worker with LLM-backed adversarial probing" as a future direction, but the current design deliberately avoids this to maintain the falsification path's security properties. The 10-vector heuristic approach provides a known, bounded, auditable check set; an LLM-based approach provides a larger but unbounded and potentially unreliable check set.

Sampling-based approaches were rejected because they create a predictable bypass: an adversary that knows proposals are checked only $p\%$ of the time can submit enough proposals to have a statistical expectation of one unchecked acceptance per $1/p$ proposals. The current system applies all 10 checks to every proposal, providing a consistent deterrent without timing-based bypass opportunities.

10.6 DR-6: Five-Role Ladder vs. Continuous Privilege Score

10.6.1 The Decision

Agent authority is structured as a five-role discrete ladder (SANDBOX, REPAIR_ANT, MEMORY_ANT, DISPATCHER, GUARD_ANT) with explicit trust threshold crossings, rather than a continuous privilege score that scales smoothly with trust.

10.6.2 Alternatives Considered

Continuous privilege score. The agent's trust score directly determines its privilege level on a continuous scale, without discrete role boundaries.

Two-role model. SANDBOX (new/untrusted) and ACTIVE (trusted), with a single promotion threshold.

Capability-based authorization. Each action type has its own trust requirement, independent of a global role assignment.

10.6.3 Reasoning

Discrete roles are auditable; continuous privilege is not. A five-role system provides human operators with named categories that correspond to meaningful behavioral constraints: a REPAIR_ANT is allowed to patch and fix, a DISPATCHER can route and coordinate, a GUARD_ANT can audit and veto. These category names are meaningful to human reviewers. A continuous privilege score provides no natural decomposition point for human review: an agent at 0.67 has neither a clear name nor a clear set of permitted actions.

Role boundaries are explicit falsification targets. The role thresholds (0.20, 0.35, 0.50, 0.70) are stated in the manuscript as formal criterion F3 (Section 5.1). A continuous privilege system has no equivalent falsification criterion; any deviation from expected behavior can always be attributed to the smoothness of the privilege function. Discrete thresholds make the system's promises verifiable.

SANDBOX is the critical hard boundary. The two-tier model (SANDBOX/ACTIVE) was rejected because the promotion from SANDBOX to any active role is the most consequential transition in the system — the point at which an agent gains any ability to affect external state. Within the active tier, the four-level ladder provides the graduated authority that reflects the colony's trust-proportional privilege philosophy. Collapsing the four active levels into one ACTIVE role would sacrifice this graduated structure.

The ladder maps to natural responsibility categories. REPAIR_ANT (fix and patch), MEMORY_ANT (archive and index), DISPATCHER (coordinate and route), and GUARD_ANT (audit and veto) correspond to distinct types of agents that different teams may need to deploy. The discrete categories make it straightforward to describe role requirements in natural language to operators, in contrast to a continuous scale where "agent with privilege 0.67" is not self-describing.

10.7 DR-7: O(n) Stigmergy vs. O(n^2) Agent-to-Agent Communication

10.7.1 The Decision

Agents coordinate through the shared pheromone field (stigmergy), not through direct agent-to-agent communication.

10.7.2 Alternatives Considered

Direct agent-to-agent messaging. Agents broadcast outcomes and risk assessments directly to all other agents, building peer knowledge of each other's histories.

Central reputation server. A dedicated reputation service maintains agent scores and distributes them on demand, with agents querying the server before submitting proposals.

Gossip protocol. Agents exchange reputation updates with randomly selected peers, eventually achieving consistency across the population.

10.7.3 Reasoning

Stigmergy scales as O(n), direct messaging as O(n^2). As argued in Section 1 and formalized in Section 3.1, the pheromone field compresses n agents' historical signals into a single queryable surface. Each agent reads the surface and writes to the surface; no agent needs to maintain a model of any other agent's state. The O(n^2) scaling of direct messaging is not just a theoretical concern: at machine speed with thousands of agent proposals per second, the message volume from all-to-all communication becomes the dominant cost.

Stigmergy does not require agent identity. Direct messaging requires agents to identify each other (who is sending this warning?) and establish trust channels (should I believe this agent?). Stigmergy bypasses both: the pheromone field encodes accumulated colony history regardless of which agents contributed. A new agent can benefit from the failure history deposited by previous agents at a location without needing to know those agents existed, identify them, or trust their direct reports. The field is the message, and the field's source multipliers (based on depositing-agent trust at deposit time) already incorporate source credibility into the deposited strength.

Stigmergy survives agent population churn. If agents are replaced, retired, or compromised, their historical contributions are already encoded in the pheromone field. Direct messaging systems must handle agent departure, message expiration, and reputation revocation explicitly. The field persists regardless of which agents are currently active.

The central reputation server was rejected because it is a single point of failure and a high-value attack target. An adversary that compromises the reputation server can manipulate trust scores for all agents simultaneously. The pheromone field is decentralized: no single component stores all trust information, and the trust multipliers applied at deposit time prevent a compromised source from retroactively inflating old signals.

10.8 DR-8: Human Confirmation Required for Pruning

10.8.1 The Decision

The PruningDaemon identifies stale modules and raises `PruningCandidate` reports but never archives automatically. Human confirmation is required for all pruning actions.

10.8.2 Alternatives Considered

Automatic pruning above confidence threshold. Candidates with confidence ≥ 0.90 (never-used-since-registration) are automatically archived without human review.

Auto-prune with 48-hour notification window. Candidates are flagged, and pruning proceeds automatically after 48 hours unless a human explicitly cancels.

No pruning mechanism. The colony accumulates modules indefinitely without systematic thinning.

10.8.3 Reasoning

Irreversibility asymmetry. Automatically archiving a module that turns out to be needed is much more costly than failing to archive a module that is genuinely stale. The cost of a false positive (pruning a needed module) includes unexpected failures, emergency restoration from backup, and potential data loss. The cost of a false negative (retaining a stale module) is wasted storage and a slightly cluttered registry. Under this asymmetry, the correct policy is to require human confirmation — accepting higher false-negative rates to minimize false positives.

The pheromone veto prevents the most dangerous false positives. Before classifying any module, the daemon checks whether a DEPENDENCY signal of strength ≥ 2.0 exists at that location. A module that is actively used will receive DEPENDENCY deposits from live `record_outcome` calls and will self-protect from archival without manual intervention. The human confirmation requirement applies

only to modules that have both low pheromone strength *and* low usage statistics — the intersection of two independent signals, reducing the false-positive risk considerably.

Future auto-prune threshold. The architecture explicitly anticipates a future extension: "Human confirmation is required until a sufficient empirical record justifies a configurable auto-prune threshold." This is a deliberate deferral rather than a permanent constraint. Once the system has been operated long enough to characterize the false-positive rate of the confidence tiers against real-world module usage patterns, an operator could reasonably set a configurable auto-prune threshold — but only for candidates whose confidence exceeds that threshold *and* whose pheromone veto is absent. This two-factor criterion is both defensible and auditable.

10.9 Summary: Core Design Philosophy

10.9.1 Note on Gate Weight Calibration

The gate weights (0.30, 0.30, 0.25, 0.15) were set by design reasoning rather than fitted to production data. Once sufficient consequence logs accumulate (on the order of several thousand gate evaluations with known outcomes), logistic regression or a gradient-boosted tree can be trained to predict EXECUTE vs. HOLD/REFUSE from the four component scores, and the resulting coefficients can replace the hand-crafted weights. This calibration step is intentionally deferred: the hand-crafted weights provide a reasonable prior distribution over gate decisions during the cold-start period when no production data exists, and they serve as the baseline against which the calibrated weights can be compared.

The eight design rationale sections above reveal a consistent underlying philosophy:

Auditability over optimality. Every place where a statistically optimal approach (Bayesian trust update, learned gate weights, LLM-backed falsification) was available but not chosen, the reason was that the simpler, more auditable approach was preferred. The consequence records, gate decisions, trust scores, and role assignments are all fully explainable from first principles without model introspection.

Prevention over detection. Pre-actuation falsification, hard overrides, and SANDBOX isolation all implement prevention: they stop bad actions before they execute. Detection (post-actuation analysis, retrospective auditing) is valuable but secondary. The Colony Kernel is designed to prevent damage, not to diagnose it after the fact.

Conservative defaults with explicit recovery paths. Every design choice that makes the system more restrictive (asymmetric trust deltas, discrete trust tiers, human-confirmed pruning, pre-actuation falsification) is paired with an explicit recovery path: trust can be earned back, proposals can be revised and resubmitted, roles can be promoted, pruning candidates can be reviewed and overridden. The system is restrictive but not punitive.

Explicit scope over feature completeness. The Colony Kernel is deliberately scoped to the single-repository, single-process deployment model. Distributed state, learned policies, LLM-backed checks, and real-time DP noise are all identified as future directions rather than current features. This scope discipline makes the current implementation tractable to audit and validate.

These design choices are not permanent constraints. They are the minimum necessary structure to provide the ecology thesis's core guarantees — that the colony gets harder to fool over time — in a form that is auditable, deployable, and falsifiable. Each rationale section above identifies the concrete extensions that would be appropriate to add once the current foundation has been validated against production data.

References

AI, LangChain. 2024. *LangGraph: Build Stateful, Multi-Actor Applications with LLMs*. Https://github.com/langchain-ai/langgraph.

Alshiekh, Mohammed, Roderick Bloem, Ruediger Ehlers, Bettina Konighofer, Scott Niekum, and Ufuk Topcu. 2018. "Safe Reinforcement Learning via Shielding." *Proceedings of the AAAI Conference on Artificial Intelligence* 32. https://doi.org/10.1609/aaai.v32i1.11797.

Amari, Shun-ichi. 1998. "Natural Gradient Works Efficiently in Learning." *Neural Computation* 10 (2): 251–76.

Amodei, Dario, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. "Concrete Problems in AI Safety." *arXiv Preprint arXiv:1606.06565*. https://arxiv.org/abs/1606.06565.

Andriushchenko, Maksym, Alexandra Souly, Mateusz Dziemian, et al. 2025. "AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents." *International Conference on Learning Representations*. https://arxiv.org/abs/2410.09024.

Anthropic. 2024. *Model Context Protocol: An Open Standard for Connecting AI Assistants to the Systems Where Data Lives*. Anthropic. https://modelcontextprotocol.io.

Apt, Krzysztof R. 2003. *Principles of Constraint Programming*. Cambridge University Press.

Arnold, Matthew, Rachel K. E. Bellamy, Michael Hind, et al. 2019. "FactSheets: Increasing Trust in AI Services Through Supplier's Declarations of Conformity." *IBM Journal of Research and Development* 63 (4/5): 6:1–13. https://doi.org/10.1147/JRD.2019.2942288.

Autio, Chloe, Reva Schwartz, Jesse Dunietz, et al. 2024. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.AI.600-1.

Bai, Yuntao, Andy Jones, Kamal Ndousse, et al. 2022. "Constitutional AI: Harmlessness from AI Feedback." *arXiv Preprint arXiv:2212.08073*. https://arxiv.org/abs/2212.08073.

Ben-Sasson, Eli, Alessandro Chiesa, Eran Tromer, and Madars Virza. 2014. "Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture." *IACR Cryptology ePrint Archive* 2014: 705.

Bonabeau, Eric, Marco Dorigo, and Guy Theraulaz. 1999. *Swarm Intelligence: From Natural to Artificial Systems*. Oxford University Press.

Bonawitz, Keith, Vladimir Ivanov, Ben Kreuter, et al. 2017. "Practical Secure Aggregation for Privacy-Preserving Machine Learning." *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 1175–91. https://doi.org/10.1145/3133956.3133982.

Buhl, Marie Davidsen, Gaurav Sett, Leonie Koessler, Jonas Schuett, and Markus Anderljung. 2024. "Safety Cases for Frontier AI." *arXiv Preprint arXiv:2410.21572*. https://arxiv.org/abs/2410.21572.

Burnett, Colin, Timothy J. Norman, and Katia Sycara. 2013. "Bootstrapping Trust Evaluations Through Stereotypes." *Proceedings of the 12th International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)* (St. Paul, MN), 437–44.

Chan, Alan, Carson Ezell, Max Kaufmann, et al. 2024. "Visibility into AI Agents." *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*. https://doi.org/10.1145/3630106.3658948.

Chan, Alan, Kevin Wei, Sihao Huang, et al. 2025. "Infrastructure for AI Agents." *arXiv Preprint arXiv:2501.10114*. https://arxiv.org/abs/2501.10114.

Christiano, Paul F., Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. "Deep Reinforcement Learning from Human Preferences." *Advances in Neural Information Processing Systems (NeurIPS)* 30. https://arxiv.org/abs/1706.03741.

Condorcet, Nicolas de. 1785. *Essai Sur l'application de l'analyse*. Imprimerie Royale.

Dash, Pritam, Tongyu Ge, Aditi Jain, Tanmay Shah, and Zhiwei Shang. 2026. "From Untrusted Input to Trusted Memory: A Systematic Study of Memory Poisoning Attacks in LLM Agents." *arXiv Preprint arXiv:2606.04329*. https://arxiv.org/abs/2606.04329.

Debenedetti, Edoardo, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. 2024. "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents." *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. https://arxiv.org/abs/2406.13352.

Dorigo, Marco, and Thomas Stützle. 2004b. *Ant Colony Optimization*. MIT Press.

Dorigo, Marco, and Thomas Stützle. 2004a. *Ant Colony Optimization*. MIT Press.

Dwork, Cynthia, and Aaron Roth. 2014. *The Algorithmic Foundations of Differential Privacy*. Vol. 9. Now Publishers.

Endsley, Mica R. 2017. "From Here to Autonomy: Lessons Learned from Human-Automation Research." *Human Factors* 59 (1): 5–27. https://doi.org/10.1177/0018720816681350.

Fang, Richard, Rohan Bindu, Akul Gupta, and Daniel Kang. 2024. "LLM Agents Can Autonomously Exploit One-Day Vulnerabilities." *arXiv Preprint arXiv:2404.08144*. https://arxiv.org/abs/2404.08144.

Fiat, Amos, and Adi Shamir. 1986. "How to Prove Yourself: Practical Solutions to Identification and Signature Problems." *Advances in Cryptology — CRYPTO '86*, 186–94.

Friston, Karl. 2010. "The Free-Energy Principle: A Unified Brain Theory?" *Nature Reviews Neuroscience* 11 (2): 127–38. https://doi.org/10.1038/nrn2787.

Friston, Karl. 2013. *Life as We Know It*. MIT Press.

Friston, Karl, Thomas FitzGerald, Francesco Rigoli, and Giovanni Pezzulo. 2017. *Active Inference: A Process Theory*. MIT Press.

Fu, Yuchuan, Xiaohan Yuan, and Dongxia Wang. 2025. "RAS-Eval: A Comprehensive Benchmark for Security Evaluation of LLM Agents in Real-World Environments." *arXiv Preprint arXiv:2506.15253*. https://arxiv.org/abs/2506.15253.

Garg, Spandan, Benjamin Steenhoek, and Yufan Huang. 2025. “Saving SWE-Bench: A Benchmark Mutation Approach for Realistic Agent Evaluation.” *arXiv Preprint arXiv:2510.08996*. https://arxiv.org/abs/2510.08996.

Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, et al. 2021. “Datasheets for Datasets.” *Communications of the ACM* 64 (12): 86–92. https://doi.org/10.1145/3458723.

Grassé, Pierre-Paul. 1959. “La Reconstruction Du Nid Et Les Coordinations Inter-Individuelles Chez Bellicositermes natalensis Et Cubitermes Sp. La Théorie de La Stigmergie: Essai d’interprétation Du Comportement Des Termites Constructeurs.” *Insectes Sociaux* 6 (1): 41–80. https://doi.org/10.1007/BF02223791.

Greenblatt, Ryan, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. 2023. “AI Control: Improving Safety Despite Intentional Subversion.” *arXiv Preprint arXiv:2312.06942*. https://arxiv.org/abs/2312.06942.

Greshake, Kai, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. “Not What You’ve Signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection.” *arXiv Preprint arXiv:2302.12173*. https://arxiv.org/abs/2302.12173.

Hawkins, Richard, Colin Paterson, Chiara Picardi, Yan Jia, Radu Calinescu, and Ibrahim Habli. 2021. “Guidance on the Assurance of Machine Learning in Autonomous Systems (AMLAS).” *arXiv Preprint arXiv:2102.01564*. https://arxiv.org/abs/2102.01564.

Hong, Sirui, Mingchen Zhuge, Jiaqi Chen, et al. 2023. “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework.” *arXiv Preprint arXiv:2308.00352*. https://arxiv.org/abs/2308.00352.

Huynh, Trung Dong, Nicholas R. Jennings, and Nigel Shadbolt. 2006. “An Integrated Trust and Reputation Model for Open Multi-Agent Systems.” *Autonomous Agents and Multi-Agent Systems* 13 (2): 119–54. https://doi.org/10.1007/s10458-005-6825-4.

Inc., CrewAI. 2024. *CrewAI: Framework for Orchestrating Role-Playing, Autonomous AI Agents*. Https://github.com/crewAIInc/crewAI.

Jordan, Michael I., and Terrence J. Sejnowski. 1999. *Graphical Models: Foundations of Neural Computation*. MIT Press.

Kamvar, Sepandar D., Mario T. Schlosser, and Hector Garcia-Molina. 2003. “The EigenTrust Algorithm for Reputation Management in P2P Networks.” *Proceedings of the 12th International Conference on World Wide Web*, 640–51. https://doi.org/10.1145/775152.775242.

Karpas, Ehud, Omri Abend, Yonatan Belinkov, et al. 2022. “MRKL Systems: A Modular, Neuro-Symbolic Architecture That Combines Large Language Models, External Knowledge Sources and Discrete Reasoning.” *arXiv Preprint arXiv:2205.00445*. https://arxiv.org/abs/2205.00445.

Kauffman, Stuart A. 1993. *The Origins of Order: Self-Organization and Selection in Evolution*. Oxford University Press.

Klimesch, Wolfgang. 1999. “EEG Alpha and Theta Oscillations Reflect Cognitive and Memory Performance: A Review and Analysis.” *Brain Research Reviews* 29 (2–3): 169–95.

Kolt, Noam. 2025. “Governing AI Agents.” *arXiv Preprint arXiv:2501.07913*. https://arxiv.org/abs/2501.07913.

Kushner, Harold J., and G. George Yin. 2003. *Stochastic Approximation and Recursive Algorithms and Applications*. 2nd ed. Springer.

Lee, John D., and Katrina A. See. 2004. “Trust in Automation: Designing for Appropriate Reliance.” *Human Factors* 46 (1): 50–80. https://doi.org/10.1518/hfes.46.1.50_30392.

Létac, Gérard. 1986. *Systèmes de Particules Et Phénomènes de Stigmergie*. Presses Universitaires de Toulouse.

Levy, Ido, Ben Wiesel, Sami Marreed, et al. 2024. “ST-WebAgentBench: A Benchmark for Evaluating Safety and Trustworthiness in Web Agents.” *arXiv Preprint arXiv:2410.06703*. https://arxiv.org/abs/2410.06703.

Li, Guohao, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. “CAMEL: Communicative Agents for ‘Mind’ Exploration of Large Language Model Society.” *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2303.17760.

Li, Minghao, Yingxiu Zhao, Bowen Yu, et al. 2023. “API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs.” *arXiv Preprint arXiv:2304.08244*. https://arxiv.org/abs/2304.08244.

Lightman, Hunter, Vineet Kosaraju, Yura Burda, et al. 2023. “Let’s Verify Step by Step.” *arXiv Preprint arXiv:2305.20050*. https://arxiv.org/abs/2305.20050.

Liu, Xiao, Hao Yu, Hanchen Zhang, et al. 2023. "AgentBench: Evaluating LLMs as Agents." *arXiv Preprint arXiv:2308.03688*. https://arxiv.org/abs/2308.03688.

Luo, Hanjun, Shenyu Dai, Chiming Ni, et al. 2025. "AgentAuditor: Human-Level Safety and Security Evaluation for LLM Agents." *arXiv Preprint arXiv:2506.00641*. https://arxiv.org/abs/2506.00641.

Lupinacci, Matteo, Francesco Aurelio Pironti, Francesco Blefari, Francesco Romeo, Luigi Arena, and Angelo Furfaro. 2025. "The Dark Side of LLMs: Agent-Based Attack Vectors for System-Level Compromise." *arXiv Preprint arXiv:2507.06850*. https://arxiv.org/abs/2507.06850.

Marsh, Stephen Paul. 1994. "Formalising Trust as a Computational Concept." PhD thesis, University of Stirling.

Mehmood, Usama, Sanaz Sheikhi, Stanley Bak, Scott A. Smolka, and Scott D. Stoller. 2021. "The Black-Box Simplex Architecture for Runtime Assurance of Autonomous CPS." *arXiv Preprint arXiv:2102.12981*. https://arxiv.org/abs/2102.12981.

Mialon, Gregoire, Clementine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. "GAIA: A Benchmark for General AI Assistants." *arXiv Preprint arXiv:2311.12983*. https://arxiv.org/abs/2311.12983.

Miller, Mark S., and Jonathan S. Shapiro. 2003. *Paradigm Regained: Abstraction Mechanisms for Access Control*. Johns Hopkins University.

Mitchell, Margaret, Simone Wu, Andrew Zaldivar, et al. 2019. "Model Cards for Model Reporting." *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220–29. https://doi.org/10.1145/3287560.3287596.

MITRE Corporation. 2026a. *MITRE ATLAS*. Https://atlas.mitre.org/.

MITRE Corporation. 2026b. *MITRE ATT&CK*. Https://attack.mitre.org/.

Mohassel, Payman, and Yupeng Zhang. 2017. "SecureML: A System for Scalable Privacy-Preserving Machine Learning." *2017 IEEE Symposium on Security and Privacy*, 19–38. https://doi.org/10.1109/SP.2017.12.

Motwani, Sumeet Ramesh, Mikhail Baranchuk, Martin Strohmeier, et al. 2024. "Secret Collusion Among AI Agents: Multi-Agent Deception via Steganography." *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2402.07510.

Myerson, Roger B. 1979. "Incentive Compatibility and the Bargaining Problem." *Econometrica* 47 (1): 61–73.

Nakano, Reiichiro, Jacob Hilton, Suchir Balaji, et al. 2021. "WebGPT: Browser-Assisted Question-Answering with Human Feedback." *arXiv Preprint arXiv:2112.09332*. https://arxiv.org/abs/2112.09332.

Near, Joseph P., and Chiké Abuah. 2021. *Programming Differential Privacy*. Vol. 1. https://programming-dp.com/.

Newman, Zachary, John Speed Meyers, and Santiago Torres-Arias. 2022. "Sigstore: Software Signing for Everybody." *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2353–67. https://doi.org/10.1145/3548606.3560596.

NIST National Cybersecurity Center of Excellence. 2026. *Accelerating the Adoption of Software and AI Agent Identity and Authorization*. National Institute of Standards; Technology. https://www.nccoe.nist.gov/projects/software-and-ai-agent-identity-and-authorization.

Open Source Security Foundation. 2026. *Supply-chain Levels for Software Artifacts (SLSA) Specification Version 1.2*. Https://slsa.dev/spec/v1.2/.

OWASP Foundation. 2025. *OWASP Top 10 for Large Language Model Applications*. Https://owasp.org/www-project-top-10-for-large-language-model-applications/.

OWASP Foundation. 2026a. *AI Agent Security Cheat Sheet*. Https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.h

OWASP Foundation. 2026b. *OWASP Top 10 for Agentic Applications for 2026*. Https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/.

Packer, Charles, Sarah Wooders, Kevin Lin, et al. 2023. "MemGPT: Towards LLMs as Operating Systems." *arXiv Preprint arXiv:2310.08560*. https://arxiv.org/abs/2310.08560.

Parasuraman, Raja, Thomas B. Sheridan, and Christopher D. Wickens. 2000. "A Model for Types and Levels of Human Interaction with Automation." *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 30 (3): 286–97. https://doi.org/10.1109/3468.844354.

Park, Joon Sung, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. "Generative Agents: Interactive Simulacra of Human Behavior." *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. https://doi.org/10.1145/3586183.3606763.

Parunak, H. Van Dyke. 1997. ""Go to the Ant": Engineering Principles from Natural Multi-Agent Systems." *Annals of Operations Research* 75: 69–101. https://doi.org/10.1023/A:1018980001403.

Patil, Shishir G., Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. "Gorilla: Large Language Model Connected with Massive APIs." *arXiv Preprint arXiv:2305.15334*. https://arxiv.org/abs/2305.15334.

Pearl, Judea. 1988. *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann.

Peng, Roger D. 2011. "Reproducible Research in Computational Science." *Science* 334 (6060): 1226–27. https://doi.org/10.1126/science.1213847.

Phan, Dung, Junxing Yang, Matthew Clark, et al. 2017. "A Component-Based Simplex Architecture for High-Assurance Cyber-Physical Systems." *2017 17th International Conference on Application of Concurrency to System Design*, 49–58. https://doi.org/10.1109/ACSD.2017.23.

Qian, Chen, Wei Liu, Hongzhang Liu, et al. 2024. "ChatDev: Communicative Agents for Software Development." *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. https://arxiv.org/abs/2307.07924.

Qin, Yujia, Shihao Liang, Yining Ye, et al. 2023. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs." *arXiv Preprint arXiv:2307.16789*. https://arxiv.org/abs/2307.16789.

Raji, Inioluwa Deborah, Andrew Smart, Rebecca N. White, et al. 2020. "Closing the AI Accountability Gap: Defining an End-to-End Framework for Internal Algorithmic Auditing." *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 33–44. https://doi.org/10.1145/3351095.3372873.

Ramstead, Maxwell J. D., Axel Constant, Paul B. Badcock, and Karl J. Friston. 2019. "Variational Ecology and the Physics of Sentient Systems." *Physics of Life Reviews* 31: 188–205.

Reisman, Dillon, Jason Schultz, Kate Crawford, and Meredith Whittaker. 2018. *Algorithmic Impact Assessments: A Practical Framework for Public Agency Accountability*. AI Now Institute. https://ainowinstitute.org/publications/algorithmic-impact-assessments-report-2.

Robbins, Herbert, and Sutton Monro. 1951. "A Stochastic Approximation Method." *The Annals of Mathematical Statistics* 22 (3): 400–407.

Rose, Scott, Oliver Borchert, Stu Mitchell, and Sean Connelly. 2020. *Zero Trust Architecture*. NIST Special Publication Nos. 800-207. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.SP.800-207.

Roughgarden, Tim. 2016. *CS269I: Incentives in Computer Science*. Stanford University course notes. https://timroughgarden.org/f16/f16.html.

Ruan, Yangjun, Honghua Dong, Andrew Wang, et al. 2023. "Identifying the Risks of LM Agents with an LM-Emulated Sandbox." *arXiv Preprint arXiv:2309.15817*. https://arxiv.org/abs/2309.15817.

Rudin, Walter. 1991. *Functional Analysis*. 2nd ed. McGraw-Hill.

Rumelhart, David E., and James L. McClelland. 1986. *Parallel Distributed Processing*. MIT Press.

Sabater, Jordi, and Carles Sierra. 2005. "Review on Computational Trust and Reputation Models." *Artificial Intelligence Review* 24 (1): 33–60. https://doi.org/10.1007/s10462-004-0595-7.

Saltzer, Jerome H., and Michael D. Schroeder. 1975. "The Protection of Information in Computer Systems." *Proceedings of the IEEE* 63 (9): 1278–308. https://doi.org/10.1109/PROC.1975.9939.

Samuel, Justin, Nick Mathewson, Justin Cappos, and Roger Dingledine. 2010. "Survivable Key Compromise in Software Update Systems." *Proceedings of the 17th ACM Conference on Computer and Communications Security*. https://doi.org/10.1145/1866307.1866315.

Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, et al. 2023. "Toolformer: Language Models Can Teach Themselves to Use Tools." *arXiv Preprint arXiv:2302.04761*. https://arxiv.org/abs/2302.04761.

Seto, David, Bruce Krogh, Lui Sha, and Alongkrit Chutinan. 1998. "The Simplex Architecture for Safe Online Control System Upgrades."

Proceedings of the 1998 American Control Conference, 3504–8. https://doi.org/10.1109/ACC.1998.703255.

Shaffer, Fred, and J. P. Ginsberg. 2017. "An Overview of Heart Rate Variability Metrics and Norms." *Frontiers in Public Health* 5: 258.

Shao, Yijia, Tianshi Li, Weiyan Shi, Yanchen Liu, and Diyi Yang. 2024. "PrivacyLens: Evaluating Privacy Norm Awareness of Language Models in Action." *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. https://arxiv.org/abs/2409.00138.

Shavit, Yonadav, Sandhini Agarwal, Miles Brundage, et al. 2023. *Practices for Governing Agentic AI Systems*. OpenAI. https://cdn.openai.com/papers/practices-for-governing-agentic-ai-systems.pdf.

Shinn, Noah, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. "Reflexion: Language Agents with Verbal Reinforcement Learning." *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2303.11366.

Simon, Herbert A. 1962. *The Architecture of Complexity*. In *Proceedings of the American Philosophical Society*, vol. 106.

Souppaya, Murugiah, Karen Scarfone, and Donna Dodson. 2022. *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. NIST Special Publication Nos. 800-218. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.SP.800-218.

Surowiecki, James. 2004. *The Wisdom of Crowds*. Anchor Books.

Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press.

Tabassi, Elham. 2023. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. National Institute of Standards; Technology. https://doi.org/10.6028/NIST.AI.100-1.

Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology. 1996. "Heart Rate Variability: Standards of Measurement, Physiological Interpretation, and Clinical Use." *Circulation* 93: 1043–65.

Torres-Arias, Santiago, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. "In-Toto: Providing Farm-to-Table Guarantees for Bits and Bytes." *28th USENIX Security Symposium*, 1393–410. https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

Trivedi, Harsh, Tushar Khot, Mareike Hartmann, et al. 2024. "AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents." *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. https://arxiv.org/abs/2407.18901.

Tversky, Amos, and Daniel Kahneman. 1991. "Loss Aversion in Riskless Choice." *The Quarterly Journal of Economics* 106 (4): 1039–61.

Uesato, Jonathan, Nate Kushman, Ramana Kumar, et al. 2022. "Solving Math Word Problems with Process- and Outcome-Based Feedback." *arXiv Preprint arXiv:2211.14275*. https://arxiv.org/abs/2211.14275.

Wan, Shengye, Cyrus Nikolaidis, Daniel Song, et al. 2024. "CYBERSECEVAL 3: Advancing the Evaluation of Cybersecurity Risks and Capabilities in Large Language Models." *arXiv Preprint arXiv:2408.01605*. https://arxiv.org/abs/2408.01605.

Wang, Lei, Chen Ma, Xueyang Feng, et al. 2023. "A Survey on Large Language Model Based Autonomous Agents." *arXiv Preprint arXiv:2308.11432*. https://arxiv.org/abs/2308.11432.

Wang, Xuezhi, Jason Wei, Dale Schuurmans, et al. 2023. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." *International Conference on Learning Representations*. https://arxiv.org/abs/2203.11171.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, et al. 2022. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *arXiv Preprint arXiv:2201.11903*. https://arxiv.org/abs/2201.11903.

Wooldridge, Michael, and Nicholas R. Jennings. 1995. "Intelligent Agents: Theory and Practice." *The Knowledge Engineering Review* 10 (2): 115–52. https://doi.org/10.1017/S0269888900008122.

Wu, Qingyun, Gagan Bansal, Jieyu Zhang, et al. 2023. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." *arXiv Preprint arXiv:2308.08155*. https://arxiv.org/abs/2308.08155.

Xi, Zhiheng, Wenxiang Chen, Xin Guo, et al. 2023. "The Rise and Potential of Large Language Model Based Agents: A Survey." *arXiv Preprint arXiv:2309.07864*. https://arxiv.org/abs/2309.07864.

Xia, Hongfei, Hongru Wang, Zeming Liu, Qian Yu, Yuhang Guo, and Haifeng Wang. 2025. "SafeToolBench: Pioneering a Prospective Benchmark to Evaluating Tool Utilization Safety in LLMs." *arXiv Preprint arXiv:2509.07315*. https://arxiv.org/abs/2509.07315.

Xie, Tianbao, Danyang Zhang, Jixuan Chen, et al. 2024. "OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments." *arXiv Preprint arXiv:2404.07972*. https://arxiv.org/abs/2404.07972.

Xu, Frank F., Yufan Song, Boxuan Li, et al. 2025. "TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks." *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*. https://arxiv.org/abs/2412.14161.

Yang, John, Carlos E. Jimenez, Alexander Wettig, et al. 2024a. "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering." *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2405.15793.

Yang, John, Carlos E. Jimenez, Alexander Wettig, et al. 2024b. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" *arXiv Preprint arXiv:2310.06770*. https://arxiv.org/abs/2310.06770.

Yao, Shunyu, Dian Yu, Jeffrey Zhao, et al. 2023. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2305.10601.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, et al. 2023. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629.

Zhan, Qiusi, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents." *arXiv Preprint arXiv:2403.02691*. https://arxiv.org/abs/2403.02691.

Zhang, Hanrong, Jingyuan Huang, Kai Mei, et al. 2024. "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-Based Agents." *arXiv Preprint arXiv:2410.02644*. https://arxiv.org/abs/2410.02644.

Zhang, Zhexin, Shiyao Cui, Yida Lu, et al. 2024. "Agent-SafetyBench: Evaluating the Safety of LLM Agents." *arXiv Preprint arXiv:2412.14470*. https://arxiv.org/abs/2412.14470.

Zhou, Shuyan, Frank F. Xu, Hao Zhu, et al. 2023. "WebArena: A Realistic Web Environment for Building Autonomous Agents." *arXiv Preprint arXiv:2307.13854*. https://arxiv.org/abs/2307.13854.

Zhu, Yuxuan, Antony Kellermann, Dylan Bowman, et al. 2025. "CVE-Bench: A Benchmark for AI Agents' Ability to Exploit Real-World Web Application Vulnerabilities." *arXiv Preprint arXiv:2503.17332*. https://arxiv.org/abs/2503.17332.

Zhu, Yuxuan, Antony Kellermann, Akul Gupta, et al. 2024. "Teams of LLM Agents Can Exploit Zero-Day Vulnerabilities." *arXiv Preprint arXiv:2406.01637*. https://arxiv.org/abs/2406.01637.

<!– New references added by expanded sections (02_theory.md, 08_active_inference.md, appendix_design_rationale.md, expanded 07_scope_and_related_work.md, expanded 04_conclusion.md, expanded 03_results.md).

These are formatted as pandoc-citeproc bibtex-compatible keys. The bibliography entries should be added to the project's .bib file (refs.bib or equivalent).

=== Mathematical Foundations (sec:theory) ===

Rudin (1991) Rudin, W. (1991). Functional Analysis (2nd ed.). McGraw-Hill.

Létac (1986) Letac, G. (1986). A contraction principle for certain Markov chains and its applications. Contemporary Mathematics, 50, 263–273.

Robbins and Monro (1951) Robbins, H., & Monro, S. (1951). A stochastic approximation method. Annals of Mathematical Statistics, 22(3), 400–407.

Kushner and Yin (2003) Kushner, H. J., & Yin, G. G. (2003). Stochastic Approximation and Recursive Algorithms and Applications (2nd ed.). Springer.

Dwork and Roth (2014) Dwork, C., & Roth, A. (2014). The Algorithmic Foundations of Differential Privacy. Foundations and Trends in Theoretical Computer Science, 9(3–4), 211–407.

Near and Abuah (2021) Near, J. P., & Abuah, C. (2021). Programming Differential Privacy. Book. https://programming-dp.com/

=== Active Inference (sec:active-inference) ===

Friston et al. (2017) Friston, K. J., Rosch, R., Parr, T., Price, C., & Bowman, H. (2017). Active inference: a process theory. Neural Computation, 29(1), 1–49.

Jordan and Sejnowski (1999) Jordan, M. I., Ghahramani, Z., Jaakkola, T. S., & Saul, L. K. (1999). An introduction to variational methods for graphical models. Machine Learning, 37(2), 183–233.

Amari (1998) Amari, S. I. (1998). Natural gradient works efficiently in learning. Neural Computation, 10(2), 251–276.

Ramstead et al. (2019) Ramstead, M. J. D., Badcock, P. B., & Friston, K. J. (2019). Answering Schrödinger's question: A free-energy formulation. Physics of Life Reviews, 24, 1–16.

Pearl (1988) Pearl, J. (1988). Probabilistic Reasoning in Intelligent Systems. Morgan Kaufmann.

Friston (2013) Friston, K. (2013). Life as we know it. Journal of the Royal Society Interface, 10(86), 20130475.

=== Information-Theoretic Security and Mechanism Design (sec:infosec-dp, sec:mechanism-design) ===

Bonawitz et al. (2017) Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H. B., Patel, S., … & Seth, K. (2017). Practical secure aggregation for privacy-preserving machine learning. CCS 2017, 1175–1191.

Mohassel and Zhang (2017) Mohassel, P., & Zhang, Y. (2017). SecureML: A system for scalable privacy-preserving machine learning. IEEE S&P 2017, 19–38.

Roughgarden (2016) Roughgarden, T. (2016). Twenty lectures on algorithmic game theory. Cambridge University Press.

Myerson (1979) Myerson, R. B. (1979). Incentive compatibility and the bargaining problem. Econometrica, 47(1), 61–73.

Surowiecki (2004) Surowiecki, J. (2004). The Wisdom of Crowds. Doubleday.

Condorcet (1785) Condorcet, M. J. A. N. de C. (1785). Essai sur l'application de l'analyse à la probabilité des décisions rendues à la pluralité des voix. Imprimerie Royale, Paris.

Simon (1962) Simon, H. A. (1962). The architecture of complexity. Proceedings of the American Philosophical Society, 106(6), 467–482.

Rumelhart and McClelland (1986) Rumelhart, D. E., McClelland, J. L., & the PDP Research Group (Eds.). (1986). Parallel Distributed Processing: Explorations in the Microstructure of Cognition. MIT Press.

=== Design Rationale (appendix_design_rationale.md) ===

Tversky and Kahneman (1991) Tversky, A., & Kahneman, D. (1991). Loss aversion in riskless choice: A reference- dependent model. Quarterly Journal of Economics, 106(4), 1039–1061.

=== Zero-Knowledge Proofs (sec:zk-proofs) ===

Fiat and Shamir (1986) Fiat, A., & Shamir, A. (1986). How to Prove Yourself: Practical Solutions to Identification and Signature Problems. Advances in Cryptology, CRYPTO 1986, 186-194.

Ben-Sasson et al. (2014) Ben-Sasson, E., Chiesa, A., Tromer, E., & Virza, M. (2014). Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture. IACR Cryptology ePrint Archive, 2014, 705.

=== Biocognitive Verification (sec:biocognitive) ===

Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology (1996) Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology. (1996). Heart Rate Variability: Standards of Measurement, Physiological Interpretation, and Clinical Use. Circulation, 93, 1043-1065.

Shaffer and Ginsberg (2017) Shaffer, F., & Ginsberg, J. P. (2017). An Overview of Heart Rate Variability Metrics and Norms. Frontiers in Public Health, 5, 258.

Klimesch (1999) Klimesch, W. (1999). EEG Alpha and Theta Oscillations Reflect Cognitive and Memory Performance: A Review and Analysis. Brain Research Reviews, 29(2-3), 169-195.